

Aula 07

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) Desenvolvimento Front-End - CSS - Teoria	3
2) Desenvolvimento Front-End - CSS - Questões Comentadas - MULTIBANCAS	110
3) Desenvolvimento Front-End - CSS - Lista de Questões - MULTIBANCAS	137
4) Desenvolvimento Front-End - Bootstrap - Teoria	156
5) Desenvolvimento Front-End - Bootstrap - Questões Comentadas - Multibancas	209
6) Desenvolvimento Front-End - Bootstrap - Lista de Questões - Multibancas	232

Sumário

Apresentação da Aula	4
CSS	5
Conceitos Básicos	5
Sintaxe CSS	8
Seletores de CSS	9
O seletor de elemento CSS	9
Todos os seletores simples de CSS	10
Como adicionar CSS	12
Comentários CSS	14
Cores CSS	15
Fundos CSS	17
Imagem de Fundo CSS	19
Propriedades de Borda do CSS	22
Largura da Borda do CSS	24
Cores de borda	26
Margens CSS	32
Preenchimento CSS	34
Altura, largura e largura máxima do CSS	36
Modelo de Caixa CSS	38
Contorno do CSS	38
Largura do Contorno do CSS	39
Cor do Contorno do CSS	39
Abreviação do Contorno do CSS	39
Texto CSS	41
Cor do texto	41
Alinhamento e direção do texto	42
Propriedades de Decoração de Texto	43
Transformação de Texto CSS	44
Espaçamento do Texto CSS	45
Sombra de Texto CSS	46
Fontes CSS	47
Estilo de fonte CSS	49
Tamanho da fonte	50
Regras de emparelhamento de fontes	53
Ícones de CSS	56
Fontes de Ícones Incríveis	56
Ícones do Bootstrap	57
Links CSS	58
Decoração de texto	59
Cor de fundo	59
Botões de link	59
Listas de CSS	60
Listas HTML e propriedades da lista CSS	1
Marcadores de itens de lista diferentes	1
Uma imagem como marcador de item de lista	1
Posicione os Marcadores de Item da Lista	2
Remover configurações padrão	2

Lista - Propriedade abreviada	2
Lista de estilos com cores	3
Tabelas CSS	4
Recolher bordas da tabela (border-collapse)	5
Tamanho da Tabela CSS	6
Alinhamento de Tabela CSS	7
Estilo de Tabela CSS	8
Tabela Responsiva CSS	9
Layout CSS - A propriedade de exibição	10
Elementos de nível de bloco	10
Elementos embutidos	10
Substituir o valor de exibição padrão	12
Largura e Largura Máxima	12
A propriedade de posição	13
position: static;	13
position: relative;	13
position: fixed;	13
position: absolute;	14
A propriedade z-index	16
Overflow	17
Layout CSS: Float and clear	19
A propriedade CSS "clear"	20
Layout CSS: display: inline-block	22
Alinhamento	24
Alinhamento horizontal:	24
Alinhamento vertical:	24
Elementos de Alinhamento Central	24
CSS Combinators	26
Pseudo-classes	27
Opacidade / Transparência	30
Pseudo-elementos	31
Todos os pseudoelementos CSS	32
Barra de Navegação	33
Barra de Navegação Vertical	34
Barra de Navegação Horizontal	35
Menus suspensos em CSS	36
Galeria de imagens	37
Sprites de imagem	38
Seletores de atributos CSS	39
Formulários CSS	40
Resumo	41
Sintaxe CSS	42
Comentário	42
Seletores de CSS	42
Todos os seletores simples de CSS	43
Cores CSS	43
Fundos CSS	44
Imagem de Fundo CSS	45

Propriedades de Borda do CSS	45
Margens CSS	46
Preenchimento CSS	46
Altura, largura e largura máxima do CSS	47
Modelo de Caixa CSS	47
Texto CSS	47
Fontes CSS	48
Ícones de CSS	48
Links CSS	49
Tabelas CSS	49
Layout CSS - A propriedade de exibição	49
A propriedade de posição	50
A propriedade z-index	51
Overflow	51
Seletores de atributos CSS	52
Referências	53

APRESENTAÇÃO DA AULA

Olá, galera! Vamos iniciar os estudos sobre o **CSS**!

Nesta aula, apresento inúmeras palavras-chave, para que vocês possam entender e internalizar o conteúdo de forma mais simples. Aproveitem o material, além dos esquemas, resumos e mnemônicos. Vamos ao que importa! 😊

CSS

Conceitos Básicos

Antes de iniciar a aula, é importante mencionar que vários exemplos dessa aula foram retirados ou inspirados em exemplos do W3Tutorials (www.w3schools.com/html). **Não fizemos isso porque somos preguiçosos, mas por dois motivos: (1) os exemplos são excelentes; (2) essa é uma das fontes de inspiração das bancas.** Além disso, eu sugiro que vocês tenham sempre aberta uma janela com um interpretador online para que vocês possam testar o que veremos. Recomendo esse:

<https://www.w3schools.com/html/>

```
<!DOCTYPE html>
<html>
<body>

<h1>Meu primeiro título.</h1>

<p>Meu primeiro parágrafo.</p>

</body>
</html>
```

Meu primeiro título.

Meu primeiro parágrafo.

CSS é a sigla para **Folhas de Estilo em Cascata**. É uma linguagem de estilos usada para descrever a apresentação de um documento escrito em uma linguagem de marcação. É mais comumente usado para estilizar documentos HTML e XML, mas pode ser usado com qualquer documento XML, incluindo SVG e XHTML. O CSS é uma tecnologia-chave usada para criar sites visualmente atraentes e consistentes. Ele é usado para controlar layout, cores, fontes e outros elementos visuais em um site.

CSS é a linguagem que usamos para estilizar um documento HTML e descreve como os elementos HTML devem ser exibidos.

Vejamos um exemplo de código CSS

```
body {
  background-color: lightblue;
}

h1 {
  color: white;
  text-align: center;
}

p {
  font-family: verdana;
```

```
font-size: 20px;  
}
```

O código é um exemplo de CSS. Ele define a cor de fundo do elemento "body" para azul claro, a cor do elemento "h1" para branco e alinha o texto do elemento "h1" ao centro. Também define a família da fonte e o tamanho da fonte do elemento "p" para Verdana e 20px, respectivamente.

Este CSS será aplicado às tags HTML correspondentes em uma página da Web, resultando em um plano de fundo de toda a página azul claro, quaisquer cabeçalhos de nível 1 sendo brancos e alinhados ao centro e quaisquer elementos "p" tendo uma família de fontes de Verdana e tamanho da fonte de 20px.

CSS (Cascading Style Sheets) é usado para controlar a aparência de elementos HTML e XML na página, incluindo cor, fontes, espaçamento, layout e muito mais. CSS permite que os desenvolvedores separem a apresentação do conteúdo, o que facilita a manutenção e o desenvolvimento de sites. Com o CSS, é possível aplicar estilos consistentes em todo o site, o que ajuda a garantir que a aparência do site seja consistente e profissional.

- CSS significa Cascading Style Sheets
- CSS descreve como os elementos HTML devem ser exibidos na tela, no papel ou em outras mídias
- CSS economiza muito trabalho. Ele pode controlar o layout de várias páginas da Web de uma só vez
- Folhas de estilo externas são armazenadas em arquivos CSS

O CSS é usado para definir estilos para suas páginas da Web, incluindo design, layout e variações de exibição para diferentes dispositivos e tamanhos de tela.

O HTML NUNCA foi destinado a conter tags para formatar uma página da web! HTML foi criado para descrever o conteúdo de uma página web, como:

```
<h1>Este é um cabeçalho</h1>  
  
<p>Isto é um parágrafo.</p>
```

Quando tags como e atributos de cor foram adicionados à especificação do HTML 3.2, começou um pesadelo para os desenvolvedores da web. O desenvolvimento de grandes sites, onde fontes e informações de cores foram adicionadas a cada página, tornou-se um processo longo e caro. Para resolver esse problema, o World Wide Web Consortium (W3C) criou o CSS.

CSS removeu a formatação de estilo da página HTML!

As definições de estilo são normalmente salvas em arquivos .css externos. Com um arquivo de folha de estilo externo, você pode alterar a aparência de um site inteiro alterando apenas um arquivo!

Sintaxe CSS

A sintaxe CSS consiste em um conjunto de regras que definem como aplicar estilos a elementos em um documento HTML. Essas regras são compostas de seletores, que especificam a quais elementos os estilos devem ser aplicados, e declarações, que definem os próprios estilos. O seletor é o elemento ou grupo de elementos aos quais os estilos serão aplicados. A propriedade é a propriedade CSS, como cor ou tamanho da fonte, que você deseja alterar. O valor é o valor para o qual você deseja definir a propriedade. Várias declarações podem ser adicionadas à mesma regra, separadas por ponto e vírgula:



- O seletor aponta para o elemento HTML que você deseja estilizar.
- O bloco de declaração contém uma ou mais declarações separadas por ponto e vírgula.
- Cada declaração inclui um nome de propriedade CSS e um valor, separados por dois pontos.
- Várias declarações CSS são separadas por ponto e vírgula e os blocos de declaração são cercados por chaves.

CSS também suporta o uso de **cascata e herança**, onde **estilos podem ser herdados** por elementos filhos de seus elementos pais.

O CSS pode ser aplicado de três maneiras:

- estilo inline, na tag HTML
- folha de estilo interna, dentro da tag head
- folha de estilo externa, em um arquivo separado com extensão .css

Você também pode usar seletores diferentes, como seletores de classe, id, atributo e pseudoclassee para direcionar elementos específicos na página.

Seletores de CSS

Os seletores CSS são usados para "encontrar" (ou selecionar) os elementos HTML que você deseja estilizar.

Podemos dividir os seletores CSS em cinco categorias:

- Seletores simples (selecionar elementos com base no nome, id, classe)
- Seletores de combinação (selecionam elementos com base em uma relação específica entre eles)
- Seletores de pseudoclasse (selecionar elementos com base em um determinado estado)
- Seletores de pseudoelementos (selecionar e estilizar uma parte de um elemento)
- Seletores de atributo (selecionar elementos com base em um atributo ou valor de atributo)

O seletor de elemento CSS

Os seletores são usados para selecionar os elementos HTML aos quais você deseja aplicar estilos. Existem vários tipos diferentes de seletores disponíveis em CSS, incluindo:

- **Seletores de elementos:** selecione elementos com base em seus nomes. Por exemplo, `p { }` seleciona todos os elementos `<p>` na página.
- **Seletores de classe:** selecione elementos com base em seu atributo de classe. Por exemplo, `.my-class { }` seleciona todos os elementos com a classe "my-class".
- **Seletores de ID:** selecione um elemento exclusivo com base em seu atributo id. Por exemplo, `#my-id { }` seleciona o elemento com o id "my-id".
- **Seletores de atributos:** selecione elementos com base em seus atributos ou valores de atributos. Por exemplo, `input[type="text"] { }` seleciona todos os elementos `<input>` com um atributo de tipo "text".
- **Seletores de pseudoclasse:** selecione elementos com base em seus estados ou comportamento dinâmico. Por exemplo, `a:hover { }` seleciona todos os elementos `<a>` quando o usuário passa o mouse sobre eles.
- **Combinando seletores:** selecione elementos com base em sua relação com outros elementos. Por exemplo, `#my-id .my-class p` seleciona todos os elementos `<p>` dentro de um elemento com a classe "my-class" que é descendente de um elemento com o id "my-id".

(FCC – TJ-CE – 2022) Para colocar a cor de fundo vermelha apenas dos campos (inputs) do tipo text de um formulário, utiliza-se a instrução CSS

- a) `input [type=text] {background-color: #00ff00}`
- b) `input.type [text] {background-color: #ff0000}`

- c) `input [type=text] {background-color: #ff0000}`
- d) `input.type!text {background: rgb(0,255,0)}`
- e) `input [type('text')] {background-color:rgb(255,0,0)}`

Comentários: Para isso podemos usar seletores de atributos: selecione elementos com base em seus atributos ou valores de atributos. Por exemplo, `input[type="text"] { }` seleciona todos os elementos `<input>` com um atributo de tipo "text". No caso da questão, devemos usar o `input [type=text] {background-color: #ff0000}`. (Gabarito: Letra C)

(FCC – MANAUSPREV – 2021) No formulário de contato de uma página web do site do Governo de XX, um desenvolvedor de software precisa especificar que todos os campos do tipo texto (text) devem ser mostrados com fonte azul. Para estabelecer este estilo usando a linguagem CSS, deve-se utilizar o comando

- a) `input [type=text] {color:blue}`
- b) `input type='text' {color:blue}`
- c) `input.text {color:blue}`
- d) `input!text {color:#0000ff}`
- e) `input#text {color:#blue}`

Comentários: Nessa questão também temos a aplicação de seletores! Nesse caso um desenvolvedor de software precisa especificar que todos os campos do tipo texto (text), ou seja, `input [type=text]`, devem ser mostrados com fonte azul... eis a questão... qual é o correto? Tanto `{color:blue}` quanto `{color:#0000ff}` estão corretos. O principal, nessa questão é saber a sintaxe correta: `input [type=text]`. Portanto, temos aí nosso gabarito! (Gabarito: Letra A)

Todos os seletores simples de CSS

Seletor	Exemplo	Descrição do exemplo
<code>#id</code>	<code>#firstname</code>	Seleciona o elemento com <code>id="firstname"</code>
<code>.class</code>	<code>.intro</code>	Seleciona todos os elementos com <code>class="intro"</code>
<code>element.class</code>	<code>p.intro</code>	Seleciona apenas elementos <code><p></code> com <code>class="intro"</code>
<code>*</code>	<code>*</code>	Seleciona todos os elementos
<code>element</code>	<code>p</code>	Seleciona todos os elementos <code><p></code>
<code>element,element,..</code>	<code>div, p</code>	Seleciona todos os elementos <code><div></code> e todos os elementos <code><p></code>

Como adicionar CSS

Existem **três** maneiras principais de incluir uma folha de estilo CSS em um documento HTML:

- **Estilos embutidos:** são estilos aplicados diretamente a um elemento HTML específico usando o atributo "estilo". Por exemplo, `<p style="color: red;">Este é um parágrafo vermelho.</p>`
- **Folha de estilo interna:** são estilos definidos na seção de cabeçalho de um documento HTML usando uma tag `<style>`. Por exemplo,

```
<head>
<style>
  p {
    color: red;
  }
</style>
</head>
```

- **Folha de estilo externa:** são estilos definidos em um arquivo .css separado e vinculados a um documento HTML usando uma tag `<link>` na seção head. Por exemplo,

```
<head>
<link rel="stylesheet" type="text/css" href="styles.css">
</head>
```

Neste exemplo, a tag `<link>` é usada para vincular uma folha de estilo externa chamada `styles.css` à página. O atributo `href` especifica o caminho para o arquivo CSS, enquanto o atributo `rel` indica que o link é para uma folha de estilo e o atributo `type` especifica o tipo de arquivo (CSS). Além disso, podemos usar a tag `<style>` no cabeçalho da página:

```
<head>
<style type="text/css"> /* ou <style rel="stylesheet" type="text/css"> */
  @import url("aleap.css");
</style>
</head>
```

Neste exemplo, a tag `<style>` é usada para incluir folhas de estilo internas na página. O atributo `type` especifica o tipo de conteúdo (CSS) e o conteúdo da tag `<style>` é o código CSS. A diretiva `@import` é usada para incluir uma folha de estilo externa chamada `aleap.css`.

O uso de uma folha de estilo externa é considerado uma **prática recomendada**, pois permite uma **melhor organização e capacidade de manutenção dos estilos** e facilita a aplicação dos mesmos estilos a várias páginas. Vejamos uma questão!

(FCC – AL-AP – 2020) Segundo os padrões de acessibilidade na web definidos para sites do Governo Eletrônico, códigos CSS devem estar sempre em arquivos externos a serem chamados

pelas páginas do sítio/portal. Por exemplo, a instrução `<link rel="stylesheet" type="text/css" href="aleap.css"/>` pode ser também escrita na forma:

- a) `<style rel="link" type="text/css"> @include url("aleap.css"); </style>`
- b) `<link lang="css" type="stylesheet" href="aleap.css"/>`
- c) `<style rel="stylesheet" type="text/css"> @import url("aleap.css"); </style>`
- d) `<link rel="external" type="text/css" src="aleap.css"/>`
- e) `<style rel="stylesheet" type="text/css" url("aleap.css")/>`

Comentários: Sabemos que há três formas de inserir CSS: Estilos embutidos, Folha de estilo interna e Folha de estilo externa. Ainda, o uso de uma folha de estilo externa é considerado uma prática recomendada, portanto, vamos lembrar como é inserir CSS usando tags! As tags são `<style>` e `<link>`. Já que a banca deu a `link` no enunciado, vamos procurar pela tag `Style`, certo? Das opções, a correta é semelhante ao nosso exemplo `<style type="text/css"> @import url("aleap.css"); </style>`. Além de utilizar a tag `<style>`, é necessário que o atributo `rel` seja configurado como `stylesheet` ao se referenciar uma folha de estilo, especialmente quando se usa a diretiva `@import` para indicar um arquivo CSS externo". Portanto, nosso gabarito é a letra C. (Gabarito: Letra C)

Comentários CSS

Comentários CSS são usados para adicionar notas ou explicações em uma folha de estilo CSS que não afeta a exibição da página da web. Os comentários são ignorados pelos navegadores e não são exibidos na página da web.

Comentários CSS são escritos usando os símbolos `/*` e `*/`. Qualquer coisa escrita entre esses símbolos é considerada um comentário e será ignorada pelo navegador.

Por exemplo:

```
/* Este é um comentário CSS */  
p{  
  cor vermelha; /* Isso também é um comentário */  
}
```

Os comentários CSS podem ser colocados em sua própria linha ou na mesma linha que uma regra CSS. Eles também [podem abranger várias linhas](#).

É uma boa prática usar comentários para [explicar o propósito de uma regra CSS](#) ou grupo de regras, ou para indicar que uma seção da folha de estilo ainda está em andamento ou precisa ser revisada. Isso pode ajudar outras pessoas que trabalham no projeto a entender o código e fazer alterações com mais facilidade.

Além disso, os comentários CSS podem ser usados para desativar temporariamente uma determinada regra ou seção da folha de estilo adicionando `/*` no início da linha com a regra e `*/` no final da linha, sem precisar removê-la completamente.

Cores CSS

As cores são especificadas usando nomes de cores predefinidos ou valores **RGB**, **HEX**, **HSL**, **RGBA**, **HSLA**. Em **CSS**, as cores podem ser especificadas usando vários métodos diferentes. Valor RGB: As cores podem ser especificadas usando o modelo de cores RGB (vermelho, verde, azul), onde cada cor é especificada por um valor entre **0** e **255**. Por exemplo, a cor vermelha pode ser especificada como **rgb(255, 0, 0)**.

- **Valor HEX:** As cores também podem ser especificadas usando um valor hexadecimal de 6 dígitos, onde cada par de dígitos representa os valores de vermelho, verde e azul. Por exemplo, a cor vermelha pode ser especificada como **#ff0000** (vermelho).
 - Branco = **"#FFFFFF"**
 - Vermelho = **"#FF0000"**
 - Verde = **"#00FF00"**
 - Azul = **"#0000FF"**
 - Preto = **"#000000"**
- **Valor HSL:** As cores também podem ser especificadas usando o modelo de cores HSL (matiz, saturação, luminosidade), onde o valor do matiz é um grau entre 0 e 360, a saturação e a luminosidade são porcentagens. Por exemplo, a cor vermelha pode ser especificada como **hsl(0,100%,50%)**.
- **Predefinido/Palavra-chave:** Há também um conjunto de nomes de cores predefinidos que podem ser usados, como "vermelho", "verde", "azul", "preto", "branco", etc.
- **Valor RGBA:** Semelhante ao valor RGB, mas permite definir um canal alfa para transparência.
- **Valor HSLA:** Semelhante ao valor HSL, mas permite definir um canal alfa para transparência.

É importante observar que nem todos os navegadores suportam valores HSL e HSLA. Por isso, é uma boa prática fornecer valores alternativos em formatos amplamente aceitos, como HEX ou RGB, para garantir a compatibilidade entre diferentes navegadores.

Às vezes, você pode encontrar códigos hexadecimais de 3 dígitos no código CSS. Eles são uma maneira abreviada de escrever códigos hexadecimais de 6 dígitos para cores específicas. As cores em CSS são definidas usando uma combinação de valores para vermelho, verde e azul, representados por números hexadecimais variando de 0 a f. Um código hexadecimal de 3 dígitos pode ser utilizado quando os componentes de cor (vermelho, verde e azul) possuem valores iguais ou semelhantes. Por exemplo, o código **#ff00cc**, onde o vermelho é máximo, o verde é zero e o azul é médio, pode ser simplificado para **#f0c**. Assim, **#f0c** é apenas uma forma abreviada de representar a mesma cor que **#ff00cc**.

EXEMPLOS DE CORES EM CSS	<H1 STYLE="BACKGROUND-COLOR:RGB(255, 99, 71);">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:#FF6347;">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:HSL(9, 100%, 64%);">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:RGBA(255, 99, 71, 0.5);">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:HSLA(9, 100%, 64%, 0.5);">...</H1>
CORES EM CSS	UM VALOR DE COR RGB REPRESENTA AS FONTES DE LUZ VERMELHA, VERDE E AZUL.
	UMA COR HEXADECIMAL É ESPECIFICADA COM: #RRGGBB, ONDE OS INTEIROS HEXADECIMAIS RR (VERMELHO), GG (VERDE) E BB (AZUL)
	HSL SIGNIFICA MATIZ, SATURAÇÃO E LEVEZA: HSL (MATIZ , SATURAÇÃO , LUMINOSIDADE)

Fundos CSS

Os fundos CSS são uma forma de definir a aparência de fundo de um elemento HTML usando código CSS. Eles permitem que os desenvolvedores escolham entre uma variedade de opções de fundo, como cores, imagens e gradientes, para personalizar a aparência visual do elemento.

Existem várias propriedades CSS que podem ser usadas para definir o fundo de um elemento, incluindo:

Propriedade	Descrição
background-color	Define a cor de fundo de um elemento. Você pode usar nomes de cores, valores rgb, hex, hsl para definir uma cor de fundo.
background-image	Define uma imagem como fundo de um elemento. A imagem é especificada por um valor de url. Você também pode usar a palavra-chave nenhum para remover qualquer imagem de plano de fundo.
background-repeat	Define como a imagem de fundo é repetida. O valor padrão é repetir, o que significa que a imagem será repetida horizontal e verticalmente. Você também pode usar repeat-x, repeat-y, no-repeat para controlar a repetição da imagem.
background-position	Define a posição da imagem de fundo. Você pode usar valores como centro, superior, inferior, esquerda, direita ou valores de comprimento específicos, como 10px 20px.
background-attachment	Define se a imagem de fundo rola com o resto do conteúdo (scroll) ou é fixa no lugar (fixed).
background-clip	Define a área onde a imagem de fundo deve ser pintada. O valor padrão é border-box, o que significa que a imagem será pintada dentro das bordas do elemento.
background-size	Define o tamanho da imagem de fundo. Você pode usar valores como conter ou cobrir para dimensionar a imagem ou valores de comprimento como 50% ou 200px para definir um tamanho específico.

Você também pode usar a propriedade abreviada **background** para definir todas as propriedades do background em uma linha.

É importante observar que a ordem dos valores na propriedade abreviada é:
plano de fundo: cor de fundo - imagem de fundo - repetição de fundo - posição de fundo

PLANO DE FUNDO: COR DE FUNDO - IMAGEM DE FUNDO - REPETIÇÃO DE FUNDO - POSIÇÃO DE FUNDO

Você pode usar essas propriedades para definir o plano de fundo de qualquer elemento HTML, tornando-o uma ferramenta poderosa para criar designs da Web atraentes e envolventes.

Imagem de Fundo CSS

A propriedade `background-image` em CSS é usada para definir uma imagem como plano de fundo de um elemento. A imagem é especificada por um valor de URL, que pode ser um caminho relativo ou absoluto. Por exemplo:

```
body {  
  background-image: url("my-image.jpg");  
}
```

Você também pode usar a palavra-chave `none` para remover qualquer imagem de plano de fundo.

A propriedade `background-repeat` controla como a imagem de fundo é repetida. O valor padrão é `repeat`, o que significa que a imagem será repetida horizontal e verticalmente. Outros valores possíveis são `repeat-x`, `repeat-y` e `no-repeat`.

```
body {  
  background-image: url("my-image.jpg");  
  background-repeat: no-repeat;  
}
```

A propriedade `background-position` define a posição da imagem de fundo. Você pode usar valores como `center`, `top`, `bottom`, `left`, `right` ou valores de comprimento específicos, como `10px 20px`.

```
body {  
  background-image: url("my-image.jpg");  
  background-position: center;  
}
```

A propriedade `background-size` define o tamanho da imagem de fundo. Você pode usar valores como `contain` ou `cover` para dimensionar a imagem ou valores de comprimento como `50%` ou `200px` para definir um tamanho específico.

```
body {  
  background-image: url("my-image.jpg");  
  background-size: cover;  
}
```

Além disso, você pode usar a propriedade abreviada `background` para definir todas as propriedades do `background` em uma linha.

```
body {  
  background:
```

A forma abreviada da propriedade **background** em CSS permite que você defina todas as propriedades background em uma única linha de código. Isso pode tornar seu código CSS mais conciso e fácil de ler. A ordem dos valores na propriedade abreviada é:

background: background-color background-image background-repeat background-position/size background-attachment

ORDEM DOS VALORES NA FORMA ABREVIADA DA PROPRIEDADE BACKGROUND

BACKGROUND-COLOR

BACKGROUND-IMAGE

BACKGROUND-REPEAT

BACKGROUND-POSITION/
SIZE

BACKGROUND-ATTACHM
ENT

Por exemplo, para definir uma cor de fundo de **#ff0000**, uma imagem "**my-image.jpg**" que se repete apenas na horizontal, posicionada no centro e fixa no lugar, você pode usar o seguinte código:

```
body {  
  background: #ff0000 url("my-image.jpg") repeat-x center fixed;  
}
```

ou

```
body {  
  background: #ff0000 url("my-image.jpg") repeat-x center/cover fixed;  
}
```

É importante observar que a propriedade abreviada **substituirá quaisquer propriedades de plano de fundo anteriores definidas individualmente**. Também é importante observar que nem todos os valores são obrigatórios, depende do que você deseja definir, para que você possa usá-lo conforme sua necessidade.

Propriedade CSS	Descrição
background	Define todas as propriedades de plano de fundo em uma declaração
background-attachment	Define se uma imagem de fundo é fixa ou rola com o restante da página
background-clip	Especifica a área de pintura do fundo
background-color	Define a cor de fundo de um elemento
background-image	Define a imagem de fundo para um elemento
background-origin	Especifica onde a(s) imagem(ns) de fundo está(ão) posicionada(s)

background-position	Define a posição inicial de uma imagem de fundo
background-repeat	Define como uma imagem de fundo será repetida
background-size	Especifica o tamanho da(s) imagem(ns) de fundo

Propriedades de Borda do CSS

As propriedades de borda do CSS permitem que você especifique o estilo, a largura e a cor da borda de um elemento. Elas permitem controlar a aparência das bordas de um elemento HTML. Algumas das principais propriedades de borda incluem:

- `border-width`: controla a largura da borda.
- `border-style`: controla o estilo da borda (por exemplo, sólido, tracejado, etc.).
- `border-color`: controla a cor da borda.
- `border-radius`: controla a curvatura das bordas (para criar bordas arredondadas).
- `border`: é uma propriedade curta que permite definir a largura, estilo e cor da borda de uma só vez.

Além disso, também é possível controlar as bordas individuais de cada lado de um elemento utilizando as propriedades `border-top`, `border-right`, `border-bottom` e `border-left`. A propriedade `border-style` especifica que tipo de borda exibir. Os seguintes valores são permitidos:

Estilo de Borda CSS	Descrição
<code>dotted</code>	Define uma borda pontilhada
<code>dashed</code>	Define uma borda tracejada
<code>solid</code>	Define uma borda sólida
<code>double</code>	Define uma borda dupla
<code>groove</code>	Define uma borda ranhurada 3D. O efeito depende do valor da cor da borda
<code>ridge</code>	Define uma borda ondulada 3D. O efeito depende do valor da cor da borda
<code>inset</code>	Define uma borda de inserção 3D. O efeito depende do valor da cor da borda
<code>outset</code>	Define uma borda inicial 3D. O efeito depende do valor da cor da borda
<code>none</code>	Não define borda
<code>hidden</code>	Define uma borda oculta

Vejamos um exemplo e seu respectivo resultado.

```
<!DOCTYPE html>
```

```
<html>
<head>
<style>
p.dotted {border-style: dotted;}
p.dashed {border-style: dashed;}
p.solid {border-style: solid;}
p.double {border-style: double;}
p.groove {border-style: groove;}
p.ridge {border-style: ridge;}
p.inset {border-style: inset;}
p.outset {border-style: outset;}
p.none {border-style: none;}
p.hidden {border-style: hidden;}
p.mix {border-style: dotted dashed solid double;}
</style>
</head>
<body>

<h2>A propriedade border-style</h2>
<p>Esta propriedade especifica que tipo de borda exibir:</p>

<p class="dotted">Uma borda pontilhada.</p>
<p class="dashed">Uma borda tracejada.</p>
<p class="solid">Uma fronteira sólida.</p>
<p class="double">Uma borda dupla.</p>
<p class="groove">Uma borda de sulco.</p>
<p class="ridge">Uma borda de cume.</p>
<p class="inset">Uma borda embutida.</p>
<p class="outset">Uma borda inicial.</p>
<p class="none">Sem borda.</p>
<p class="hidden">Uma fronteira oculta.</p>
<p class="mix">Uma fronteira mista.</p>

</body>
</html>
```

A propriedade border-style

Esta propriedade especifica que tipo de borda exibir:

Uma borda pontilhada.

Uma borda tracejada.

Uma fronteira sólida.

Uma borda dupla.

Uma borda de sulco.

Uma borda de cume.

Uma borda embutida.

Uma borda inicial.

Sem borda.

Uma fronteira oculta.

Uma borda mista.

Largura da Borda do CSS

A propriedade border-width especifica a largura das quatro bordas de um elemento HTML. A largura da borda pode ser especificada usando diferentes unidades de medida, como pixels (px), pontos (pt), centímetros (cm) e porcentagem (%). Usando um tamanho específico (em px, pt, cm, em, etc) ou usando um dos três valores predefinidos: thin, medium, or thick:

```
<!DOCTYPE html>
<html>
<head>
<style>
p.one {
  border-style: solid;
  border-width: 5px;
}

p.two {
  border-style: solid;
  border-width: medium;
}

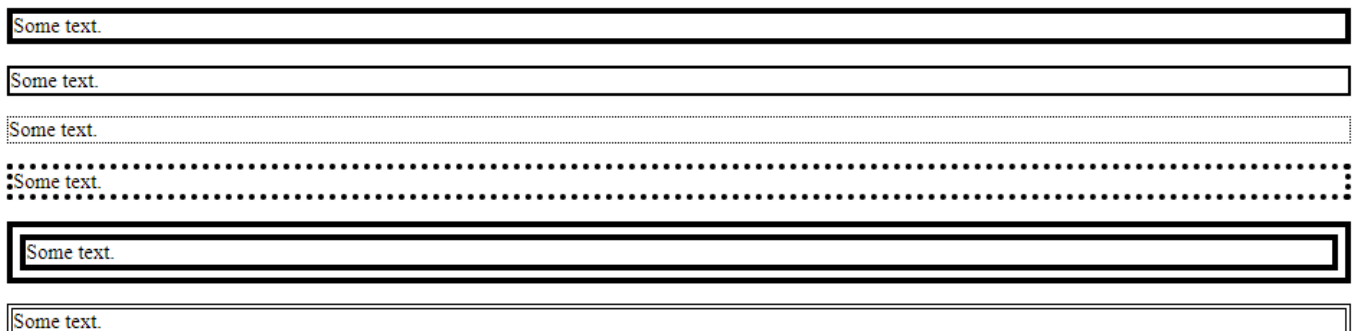
p.three {
  border-style: dotted;
  border-width: 2px;
}

p.four {
  border-style: dotted;
  border-width: thick;
}
```

```
}  
  
p.five {  
  border-style: double;  
  border-width: 15px;  
}  
  
p.six {  
  border-style: double;  
  border-width: thick;  
}  
</style>  
</head>  
<body>  
  
<h2>The border-width Property</h2>  
<p>This property specifies the width of the four borders:</p>  
  
<p class="one">Some text.</p>  
<p class="two">Some text.</p>  
<p class="three">Some text.</p>  
<p class="four">Some text.</p>  
<p class="five">Some text.</p>  
<p class="six">Some text.</p>  
  
<p>A propriedade "border-width" não funciona se for usada sozinha.  
Sempre especifique a propriedade "border-style" para definir as bordas  
primeiro.</p>  
  
</body>  
</html>
```

The border-width Property

This property specifies the width of the four borders:



A propriedade "border-width" não funciona se for usada sozinha. Sempre especifique a propriedade "border-style" para definir as bordas primeiro.

A propriedade `border-width` pode ter de um a quatro valores (para a borda superior, borda direita, borda inferior e borda esquerda):

```
p.one {  
  border-style: solid;  
  border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */  
}  
  
p.two {  
  border-style: solid;  
  border-width: 20px 5px; /* 20px top and bottom, 5px on the sides */  
}  
  
p.three {  
  border-style: solid;  
  border-width: 25px 10px 4px 35px; /* 25px top, 10px right, 4px bottom and  
35px left */  
}
```

Cores de borda

A propriedade **border-color** do CSS permite **controlar a cor das quatro bordas** de um elemento HTML. A cor pode ser especificada usando uma variedade de formatos, incluindo nomes de cores (como "red" ou "blue"), códigos hexadecimais (como "#ff0000" ou "#0000ff") ou valores RGB ou RGBA (como "rgb(255, 0, 0)" ou "rgba(255, 0, 0, 0.5)").

A cor pode ser definida por:

- **nome** - especifique um nome de cor, como "vermelho"
- **HEX** - especifique um valor HEX, como "#ff0000"
- **RGB** - especifique um valor RGB, como "rgb(255,0,0)"
- **HSL** - especifique um valor HSL, como "hsl(0, 100%, 50%)"
- transparente

Se a propriedade **border-color** não estiver definida, então ela herda a cor do elemento. Vejamos um exemplo.

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
p.one {  
  border-style: solid;  
  border-color: red;  
}  
  
p.two {  
  border-style: solid;  
  border-color: green;  
}
```

```
p.three {  
  border-style: dotted;  
  border-color: blue;  
}  
</style>  
</head>  
<body>  
  
<h2>A propriedade border-color</h2>  
<p>Esta propriedade especifica a cor das quatro bordas:</p>  
  
<p class="one">A solid red border</p>  
<p class="two">A solid green border</p>  
<p class="three">A dotted blue border</p>  
  
<p><b>Nota:</b> A propriedade "border-color" não funciona se for usada  
sozinha. Use a propriedade "border-style" para definir as bordas primeiro.</p>  
  
</body>  
</html>
```

A propriedade border-color

Esta propriedade especifica a cor das quatro bordas:



Nota: A propriedade "border-color" não funciona se for usada sozinha. Use a propriedade "border-style" para definir as bordas primeiro.

Também é possível controlar a cor da borda individualmente para cada lado de um elemento utilizando as propriedades border-top-color, border-right-color, border-bottom-color e border-left-color.

```
border-top-color: red;  
border-right-color: #0000ff;
```

Note que se você usar a propriedade border curta, você pode definir a cor, largura e estilo de borda de uma só vez.

```
/* Definir a cor, largura e estilo da borda como vermelho, 5 pixels e sólido*/  
border: 5px solid red;
```

Não entendeu? Então, vejamos como funciona. As propriedades para controlar cada lado individualmente são:

- border-top: controla a borda superior.

- border-right: controla a borda direita.
- border-bottom: controla a borda inferior.
- border-left: controla a borda esquerda.

Sempre nessa ordem. Assim, se a propriedade border-style tiver quatro valores:

border-style: dotted solid double dashed

- borda superior é pontilhada;
- borda direita é sólida;
- borda inferior é dupla;
- borda esquerda é tracejada.

Se a propriedade border-style tiver três valores:

border-style: dotted solid double;

- borda superior é pontilhada;
- as bordas direita e esquerda são sólidas;
- borda inferior é dupla.

Se a propriedade border-style tiver dois valores:

border-style: dotted solid;

- as bordas superior e inferior são pontilhadas;
- as bordas direita e esquerda são sólidas.

Se a propriedade border-style tiver um valor:

border-style: dotted;

- todas as quatro bordas são pontilhadas.

Também é possível controlar as propriedades de borda individualmente para cada lado utilizando as propriedades border-top-width, border-top-style, border-top-color, border-right-width, border-right-style, border-right-color, border-bottom-width, border-bottom-style, border-bottom-color, border-left-width, border-left-style, and border-left-color.

Há muitas propriedades a serem consideradas ao lidar com bordas, entretanto, para encurtar o código, também é possível **especificar todas as propriedades** de borda individuais em uma propriedade.

A propriedade **border** é uma propriedade abreviada para as seguintes propriedades de borda individuais:

```
/* Sintaxe geral */  
border: width style color;  
  
/* Definir a borda como vermelha, largura de 5 pixels e sólida */  
border: 5px solid red;
```

```
/* Definir a borda como azul, largura de 2 pixels e tracejada */  
border: 2px dotted blue;  
  
/* Definir a borda como verde, largura de 1 ponto e dupla */  
border: 1pt double green;  
  
/* Definir a borda superior como vermelha, largura de 5 pixels e sólida */  
border-top: 5px solid red;  
  
/* Definir a borda direita como azul, largura de 2 pixels e tracejada */  
border-right: 2px dotted blue;  
  
/* Definir a borda inferior e esquerda como verde, largura de 1 ponto e dupla */  
border-bottom: 1pt double green;  
border-left: 1pt double green;
```

As propriedades de bordas arredondadas do CSS permitem criar bordas arredondadas em elementos HTML. A propriedade principal para controlar bordas arredondadas é a border-radius.

```
/* Sintaxe geral */  
border-radius: horizontal vertical;  
  
/* Definir borda arredondada com raio de 20 pixels */  
border-radius: 20px;  
  
/* Definir borda arredondada com raio horizontal de 30 pixels e raio vertical de 10 pixels */  
border-radius: 30px 10px;  
  
/* Definir raio de borda arredondada superior esquerda como 30 pixels */  
border-top-left-radius: 30px;  
  
/* Definir raio de borda arredondada superior direita como 10 pixels */  
border-top-right-radius: 10px;  
  
/* Definir raio de borda arredondada inferior esquerda como 5 pixels */  
border-bottom-left-radius: 5px;  
  
/* Definir raio de borda arredondada inferior direita como 15 pixels */  
border-bottom-right-radius: 15px;
```

Professora, para quê tanto exemplo? Porque cai em prova!

Vejamos agora todas as propriedades de borda CSS

Propriedades de Borda CSS	Descrição
border	Define todas as propriedades de borda em uma declaração
border-bottom	Define todas as propriedades da borda inferior em uma declaração
border-bottom-color	Define a cor da borda inferior
border-bottom-style	Define o estilo da borda inferior
border-bottom-width	Define a largura da borda inferior
border-color	Define a cor das quatro bordas
border-left	Define todas as propriedades da borda esquerda em uma declaração
border-left-color	Define a cor da borda esquerda
border-left-style	Define o estilo da borda esquerda
border-left-width	Define a largura da borda esquerda
border-radius	Define todas as quatro propriedades border-*-radius para cantos arredondados
border-right	Define todas as propriedades da borda direita em uma declaração
border-right-color	Define a cor da borda direita
border-right-style	Define o estilo da borda direita
border-right-width	Define a largura da borda direita
border-style	Define o estilo das quatro bordas
border-top	Define todas as propriedades da borda superior em uma declaração
border-top-color	Define a cor da borda superior
border-top-style	Define o estilo da borda superior
border-top-width	Define a largura da borda superior
border-width	Define a largura das quatro bordas

Margens CSS

As margens são usadas para **criar espaço ao redor dos elementos**, fora de qualquer borda definida. Elas **permitem controlar o espaço fora de um elemento HTML**. Consiste na área transparente ao redor de um elemento e o separa dos elementos vizinhos.

A propriedade principal para controlar margens é a **margin**.

```
margin: top right bottom left;
```

Em que **top, right, bottom e left** são **valores que determinam a largura da margem** em cima, à direita, embaixo e à esquerda, respectivamente. Você pode usar unidades como **pixels, pontos ou porcentagens**. Todas as propriedades de margem **podem ter os seguintes valores**:

- **auto** - o navegador calcula a margem
- **length** (comprimento) - especifica uma margem em px, pt, cm, etc.
- **%** - especifica uma margem em % da largura do elemento recipiente
- **inherit** (herdar) - especifica que a margem deve ser herdada do elemento pai

Você pode definir a propriedade margin para **autocentrizar horizontalmente** o elemento em seu contêiner. O elemento ocupará a largura especificada e o espaço restante será dividido igualmente entre as margens esquerda e direita.

```
/* Definir margem de 10 pixels em todos os lados */  
margin: 10px;  
  
/* Definir margem de 5 pixels em cima e embaixo, e 20 pixels à direita e esquerda */  
margin: 5px 20px;  
  
/* Definir margem de 10 pixels em cima, 20 pixels à direita, 30 pixels embaixo e 40 pixels à esquerda */  
margin: 10px 20px 30px 40px;
```

Você também pode controlar as margens individualmente para cada lado usando as propriedades **margin-top, margin-right, margin-bottom, e margin-left**.

```
/* Definir margem superior como 10 pixels */  
margin-top: 10px;  
  
/* Definir margem direita como 20 pixels */  
margin-right: 20px;  
  
/* Definir margem inferior como 30 pixels */  
margin-bottom: 30px;  
  
/* Definir margem esquerda como 40 pixels */
```

```
margin-left: 40px;
```

As margens são importantes para controlar o espaçamento e a disposição de elementos na página web, e podem ser usadas para criar uma aparência mais organizada e estruturada para seu site.

Propriedades de Margem CSS	Descrição
margin	Uma propriedade abreviada para definir todas as propriedades de margem em uma declaração
margin-bottom	Define a margem inferior de um elemento
margin-left	Define a margem esquerda de um elemento
margin-right	Define a margem direita de um elemento
margin-top	Define a margem superior de um elemento

Às vezes, as **margens superior e inferior** dos elementos **são recolhidas em uma única margem** que é igual à maior das duas margens. Isso não acontece nas margens esquerda e direita! Apenas as margens **superior e inferior**!

As margens são importantes para controlar o espaçamento e a disposição de elementos na página web, e podem ser usadas para criar uma aparência mais organizada e estruturada para seu site.

Preenchimento CSS

O preenchimento CSS controla o espaço dentro de um elemento HTML. O preenchimento é a área preenchida dentro de um elemento, ocupando o espaço entre o seu conteúdo e sua borda.

A propriedade principal para controlar o preenchimento é a `padding`. A sintaxe geral é:

```
padding: top right bottom left;
```

Em que `top`, `right`, `bottom` e `left` são valores que determinam a largura do preenchimento em cima, à direita, embaixo e à esquerda, respectivamente. Você pode usar unidades como pixels, pontos ou porcentagens. Vejamos os exemplos:

```
/* Definir preenchimento de 10 pixels em todos os lados */  
padding: 10px;  
  
/* Definir preenchimento de 5 pixels em cima e embaixo, e 20 pixels à direita  
e esquerda */  
padding: 5px 20px;  
  
/* Definir preenchimento de 10 pixels em cima, 20 pixels à direita, 30 pixels  
embaixo e 40 pixels à esquerda */  
padding: 10px 20px 30px 40px;
```

Você também pode controlar o preenchimento individualmente para cada lado usando as propriedades `padding-top`, `padding-right`, `padding-bottom`, e `padding-left`.

Todas as propriedades de preenchimento podem ter os seguintes valores:

- `length` - especifica um preenchimento em px, pt, cm, etc.
- `%` - especifica um preenchimento em % da largura do elemento que o contém
- `inherit` - especifica que o preenchimento deve ser herdado do elemento pai

Frise-se que valores negativos não são permitidos.

```
/* Definir preenchimento superior como 10 pixels */  
padding-top: 10px;  
  
/* Definir preenchimento direita como 20 pixels */  
padding-right: 20px;  
  
/* Definir preenchimento inferior como 30 pixels */  
padding-bottom: 30px;  
  
/* Definir preenchimento esquerda como 40 pixels */  
padding-left: 40px;
```

A propriedade CSS **width** especifica a **largura da área de conteúdo do elemento**. A área de conteúdo é a **parte dentro do preenchimento, borda e margem de um elemento** (o modelo de caixa). Portanto, se um elemento tiver uma largura especificada, o preenchimento adicionado a esse elemento será adicionado à largura total do elemento. Isso geralmente é um resultado indesejável.

Propriedades de Preenchimento CSS	Descrição
padding	Uma propriedade abreviada para definir todas as propriedades de preenchimento em uma declaração
padding-bottom	Define o preenchimento inferior de um elemento
padding-left	Define o preenchimento esquerdo de um elemento
padding-right	Define o preenchimento correto de um elemento
padding-top	Define o preenchimento superior de um elemento

O preenchimento é importante para controlar o espaçamento entre o conteúdo e a borda de um elemento, e pode ser usado para criar uma aparência mais atraente e organizada para seu site.

Altura, largura e largura máxima do CSS

A altura, largura e largura máxima são propriedades CSS que controlam o tamanho de um elemento HTML.

A propriedade `height` define a altura de um elemento, a propriedade `width` define a largura de um elemento e a propriedade `max-width` define a largura máxima de um elemento. Vejamos a sintaxe para definir altura, largura e largura máxima, respectivamente:

```
/* Sintaxe para definir altura, largura e largura máxima, respectivamente */  
height: value;  
width: value;  
max-width: value;
```

Em que `value` é o valor da altura, largura ou largura máxima. Você pode usar unidades como pixels, pontos ou porcentagens.

```
/* Definir altura como 200 pixels */  
height: 200px;  
  
/* Definir largura como 50% */  
width: 50%;  
  
/* Definir largura máxima como 500 pixels */  
max-width: 500px;
```

A propriedade `max-width` é usada para definir a largura máxima de um elemento. Ela pode ser especificada em valores de comprimento, como `px`, `cm`, etc., ou em porcentagem (%) do bloco que o contém, ou definido como nenhum (este é o padrão. Significa que não há largura máxima).

Quando a janela do navegador é menor que a largura do elemento (500px), o navegador então adiciona uma barra de rolagem horizontal à página. Uma boa opção é usar `max-width` nessa situação, assim, irá melhorar o manuseio de janelas pequenas pelo navegador.

Observe que, se a largura máxima for definida e a largura do elemento exceder o valor da largura máxima, a largura do elemento será automaticamente ajustada para o valor da largura máxima.

Nota que, se por algum motivo você usar a `width` e `max-width` no mesmo elemento e o valor da `width` for maior que a `max-width`, a `max-width` será usada (e a `width` será ignorada).

As propriedades `height` e `width` podem ter os seguintes valores:

- `auto`- Este é o padrão. O navegador calcula a altura e a largura
- `length`- Define a altura/largura em `px`, `cm`, etc.
- `%`- Define a altura/largura em porcentagem do bloco que o contém

- **initial**- Define a altura/largura para seu valor padrão
- **inherit**- A altura/largura será herdada de seu valor pai

Propriedades de Preenchimento CSS	Descrição
height	Define a altura de um elemento
max-height	Define a altura máxima de um elemento
max-width	Define a largura máxima de um elemento
min-height	Define a altura mínima de um elemento
min-width	Define a largura mínima de um elemento
width	Define a largura de um elemento

A altura, largura e largura máxima são importantes para controlar o tamanho de elementos na página web e podem ser usadas para criar uma aparência mais organizada e estruturada para seu site.

Modelo de Caixa CSS

O modelo de caixa CSS é o modelo que define como o **conteúdo, borda, preenchimento e margem** de um elemento HTML **são exibidos e gerenciados na página**. O modelo de caixa CSS é composto pelas seguintes partes:

- **Conteúdo**: é o conteúdo real dentro do elemento, como texto, imagens, etc.
- **Preenchimento**: é o espaço entre o conteúdo e a borda, controlado pela propriedade padding.
- **Borda**: é uma linha ao redor do preenchimento e do conteúdo, controlada pelas propriedades border-width, border-style e border-color.
- **Margem**: é o espaço fora da borda, controlado pela propriedade margin.

Cada parte **pode ser controlada individualmente** para dar ao elemento o visual desejado. Além disso, o modelo de caixa também afeta **como o tamanho de um elemento é calculado**, incluindo sua altura, largura, etc.

A compreensão do modelo de caixa CSS é fundamental para o desenvolvimento de páginas web profissionais e é usado para **criar layouts, espaçamento e visualização de elementos na página**.

A largura total de um elemento deve ser calculada assim:

- $\text{Largura total do elemento} = \text{largura} + \text{preenchimento esquerdo} + \text{preenchimento direito} + \text{borda esquerda} + \text{borda direita} + \text{margem esquerda} + \text{margem direita}$

A altura total de um elemento deve ser calculada assim:

- $\text{Altura total do elemento} = \text{altura} + \text{preenchimento superior} + \text{preenchimento inferior} + \text{borda superior} + \text{borda inferior} + \text{margem superior} + \text{margem inferior}$

Contorno do CSS

Um contorno é uma linha desenhada ao redor dos elementos, **FORA** das bordas, para fazer o elemento "se destacar".

CSS tem as seguintes propriedades de contorno:

- outline-style
- outline-color
- outline-width
- outline-offset
- outline

Estilo de Contorno CSS

A propriedade **outline-style** especifica o estilo do contorno e pode ter um dos seguintes valores:

- dotted- Define um contorno pontilhado
- dashed- Define um contorno tracejado
- solid- Define um contorno sólido
- double- Define um contorno duplo
- groove- Define um contorno ranhurado 3D
- ridge- Define um contorno estriado 3D
- inset- Define um contorno de inserção 3D
- outset- Define um esboço inicial 3D
- none- Não define contorno
- hidden- Define um contorno oculto

Largura do Contorno do CSS

A propriedade `outline-width` especifica a largura do contorno e pode ter um dos seguintes valores:

- fino (normalmente 1px)
- médio (normalmente 3px)
- espessura (normalmente 5px)
- Um tamanho específico (em px, pt, cm, em, etc)

Cor do Contorno do CSS

A propriedade `outline-color` é usada para definir a cor do contorno.

A cor pode ser definida por:

- nome - especifique um nome de cor, como "vermelho"
- HEX - especifique um valor hexadecimal, como "#ff0000"
- RGB - especifique um valor RGB, como "rgb(255,0,0)"
- HSL - especifique um valor HSL, como "hsl(0, 100%, 50%)"
- invert - realiza uma inversão de cores (o que garante que o contorno fique visível, independentemente da cor do fundo)

Abreviação do Contorno do CSS

A propriedade `outline` é uma propriedade de abreviação CSS que permite **definir todas as propriedades de linha de uma só vez**. A propriedade de linha define uma linha ao redor de um elemento que é exibida fora da borda, mas ainda faz parte do modelo de caixa.

```
outline: width style color;
```

- width é a espessura da linha, em pixels ou outra unidade.
- style é o estilo da linha, como sólido, pontilhado etc.
- color é a cor da linha.

```
/* Define uma linha de 1 px de largura, sólida e verde */
```

```
outline: 1px solid green;
```

Observe que, ao contrário da propriedade border, a propriedade outline **não afeta o tamanho do elemento**, portanto, **não afeta a disposição dos outros elementos na página**. Além disso, a propriedade outline não tem preenchimento.

A propriedade outline é uma alternativa mais rápida às propriedades separadas de linha, como outline-width, outline-style e outline-color, e é usada para definir rapidamente a aparência da linha em torno de um elemento.

Propriedades de Contorno CSS	Descrição
outline	Uma propriedade abreviada para definir a largura do contorno, o estilo do contorno e a cor do contorno em uma declaração
outline-color	Define a cor de um contorno
outline-offset	Especifica o espaço entre um contorno e a borda ou borda de um elemento
outline-style	Define o estilo de um contorno
outline-width	Define a largura de um contorno

Texto CSS

A propriedade color é usada para definir a cor do texto. A cor é especificada por:

um nome de cor - como "vermelho"

um valor HEX - como "#ff0000"

um valor RGB - como "rgb(255,0,0)"

Consulte Valores de cores CSS para obter uma lista completa de possíveis valores de cores.

O CSS oferece uma ampla gama de propriedades para formatar o texto, incluindo:

Propriedade	Descrição
font-family	Define a fonte para o texto.
font-size	Define o tamanho da fonte.
font-weight	Define o peso da fonte (normal, bold, etc.).
line-height	Define a altura da linha.
letter-spacing	Define o espaçamento entre letras.
word-spacing	Define o espaçamento entre palavras.
text-align	Define o alinhamento do texto (esquerda, direita, centro, justificado).
text-decoration	Define a decoração do texto (sublinhado, riscado, etc.).
text-transform	Define a transformação do texto (maiúsculas, minúsculas, capitalização).
color	Define a cor do texto.
text-shadow	Define uma sombra para o texto.

Essas propriedades permitem aos desenvolvedores de páginas web personalizar a aparência do texto de acordo com suas necessidades e criar aparências atraentes e profissionais.

A formatação do texto também pode ser herdada de um elemento pai para seus elementos filhos, a menos que seja especificamente sobrescrita para um elemento filho.

Cor do texto

A propriedade color é usada para definir a cor do texto. A cor pode ser especificada como uma string de cor RGB, HEX, RGBA ou como uma palavra-chave (por exemplo, "red" ou "blue"). A cor é especificada por:

- um nome de cor - como "vermelho"
- um valor HEX - como "#ff0000"
- um valor RGB - como "rgb(255,0,0)"

```
p {  
  color: blue;  
}
```

Propriedades de Cor do Texto	Descrição
color	Especifica a cor do texto
opacity	Esta propriedade define a opacidade do texto. Um valor de 1 é totalmente opaco, enquanto um valor de 0 é totalmente transparente.
text-shadow	Adiciona uma sombra ao texto. É possível especificar a cor da sombra, a distância da sombra em relação ao texto e o tamanho da sombra.
background-color	Define a cor de fundo do elemento que contém o texto. Quando o texto tem uma cor diferente da cor de fundo, a propriedade background-color torna-se evidente.
text-decoration	Permite adicionar uma linha, sublinhado ou overline ao texto. É possível especificar a cor e o estilo da linha.

Alinhamento e direção do texto

A propriedade text-align é usada para alinhar o texto a esquerda, a direita, no centro ou justificado.

```
p {  
  text-align: center;  
}
```

O exemplo a seguir mostra o texto alinhado ao centro e à esquerda e à direita (o alinhamento à esquerda é padrão se a direção do texto for da esquerda para a direita e o alinhamento à direita é padrão se a direção do texto for da direita para a esquerda):

```
h1 {  
  text-align: center;
```

```
}  
  
h2 {  
  text-align: left;  
}  
  
h3 {  
  text-align: right;  
}  
  
div {  
  text-align: justify;  
}
```

Quando a propriedade text-align é definida como "[justify](#)", cada linha é esticada para que cada linha tenha a mesma largura e as margens esquerda e direita sejam retas (como em revistas e jornais):

A propriedade direction é usada para especificar a direção do texto, ou seja, se ele será escrito da esquerda para a direita (ltr) ou da direita para a esquerda (rtl). Já, a propriedade text-align-last é usada para alinhar a [última linha de um texto justificado](#).

A propriedade Bidirecionalidade do Texto Unicode [unicode-bidi](#) é usada para controlar a direção de exibição de caracteres bidirecionais (por exemplo, árabes e hebreus) no texto.

Propriedades de alinhamento/direção do texto CSS	Descrição
direction	Especifica a direção do texto/direção da escrita
text-align	Especifica o alinhamento horizontal do texto
text-align-last	Especifica como alinhar a última linha de um texto
unicode-bidi	Usado junto com a propriedade de direção para definir ou retornar se o texto deve ser substituído para suportar vários idiomas no mesmo documento
vertical-align	Define o alinhamento vertical de um elemento

Propriedades de Decoração de Texto

- A propriedade text-decoration é usada para adicionar uma decoração ao texto, [como sublinhado, riscado](#), entre outros.
- A propriedade [text-decoration-line](#) é usada para especificar a linha de decoração a ser aplicada ao texto.

- A propriedade `text-decoration-color` é usada para especificar a cor da decoração de texto.
- A propriedade `text-decoration-style` é usada para especificar o estilo da decoração de texto, como sólido, duplo, entre outros.
- A propriedade `text-decoration-thickness` é usada para especificar a espessura da decoração de texto.

```
a {  
  text-decoration: underline;  
}  
  
a {  
  text-decoration-line: underline;  
  text-decoration-color: blue;  
  text-decoration-style: double;  
  text-decoration-thickness: 2px;  
}
```

Propriedades de Decoração de Texto CSS	Descrição
<code>text-decoration</code>	Define todas as propriedades de decoração de texto em uma declaração
<code>text-decoration-color</code>	Especifica a cor da decoração do texto
<code>text-decoration-line</code>	Especifica o tipo de decoração de texto a ser usado (sublinhado, sobrelinhado, etc.)
<code>text-decoration-style</code>	Especifica o estilo da decoração do texto (sólido, pontilhado, etc.)
<code>text-decoration-thickness</code>	Especifica a espessura da linha de decoração do texto

Transformação de Texto CSS

A propriedade `text-transform` é usada para especificar **letras maiúsculas e minúsculas** em um texto. Ele pode ser usado para transformar tudo em letras maiúsculas ou minúsculas, ou colocar em maiúscula a primeira letra de cada palavra:

```
p.uppercase {  
  text-transform: uppercase;  
}  
  
p.lowercase {
```

```
text-transform: lowercase;
}

p.capitalize {
text-transform: capitalize;
}
```

Propriedades de Transformação de Texto CSS	Descrição
text-transform	Controla a capitalização do texto

Espaçamento do Texto CSS

- A propriedade **text-indent** é usada para especificar a indentação do primeiro parágrafo de um elemento.
- A propriedade **letter-spacing** é usada para especificar o espaçamento entre as letras em um elemento.
- A propriedade **line-height** é usada para especificar a altura da linha de um elemento.
- A propriedade **word-spacing** é usada para especificar o espaçamento entre as palavras em um elemento.
- A propriedade **white-space** é usada para especificar como o espaço em branco é tratado em um elemento.

```
p {
text-indent: 50px;
letter-spacing: 2px;
line-height: 1.5;
word-spacing: 10px;
white-space: pre-wrap;
}
```

Propriedades de Decoração de Texto CSS	Descrição
letter-spacing	Especifica o espaço entre os caracteres em um texto
line-height	Especifica a altura da linha
text-indent	Especifica o recuo da primeira linha em um bloco de texto
white-space	Especifica como lidar com espaço em branco dentro de um elemento

word-spacing

Especifica o espaço entre as palavras em um texto

Sombra de Texto CSS

A propriedade text-shadow é usada para adicionar uma sombra ao texto em um elemento CSS. Ela permite especificar a posição horizontal, vertical e a opacidade da sombra, além da cor.

Em seu uso mais simples, você especifica apenas a sombra horizontal (2px) e a sombra vertical (2px):

```
h1 {  
  text-shadow: 2px 2px;  
}
```

```
p {  
  text-shadow: 2px 2px 5px gray;  
}
```

No último exemplo, a sombra tem uma posição horizontal de 2px à direita do texto, uma posição vertical de 2px abaixo do texto e tem uma opacidade de 5px com a cor cinza.

Propriedades de Sombra de Texto CSS	Descrição
text-shadow	Especifica o efeito de sombra adicionado ao texto

Fontes CSS

A propriedade **font** é uma **propriedade abreviada** que permite **especificar várias propriedades de fonte em um único valor**, como **tamanho, estilo, variante e peso**. Isso facilita a escrita de regras de estilo e economiza tempo. A ordem das propriedades especificadas na propriedade abreviada é **font-style, font-variant, font-weight, font-size, line-height e font-family**.

Escolher a fonte certa tem um grande impacto em **como os leitores experimentam um site**. A fonte certa pode criar uma **identidade forte para sua marca**. Usar uma fonte fácil de ler é importante. A fonte agrega valor ao seu texto. Também é importante escolher a cor e o tamanho do texto corretos para a fonte.

Em CSS existem cinco famílias de fontes genéricas:

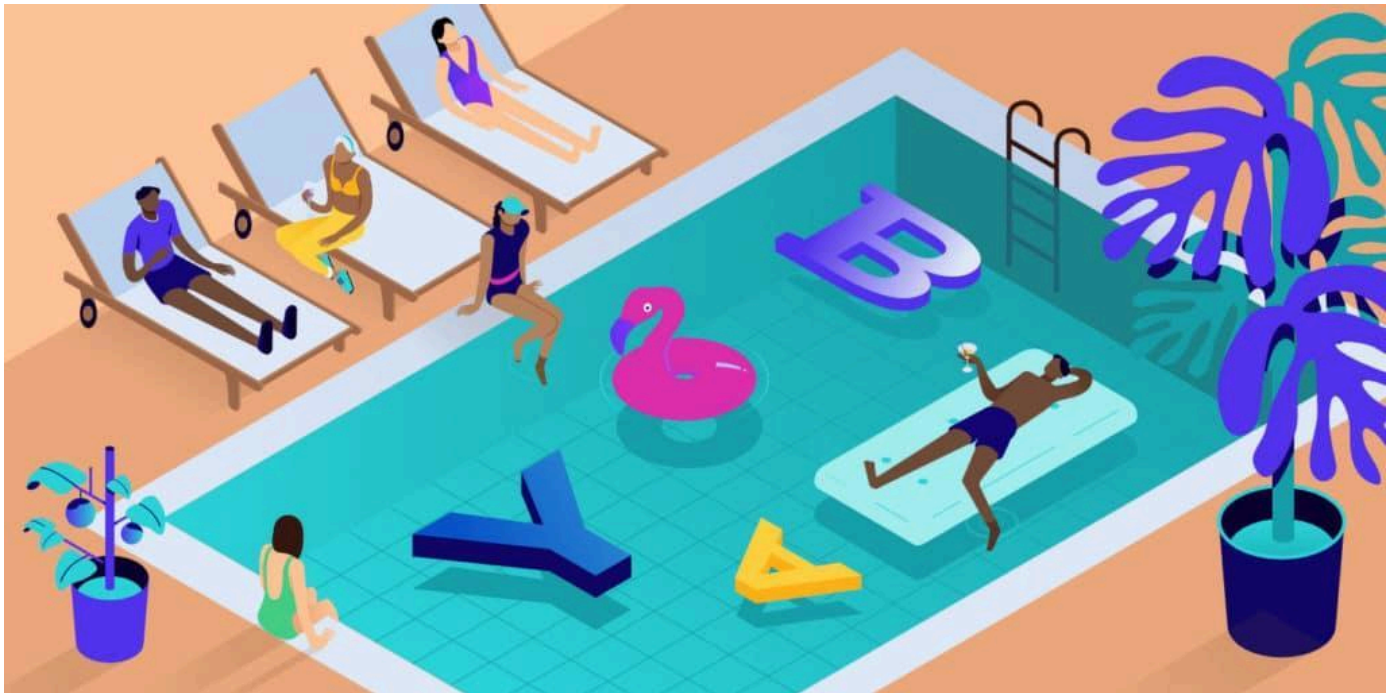
- As fontes **serifadas** têm um pequeno traço nas bordas de cada letra. Eles criam uma sensação de formalidade e elegância.
- As fontes **sem serifa** têm linhas limpas (sem pequenos traços anexados). Eles criam um visual moderno e minimalista.
- Fontes **monoespaçadas** - aqui todas as letras têm a mesma largura fixa. Eles criam uma aparência mecânica.
- As fontes **cursivas** imitam a caligrafia humana.
- Fontes **fantasia** são fontes decorativas/divertidas.



Dica: A propriedade **font-family** deve conter vários nomes de fontes como um sistema "fallback", para garantir a compatibilidade máxima entre navegadores/sistemas operacionais. Comece com a fonte desejada e termine com uma família genérica (para permitir que o navegador escolha uma fonte semelhante na família genérica, se nenhuma outra fonte estiver disponível). Os nomes das fontes devem ser separados por vírgula. Leia mais sobre fontes alternativas no próximo capítulo.

```
.p1 {  
  font-family: "Times New Roman", Times, serif;  
}  
  
.p2 {  
  font-family: Arial, Helvetica, sans-serif;  
}  
  
.p3 {
```

```
font-family: "Lucida Console", "Courier New", monospace;  
}
```



O que são fontes seguras da Web? Fontes seguras da Web são fontes **instaladas universalmente em todos os navegadores e dispositivos**. No entanto, não existem fontes 100% totalmente seguras para a Web. Há sempre uma chance de que uma fonte não seja encontrada ou instalada corretamente. Portanto, **é muito importante sempre usar fontes alternativas**.

Isso significa que você **deve adicionar uma lista de "fontes de backup"** semelhantes na font-family. Se a primeira fonte não funcionar, o navegador tentará a próxima, a próxima e assim por diante. Sempre termine a lista com um nome de família de fonte genérico.

Vejamos as melhores fontes seguras da Web para HTML e CSS!

A lista a seguir apresenta as melhores fontes seguras da Web para HTML e CSS:

- Arial (sem serifa)
- Verdana (sem serifa)
- Tahoma (sem serifa)
- Trebuchet MS (sem serifa)
- Times New Roman (serif)
- Geórgia (serifa)
- Garamond (serifa)
- Courier New (monoespacial)
- Brush Script MT (cursiva)

Arial é a fonte mais amplamente usada para mídia online e impressa. Arial também é a fonte **padrão no Google Docs**. Arial é uma das fontes da web mais seguras e está disponível em **todos os principais sistemas operacionais**.

Os fallbacks de fonte são fontes de backup que **são usadas caso a fonte original não esteja disponível** em um dispositivo ou sistema operacional. Abaixo estão alguns fallbacks de fontes comumente usados, organizados pelas 5 famílias de fontes genéricas:

- Serif: Times New Roman, Georgia
- Sans-serif: Arial, Verdana, Helvetica
- Monospace: Courier New, Lucida Console

Novamente, pessoal, ao especificar uma fonte, é importante **listar várias opções de fallback**, para **garantir que a fonte desejada apareça de maneira consistente** em diferentes dispositivos e sistemas operacionais. É recomendado usar **fontes genéricas, como serif, sans-serif ou monospace, como último recurso**.

Estilo de fonte CSS

A propriedade **font-style** é usada principalmente para especificar texto em itálico. Esta propriedade tem três valores:

- normal - O texto é mostrado normalmente
- itálico - O texto é mostrado em itálico
- oblique - O texto está "inclinado" (oblique é muito semelhante ao itálico, mas menos suportado)

```
p{
  font-style: normal;
  font-style: italic;
  font-style: oblique;
}
```

```
p{
  font-weight: normal;
  font-weight: bold;
}
```

A propriedade **font-variant** especifica se um texto deve ou não ser exibido em uma fonte minúscula. Em uma fonte minúscula, todas as letras minúsculas são convertidas em maiúsculas. No entanto, as letras maiúsculas convertidas aparecem em um tamanho de fonte menor do que as letras maiúsculas originais no texto.

```
p.normal {
  font-variant: normal;
}

p.small {
  font-variant: small-caps;
}
```

Tamanho da fonte

A propriedade font-size define o tamanho do texto. Ser capaz de gerenciar o tamanho do texto é importante em web design. No entanto, você não deve usar ajustes de tamanho de fonte para fazer parágrafos parecerem títulos ou títulos parecerem parágrafos. Sempre use as tags HTML apropriadas, como <h1> - <h6> para títulos e <p> para parágrafos.

O valor do tamanho da fonte pode ser um tamanho **absoluto ou relativo**.

Tamanho absoluto:

- Define o texto para um tamanho especificado
- Não permite que um usuário altere o tamanho do texto em todos os navegadores (ruim por motivos de acessibilidade)
- O tamanho absoluto é útil quando o tamanho físico da saída é conhecido

Tamanho relativo:

- Define o tamanho relativo aos elementos circundantes
- Permite que um usuário altere o tamanho do texto nos navegadores
- Nota: Se você não especificar um tamanho de fonte, o tamanho padrão para texto normal, como parágrafos, é 16px (16px=1em).

Para permitir que os usuários redimensionem o texto (no menu do navegador), muitos desenvolvedores usam em vez de pixels. **1em é igual ao tamanho da fonte atual**. O tamanho de texto padrão nos navegadores é 16px. Portanto, **o tamanho padrão de 1em é 16px**. O tamanho pode ser calculado de pixels para em usando esta fórmula: pixels /16= em

```
h1 {  
  font-size: 2.5em; /* 40px/16=2.5em */  
}  
  
h2 {  
  font-size: 1.875em; /* 30px/16=1.875em */  
}  
  
p {  
  font-size: 0.875em; /* 14px/16=0.875em */  
}
```

No exemplo acima, o tamanho do texto em em é o mesmo do exemplo anterior em pixels. Porém, com o tamanho em, é possível ajustar o tamanho do texto em todos os navegadores. Infelizmente, ainda há um problema com versões mais antigas do Internet Explorer. O texto fica maior do que deveria quando aumentado e menor do que deveria quando reduzido.

As Fontes do Google são uma coleção de fontes de caracteres disponíveis



gratuitamente para uso na web. Elas podem ser incorporadas em uma página web através de um link para o CSS da fonte fornecido pelo Google.

A vantagem de usar as Fontes do Google é que elas são otimizadas para a web, o que significa que são carregadas rapidamente e renderizadas de maneira consistente em diferentes dispositivos e sistemas operacionais. Além disso, o Google oferece uma ampla seleção de fontes, incluindo fontes serif, sans-serif, monospace e muito mais.

Aqui está um exemplo de como incorporar uma fonte do Google em uma página web:

```
<link
href="https://fonts.googleapis.com/css2?family=Open+Sans:italic,bold&display=swap" rel="stylesheet">

<style>
p {
  font-family: 'Open Sans', sans-serif;
  font-style: italic;
  font-weight: bold;
}
</style>
```

Nesse exemplo, estamos incorporando a fonte "Open Sans" do Google e definindo sua família de fontes, estilo e peso na seção de estilo da página.

O Google também ativou diferentes efeitos de fonte que você pode usar. Para habilitar efeitos de fonte no Google Fonts, você precisa selecionar uma fonte que tenha efeitos disponíveis e adicioná-la à sua página. Algumas fontes possuem vários efeitos, como itálico, negrito, sublinhado, efeito especial, entre outros. Para habilitar esses efeitos, basta incluí-los na tag "link" que você usa para incorporar a fonte na sua página.

Primeiro adicione à API do Google e, em seguida, adicione um nome de classe especial ao elemento que usará o efeito especial. O nome da classe sempre começa e termina com `.effect=effectnamefont-effect-effectname`

Vejamos como ficam esses efeitos especiais na fonte!

```
<head>
<link rel="stylesheet"
href="https://fonts.googleapis.com/css?family=Sofia&effect=neon|outline|emboss|shadow-multiple">
<style>
body {
  font-family: "Sofia", sans-serif;
  font-size: 30px;
}
</style>
</head>
```

```
<body>  
  
<h1 class="font-effect-neon">Neon Effect</h1>  
<h1 class="font-effect-outline">Outline Effect</h1>  
<h1 class="font-effect-emboss">Emboss Effect</h1>  
<h1 class="font-effect-shadow-multiple">Multiple Shadow Effect</h1>  
  
</body>
```



Regras de emparelhamento de fontes

As regras de emparelhamento de fontes são usadas para escolher fontes que tenham uma boa combinação entre si, com base em características como altura de linha, largura, espessura e outros atributos visuais. Vejamos as regras básicas.

1. Complemento

É sempre seguro encontrar pares de fontes que se complementam. Uma ótima combinação de fontes deve harmonizar, sem ser muito parecida ou muito diferente.

2. Use superfamílias de fontes

Uma superfamília de fontes é um conjunto de fontes projetadas para funcionar bem juntas. Portanto, usar fontes diferentes dentro da mesma superfamília é seguro. Por exemplo, a superfamília Lucida contém as seguintes fontes: Lucida Sans, Lucida Serif, Lucida Typewriter Sans, Lucida Typewriter Serif e Lucida Math.

3. O contraste é rei

Duas fontes muito semelhantes geralmente entrarão em conflito. No entanto, os contrastes, feitos da maneira certa, trazem o melhor de cada fonte. Exemplo: Combinar serif com sans serif

é uma combinação bem conhecida. Uma superfamília forte inclui variações com serifa e sem serifa da mesma fonte (por exemplo, Lucida e Lucida Sans).

4. Escolha apenas um chefe

Uma fonte deve ser o chefe. Isso estabelece uma hierarquia para as fontes em sua página. Isso pode ser conseguido variando o tamanho, peso e cor.

5. Escolha fontes que tenham uma altura de linha semelhante: Isso ajuda a manter a consistência e legibilidade em todo o texto.

6. Escolha fontes que tenham uma largura semelhante: Isso ajuda a garantir que o texto seja fácil de ler e que as palavras não sejam muito compactadas ou espaçadas.

7. Escolha fontes que tenham uma espessura semelhante. Isso ajuda a manter um equilíbrio visual entre o texto e a fonte, garantindo que a fonte não seja muito pesada ou muito leve em relação ao texto.

8. Escolha fontes que tenham uma personalidade semelhante. Isso pode incluir coisas como a forma dos caracteres, a inclinação das letras, ou outros detalhes estilísticos que fazem com que as fontes pareçam coesas quando usadas juntas.

9. Escolha fontes que tenham um contraste visual adequado. Isso significa escolher fontes que se destaquem o suficiente uma da outra, mas não tanto que causem distração ou dificultem a leitura.

Ao escolher fontes para usar juntas em um projeto, é importante ter em mente essas regras de emparelhamento para garantir que as fontes escolhidas sejam legíveis, visualmente agradáveis e combinem bem entre si.

Para encurtar o código, também é possível especificar todas as propriedades de fonte individuais em uma propriedade. A propriedade `font` é uma propriedade abreviada para:

- `font-style`
- `font-variant`
- `font-weight`
- `font-size/line-height`
- `font-family`

Os valores `font-size` e `font-family` são obrigatórios. Se um dos outros valores estiver ausente, seu valor padrão será usado.

Fontes CSS	Descrição
<code>font</code>	Define todas as propriedades da fonte em uma declaração
<code>font-family</code>	Especifica a família de fontes para texto

font-size	Especifica o tamanho da fonte do texto
font-style	Especifica o estilo de fonte do texto
font-variant	Especifica se um texto deve ou não ser exibido em uma fonte minúscula
font-weight	Especifica o peso de uma fonte

Ícones de CSS

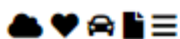
Os ícones podem ser facilmente adicionados à sua página HTML, usando uma biblioteca de ícones.

- **Fontes de ícones:** Utilizando fontes de ícones como Font Awesome, Glyphicons, etc.
- **Imagens:** Adicionando imagens através de URLs ou arquivos locais.
- **SVG:** Inserindo imagens SVG diretamente no HTML ou através de arquivos externos.
- **CSS pseudo-elementos:** Utilizando pseudo-elementos como :before ou :after para criar ícones usando CSS.

Como adicionar ícones? A maneira mais simples de adicionar um ícone à sua página HTML é com uma biblioteca de ícones, como Font Awesome. Adicione o nome da classe de ícone especificada a qualquer elemento HTML embutido (como <i> ou). Todos os ícones nas bibliotecas de ícones abaixo são vetores escaláveis que podem ser personalizados com CSS (tamanho, cor, sombra, etc.)

Fontes de Ícones Incríveis

Some Font Awesome icons:



Styled Font Awesome icons (size and color):



Para usar os ícones do Font Awesome, acesse fontawesome.com, faça login e obtenha um código para adicionar na <head> seção de sua página HTML:

```
<script src="https://kit.fontawesome.com/yourcode.js"
crossorigin="anonymous"></script>
```

```
<!DOCTYPE html>
<html>
<head>
<script src="https://kit.fontawesome.com/a076d05399.js"
crossorigin="anonymous"></script>
</head>
<body>

<i class="fas fa-cloud"></i>
<i class="fas fa-heart"></i>
<i class="fas fa-car"></i>
<i class="fas fa-file"></i>
<i class="fas fa-bars"></i>

</body>
</html>
```

Ícones do Bootstrap

Bootstrap icon library

Some Bootstrap icons:



Styled Bootstrap icons (size and color):



Para usar os glyphs do Bootstrap, adicione a seguinte linha dentro da <head>seção de sua página HTML:

<link rel="stylesheet"

href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">
</head>
<body>

<i class="glyphicon glyphicon-cloud"></i>
<i class="glyphicon glyphicon-remove"></i>
<i class="glyphicon glyphicon-user"></i>
<i class="glyphicon glyphicon-envelope"></i>
<i class="glyphicon glyphicon-thumbs-up"></i>

</body>
</html>
```

ADICIONAR ÍCONES

FONTES DE ÍCONES: UTILIZANDO FONTES DE ÍCONES COMO FONT AWESOME, GLYPHICONS, ETC

IMAGENS: ADICIONANDO IMAGENS ATRAVÉS DE URLS OU ARQUIVOS LOCAIS.

SVG: INSERINDO IMAGENS SVG DIRETAMENTE NO HTML OU ATRAVÉS DE ARQUIVOS EXTERNOS.

CSS PSEUDO-ELEMENTOS: UTILIZANDO PSEUDO-ELEMENTOS COMO :BEFORE OU :AFTER PARA CRIAR ÍCONES USANDO CSS.

Links CSS

Style a link with a color

This is a link

Com CSS, os links podem ser estilizados de várias maneiras diferentes. Os links podem ser estilizados com qualquer propriedade CSS (por exemplo color, font-family, background, etc.). Veja na imagem um link com a cor hotpink.

```
a {  
  color: hotpink;  
}
```

Além disso, os links podem ter estilos diferentes, dependendo do estado em que estão.

Os quatro estados dos links são:

- a:link- um link normal não visitado
- a:visited- um link que o usuário visitou
- a:hover- um link quando o usuário passa o mouse sobre ele
- a:active- um link no momento em que é clicado

```
/* unvisited link */  
a:link {  
  color: red;  
}  
  
/* visited link */  
a:visited {  
  color: green;  
}  
  
/* mouse over link */  
a:hover {  
  color: hotpink;  
}  
  
/* selected link */  
a:active {  
  color: blue;  
}
```

Ao definir o estilo para vários estados de link, existem algumas regras de ordem:

- a:hover DEVE vir depois de a:link e a:visited
- a:active DEVE vir depois de a:hover

Decoração de texto

A propriedade text-decoration é usada principalmente para remover sublinhados de links:

```
a:link {
  text-decoration: none;
}

a:visited {
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

a:active {
  text-decoration: underline;
}
```

Cor de fundo

A background-color propriedade pode ser usada para especificar uma cor de fundo para links:

```
a:link {
  background-color: yellow;
}

a:visited {
  background-color: cyan;
}

a:hover {
  background-color: lightgreen;
}

a:active {
  background-color: hotpink;
}
```

Botões de link

Este exemplo demonstra um exemplo mais avançado onde combinamos várias propriedades CSS para exibir links como caixas/botões:

```
a:link, a:visited {
  background-color: #f44336;
  color: white;
```

```
padding: 14px 25px;
text-align: center;
text-decoration: none;
display: inline-block;
}

a:hover, a:active {
  background-color: red;
}
```

Este exemplo demonstra como adicionar outros estilos aos hiperlinks:

```
a.one:link {color: #ff0000;}
a.one:visited {color: #0000ff;}
a.one:hover {color: #ffcc00;}

a.two:link {color: #ff0000;}
a.two:visited {color: #0000ff;}
a.two:hover {font-size: 150%;}

a.three:link {color: #ff0000;}
a.three:visited {color: #0000ff;}
a.three:hover {background: #66ff66;}

a.four:link {color: #ff0000;}
a.four:visited {color: #0000ff;}
a.four:hover {font-family: monospace;}

a.five:link {color: #ff0000; text-decoration: none;}
a.five:visited {color: #0000ff; text-decoration: none;}
a.five:hover {text-decoration: underline;}
```

Listas de CSS

Listas não ordenadas:

- Café
- Chá
- Coca Cola

- Café
- Chá
- Coca Cola

Listas Ordenadas:

1. Café
2. Chá
3. Coca Cola

- I. Café
- II. Chá
- III. Coca Cola

Listas HTML e propriedades da lista CSS

Em HTML, existem dois tipos principais de listas:

- listas não ordenadas () - os itens da lista são marcados com marcadores
- listas ordenadas () - os itens da lista são marcados com números ou letras

As propriedades da lista CSS permitem:

- Definir diferentes marcadores de itens de lista para listas ordenadas
- Definir diferentes marcadores de itens de lista para listas não ordenadas
- Definir uma imagem como marcador de item da lista
- Adicionar cores de fundo a listas e itens de lista

Marcadores de itens de lista diferentes

A propriedade `list-style-type` especifica o tipo de marcador de item de lista. O exemplo a seguir mostra alguns dos marcadores de item de lista disponíveis:

```
ul.a {  
  list-style-type: circle;  
}  
  
ul.b {  
  list-style-type: square;  
}  
  
ol.c {  
  list-style-type: upper-roman;  
}  
  
ol.d {  
  list-style-type: lower-alpha;  
}
```

Uma imagem como marcador de item de lista

A propriedade `list-style-image` especifica uma imagem como o marcador de item da lista:

```
ul {  
  list-style-image: url('sqpurple.gif');  
}
```

Posicione os Marcadores de Item da Lista

A propriedade `list-style-position` especifica a posição dos marcadores de item de lista (marcadores). "`list-style-position: outside;`" significa que os marcadores estarão fora do item da lista. O início de cada linha de um item de lista será alinhado verticalmente. Este é o padrão:

- Coffee - A brewed drink prepared from roasted coffee beans...
- Tea
- Coca-cola

"list-style-position: inside;" significa que os marcadores estarão **dentro do item da lista**. Como faz parte do item da lista, ele fará parte do texto e colocará o texto no início:

- Coffee - A brewed drink prepared from roasted coffee beans...
- Tea
- Coca-cola

```
ul.a {  
  list-style-position: outside;  
}  
  
ul.b {  
  list-style-position: inside;  
}
```

Remover configurações padrão

A list-style-type:none propriedade também pode ser usada para remover os marcadores/marcadores. Observe que a lista também possui margem e preenchimento padrão. Para remover isso, adicione margin:0 e padding:0 a ou :

```
ul {  
  list-style-type: none;  
  margin: 0;  
  padding: 0;  
}
```

Lista - Propriedade abreviada

A propriedade list-style é uma propriedade abreviada. É usado para definir todas as propriedades da lista em uma declaração:

```
ul {  
  list-style: square inside url("sqpurple.gif");  
}
```

Ao usar a propriedade abreviada, a ordem dos valores da propriedade é:

- list-style-type(se uma imagem de estilo de lista for especificada, o valor desta propriedade será exibido se a imagem por algum motivo não puder ser exibida)
- list-style-position(especifica se os marcadores de itens de lista devem aparecer dentro ou fora do fluxo de conteúdo)
- list-style-image(especifica uma imagem como marcador de item da lista)

Se um dos valores de propriedade acima estiver faltando, o valor padrão para a propriedade ausente será inserido, se houver.

Lista de estilos com cores

Também podemos estilizar listas com cores, para torná-las um pouco mais interessantes. Qualquer coisa adicionada à tag ou afeta a lista inteira, enquanto as propriedades adicionadas à tag afetarão os itens individuais da lista:

Propriedades de Listas CSS	Descrição
list-style	Define todas as propriedades de uma lista em uma declaração
list-style-image	Especifica uma imagem como marcador de item de lista
list-style-position	Especifica a posição dos marcadores de item de lista (marcadores)
list-style-type	Especifica o tipo de marcador de item de lista

Tabelas CSS

A aparência de uma tabela HTML pode ser muito melhorada com CSS! 😊 As tabelas em CSS podem ser estilizadas usando os seguintes conceitos:

- **CSS table properties:** Propriedades como "border-collapse", "border-spacing", "width", "height", etc. podem ser usadas para controlar o aspecto de uma tabela.
- **CSS table-cell properties:** Propriedades como "text-align", "vertical-align", "padding", etc. podem ser usadas para controlar o aspecto das células da tabela.
- **CSS pseudo-classes:** Pseudo-classes como ":hover" podem ser usadas para mudar o estilo da tabela quando o usuário passa o mouse sobre ela.
- **CSS media queries:** Media queries podem ser usadas para criar tabelas responsivas, que se adaptam a diferentes tamanhos de tela.

Ao estilizar tabelas em CSS, é importante lembrar que o estilo da tabela deve ser claro, fácil de ler e acessível para todos os usuários.

As **bordas da tabela** em CSS podem ser estilizadas usando as seguintes propriedades:

- **border-style:** define o estilo da borda (solid, dotted, double, etc.).
- **border-width:** define a largura da borda.
- **border-color:** define a cor da borda.
- **border-collapse:** define se as bordas das células compartilham espaço (collapse) ou não (separate).
- **border-spacing:** define o espaçamento entre as células na tabela.

Exemplo:

```
table {  
  border-collapse: collapse;  
  width: 100%;  
}  
  
th, td {  
  border: 1px solid black;  
  text-align: left;  
  padding: 8px;  
}  
  
th {  
  background-color: #f2f2f2;  
}
```

Neste exemplo, as bordas da tabela são definidas como "collapse", com uma largura de 1px e cor preta. As células da tabela têm alinhamento à esquerda e 8px de padding. A célula de cabeçalho tem uma cor de fundo cinza claro.

Uma tabela **full-width** é uma tabela que ocupa a largura total da tela, independentemente do tamanho da janela do navegador. Isso pode ser alcançado usando CSS com as seguintes etapas:

1. **Remover margens e padding da tabela:** Isso pode ser feito usando o seguinte código: `table { margin: 0; padding: 0; }.`
2. **Definir a largura da tabela como 100%:** Isso pode ser feito usando o seguinte código: `table { width: 100%; }.`
3. **Definir a largura de cada coluna da tabela:** Isso pode ser feito usando o seguinte código: `th, td { width: [porcentagem]; }.`, onde [porcentagem] é a porcentagem da largura total da tabela que cada coluna deve ocupar.
4. **Centralizar a tabela na tela:** Isso pode ser feito usando o seguinte código: `table { margin: 0 auto; }.`

```
table {  
  margin: 0 auto;  
  padding: 0;  
  width: 100%;  
}  
  
th, td {  
  width: 25%;  
  border: 1px solid black;  
  text-align: left;  
  padding: 8px;  
}  
  
th {  
  background-color: #f2f2f2;  
}
```

Neste exemplo, a tabela é definida com **margem automática, sem padding e largura total da tela**. Cada coluna ocupa 25% da largura total da tabela e tem bordas pretas, alinhamento à esquerda e 8px de padding. A célula de cabeçalho tem uma cor de fundo cinza claro.

Recolher bordas da tabela (border-collapse)

As bordas da tabela podem ser **colapsadas** usando a propriedade "**border-collapse**" em CSS. A propriedade "**border-collapse**" determina se as bordas das células da tabela compartilham espaço (collapse) ou não (separate).

Para colapsar as bordas da tabela, você pode usar o seguinte código:

```
table {  
  border-collapse: collapse;  
  width: 100%;  
}
```

```
}  
  
th, td {  
  border: 1px solid black;  
  text-align: left;  
  padding: 8px;  
}  
  
th {  
  background-color: #f2f2f2;  
}
```

Neste exemplo, as bordas da tabela são definidas como "collapse", ou seja, **compartilham espaço**. As células da tabela têm bordas pretas com largura de 1px, alinhamento à esquerda e 8px de padding. A célula de cabeçalho tem uma cor de fundo cinza claro.

Tamanho da Tabela CSS

O tamanho de uma tabela em CSS pode ser definido usando as seguintes propriedades:

- width: define a largura da tabela em pixels ou porcentagem.
- height: define a altura da tabela em pixels ou porcentagem.

```
table {  
  width: 75%;  
  height: 300px;  
  margin: 0 auto;  
  border-collapse: collapse;  
}  
  
th, td {  
  border: 1px solid black;  
  text-align: left;  
  padding: 8px;  
}  
  
th {  
  background-color: #f2f2f2;  
}
```

Neste exemplo, a tabela tem 75% da largura da tela e 300px de altura. As bordas da tabela são definidas como "collapse". As células da tabela têm bordas pretas com largura de 1px, alinhamento à esquerda e 8px de padding. A célula de cabeçalho tem uma cor de fundo cinza claro.

Alinhamento de Tabela CSS

O alinhamento de uma tabela em CSS pode ser definido usando as seguintes propriedades:

1. **margin**: define a margem ao redor da tabela.
2. **text-align**: define o alinhamento do texto nas células da tabela.
3. **vertical-align**: define o alinhamento vertical do conteúdo nas células da tabela.

Exemplo:

```
table {  
  width: 75%;  
  height: 300px;  
  margin: 0 auto;  
  border-collapse: collapse;  
  text-align: center;  
}  
  
th, td {  
  border: 1px solid black;  
  text-align: left;  
  padding: 8px;  
  vertical-align: middle;  
}  
  
th {  
  background-color: #f2f2f2;  
  text-align: center;  
}
```

Neste exemplo, a tabela tem 75% da largura da tela e 300px de altura. A tabela está centralizada na tela com margem automática. O texto nas células da tabela está alinhado à esquerda, enquanto o texto no cabeçalho está centralizado. O conteúdo nas células da tabela está alinhado ao centro verticalmente. As bordas da tabela são definidas como "collapse". As células da tabela têm bordas pretas com largura de 1px e 8px de padding. A célula de cabeçalho tem uma cor de fundo cinza claro.

Estilo de Tabela CSS

O estilo de uma tabela em CSS pode ser definido usando as seguintes propriedades:

1. **border**: define o estilo da borda da tabela.
2. **background-color**: define a cor de fundo da tabela.
3. **color**: define a cor do texto nas células da tabela.
4. **font-size**: define o tamanho da fonte do texto nas células da tabela.
5. **padding**: define o espaço entre o conteúdo e a borda da célula da tabela.

Exemplo:

```
table {
```

```
width: 75%;  
height: 300px;  
margin: 0 auto;  
border-collapse: collapse;  
text-align: center;  
border: 2px solid blue;  
background-color: #f2f2f2;  
color: black;  
font-size: 14px;  
}  
  
th, td {  
border: 1px solid blue;  
text-align: left;  
padding: 8px;  
vertical-align: middle;  
}  
  
th {  
background-color: blue;  
color: white;  
}
```

Neste exemplo, a tabela tem 75% da largura da tela e 300px de altura. A tabela está centralizada na tela com margem automática. A borda da tabela é azul com largura de 2px, a cor de fundo é cinza claro e a cor do texto é preta. O tamanho da fonte do texto nas células da tabela é 14px. As bordas das células da tabela são azul com largura de 1px, o texto nas células da tabela está alinhado à esquerda, o conteúdo nas células da tabela está alinhado ao centro verticalmente e há 8px de padding entre o conteúdo e a borda da célula da tabela. A célula de cabeçalho tem uma cor de fundo azul e a cor do texto é branca.

Tabela Responsiva CSS

A tabela responsiva é uma tabela que se adapta ao tamanho da tela, tornando-a legível em dispositivos de diferentes tamanhos, como computadores, tablets e smartphones. Isso é alcançado usando CSS (Folha de Estilo em cascata) e media queries. As media queries são usadas para detectar o tamanho da tela e aplicar estilos específicos para diferentes intervalos de tamanho de tela. Por exemplo, você pode fazer com que as colunas da tabela sejam exibidas uma abaixo da outra em dispositivos menores.

Propriedades de Tabelas CSS	Descrição
border	Define todas as propriedades de borda em uma declaração
border-collapse	Especifica se as bordas da tabela devem ou não ser recolhidas

border-spacing	Especifica a distância entre as bordas das células adjacentes
caption-side	Especifica o posicionamento de uma legenda de tabela
empty-cells	Especifica se deve ou não exibir bordas e fundo em células vazias em uma tabela
table-layout	Define o algoritmo de layout a ser usado para uma tabela

Layout CSS - A propriedade de exibição

A propriedade de exibição `display` especifica se/como um elemento é exibido. Cada elemento HTML tem um valor de exibição padrão, dependendo do tipo de elemento. O valor de exibição padrão para a maioria dos elementos é `block` ou `inline`.

Elementos de nível de bloco

Um elemento de nível de bloco sempre começa em uma nova linha e ocupa toda a largura disponível (estende-se para a esquerda e para a direita o máximo possível).

O elemento `<div>` é um elemento de nível de bloco. Exemplos de elementos de nível de bloco:

ELEMENTOS DE NÍVEL DE BLOCO						
<code><DIV></code>	<code><H1></code> - <code><H6></code>	<code><P></code>	<code><FORMA></code>	<code><CABEÇALHO></code>	<code><RODAPÉ></code>	<code><SEÇÃO></code>

Elementos embutidos

Um elemento `inline` não começa em uma nova linha e ocupa apenas a largura necessária. Exemplos de elementos embutidos:

ELEMENTOS EMBUTIDOS		
<code></code>	<code><A></code>	<code></code>

`display: none;` é comumente usado com JavaScript para **ocultar e mostrar elementos sem excluí-los e recriá-los**. `display: none` é uma propriedade de CSS que **especifica que um elemento HTML não deve ser exibido na página**. Esse valor oculta completamente o elemento, incluindo todo o espaço que ele ocuparia.

Ao usar `display: none`, o elemento não é renderizado pelo navegador, o que pode ajudar a otimizar o desempenho da página. Além disso, você pode usar JavaScript para alterar o valor de `display` de um elemento de `none` para `block` ou outro valor de exibição, mostrando ou ocultando o elemento dinamicamente.

Note que o elemento com `display: none` ainda existe no DOM, mas não é visível na página. Se você precisar ocultar um elemento apenas temporariamente ou alternar sua exibição, pode usar outras propriedades, como `visibility` ou `opacity`, em vez de `display: none`.

display: inline é um valor de CSS que especifica que um elemento HTML deve ser exibido como uma linha de texto. Isso significa que o elemento ocupará apenas o espaço necessário para exibir seu conteúdo, sem quebrar para a próxima linha.

Ao usar **display: inline**, o elemento é exibido na mesma linha que outros elementos inline ao seu redor, como links e palavras em negrito. Além disso, o tamanho do elemento é determinado pelo seu conteúdo, e as propriedades **width** e **height** não são aplicadas.

Note que os elementos inline só podem conter outros elementos inline, e não elementos de bloco, como parágrafos ou **divs**. Se você precisar exibir um elemento como um bloco, pode usar **display: block** ou **display: inline-block**.

display: block é um valor de CSS que especifica que um elemento HTML deve ser exibido como um bloco de conteúdo. Isso significa que o elemento ocupará toda a largura disponível de seu container e será exibido em uma nova linha, separado dos outros elementos.

Ao usar **display: block**, o elemento é exibido como um bloco independente, com largura e altura definidas pelas propriedades **width** e **height**, ou automaticamente pelo conteúdo. Além disso, você pode controlar a margem, borda e **padding** do elemento para definir o espaçamento em torno dele.

Note que os elementos de bloco só podem conter outros elementos de bloco ou elementos inline, e não elementos inline como links ou palavras em negrito. Se você precisar exibir um elemento como uma linha, pode usar **display: inline** ou **display: inline-block**.

visibility: hidden é uma propriedade de CSS que especifica que um elemento HTML deve ser oculto na página, mas ainda ocupar o espaço que ele ocuparia se estivesse visível. Isso significa que os elementos adjacentes a ele serão realocados para preencher o espaço vago.

Ao usar **visibility: hidden**, o elemento é renderizado pelo navegador, mas fica invisível. Além disso, você pode usar JavaScript para alterar o valor de **visibility** de um elemento de **hidden** para **visible**, mostrando o elemento dinamicamente.

Note que o elemento com **visibility: hidden** ainda existe no DOM e pode afetar o layout da página. Se você precisar remover completamente um elemento da página, pode usar a propriedade **display: none**.

display: flex é um valor de CSS que transforma um elemento HTML em um container flexível. Isso significa que os elementos filhos dentro desse container podem ser alinhados em diferentes direções e dimensionados de acordo com a largura ou altura disponíveis.

Ao usar **display: flex**, você pode controlar como os elementos filhos são dispostos usando propriedades CSS como **justify-content**, **align-items**, **flex-wrap** e **flex-direction**. Por exemplo, você pode centralizar elementos filhos vertical e horizontalmente, controlar se eles devem quebrar para a próxima linha ou não, e definir a direção do fluxo dos elementos.

O uso de **display: flex** é muito útil para **criar layouts flexíveis e responsivos**, onde os elementos precisam se ajustar a diferentes tamanhos de tela e resoluções de dispositivos.

Substituir o valor de exibição padrão

Conforme mencionado, cada elemento tem um valor de exibição padrão. No entanto, você pode substituir isso. Alterar um elemento inline para um elemento de bloco, ou vice-versa, pode ser útil para fazer a página parecer de uma maneira específica e ainda seguir os padrões da web. Um exemplo comum é criar elementos embutidos `<li>` para menus horizontais:

Propriedades de Exibição CSS	Descrição
<code>display</code>	Especifica como um elemento deve ser exibido
<code>visibility</code>	Especifica se um elemento deve ou não ser visível

Largura e Largura Máxima

Um elemento de nível de bloco **sempre ocupa toda a largura disponível** (estende-se para a esquerda e para a direita o máximo possível). Definir o `width` de um elemento de nível de bloco **impedirá que ele se estenda até as bordas** de seu contêiner. Em seguida, você pode definir as margens como automáticas, para centralizar horizontalmente o elemento em seu contêiner. O elemento ocupará a largura especificada e o espaço restante será dividido igualmente entre as duas margens:

Usar `max-width` em vez disso, nessa situação, **melhorará o manuseio de janelas** pequenas pelo navegador. Isso é importante ao tornar um site utilizável em pequenos dispositivos.

A propriedade de posição

A propriedade CSS "position" é usada para definir como um elemento HTML é posicionado em relação ao seu elemento pai ou à página (estático, relativo, fixo, absoluto ou fixo).

. As opções de posição incluem:

- **static**: o elemento é posicionado de acordo com o fluxo normal do documento (padrão)
- **relative**: o elemento é posicionado em relação a sua posição normal, permitindo o uso de "top", "right", "bottom" e "left" para mover o elemento a partir da sua posição original
- **fixed**: o elemento é posicionado em relação à janela do navegador e não se move quando a página é rolada.
- **absolute**: o elemento é posicionado em relação ao primeiro elemento pai com posição não-estática (se não houver, será posicionado em relação à janela do navegador)
- **sticky**: o elemento é posicionado de forma semelhante ao "relative" até que ele atinja uma certa posição na tela, a partir daí ele se comporta como "fixed".

Os elementos são então posicionados usando as propriedades **superior, inferior, esquerda e direita**. No entanto, essas propriedades não funcionarão, a menos que a propriedade **position** seja definida primeiro. Eles também funcionam de forma diferente, dependendo do valor da posição.

position: static;

Os elementos HTML são posicionados estáticos por padrão. Os elementos posicionados estáticos **não são afetados pelas propriedades superior, inferior, esquerda e direita**.

Um elemento com position: static; não é posicionado de nenhuma maneira especial; está sempre posicionado de acordo com o fluxo normal da página:

position: relative;

Um elemento com position: relative; é posicionado em relação à sua posição normal.

Definir as propriedades superior, direita, inferior e esquerda de um elemento relativamente posicionado fará com que ele seja ajustado para longe de sua posição normal. **Outros conteúdos não serão ajustados para caber em qualquer lacuna deixada pelo elemento.**

position: fixed;

Um elemento com position: fixed; é posicionado em relação à viewport, o que significa que ele sempre permanece no mesmo lugar, mesmo que a página seja rolada. As propriedades superior, direita, inferior e esquerda são usadas para posicionar o elemento. **Um elemento fixo não deixa uma lacuna na página onde normalmente estaria localizado.**

position: absolute;

Um elemento com `position: absolute;` é posicionado em relação ao ancestral posicionado mais próximo (em vez de posicionado em relação à viewport, como fixo).

No entanto; se um elemento posicionado absoluto não tiver ancestrais posicionados, ele usará o corpo do documento e se moverá junto com a rolagem da página.

`position: sticky;`

Um elemento com `position: sticky;` é posicionado com base na posição de rolagem do usuário. Um elemento fixo alterna entre `relative` e `fixed`, dependendo da posição de rolagem. Ele é posicionado relativo até que uma determinada posição de deslocamento seja encontrada na viewport - então ele "**gruda**" no lugar (como `position: fixed`).

Aaaantes de partir para o próximo tópico, vamos retomar as propriedades de posição em CSS! Essas propriedades permitem controlar a localização de elementos HTML em relação a outros elementos ou à página. As principais propriedades de posição são "**position**", "**top**", "**right**", "**bottom**" e "**left**". A propriedade "**position**" define se um elemento é posicionado de acordo com o fluxo normal do documento (static), em relação a sua posição normal (relative), em relação ao primeiro elemento pai com posição não-estática (absolute) ou fixo em relação à janela do navegador (fixed). As propriedades "**top**", "**right**", "**bottom**" e "**left**" são usadas para ajustar a posição de um elemento em relação ao seu elemento pai ou à página.

Propriedades de Posição CSS	Descrição
bottom	Define a borda da margem inferior para uma caixa posicionada
clip	Corta um elemento absolutamente posicionado
left	Define a borda da margem esquerda para uma caixa posicionada
position	Especifica o tipo de posicionamento de um elemento
right	Define a borda da margem direita para uma caixa posicionada
top	Define a borda da margem superior para uma caixa posicionada

A propriedade z-index

A propriedade CSS "z-index" é usada para controlar a ordem de sobreposição de elementos HTML na tela. Elementos com um valor de "z-index" mais alto são exibidos "acima" de elementos com um valor de "z-index" mais baixo. A propriedade "z-index" só tem efeito se a propriedade "position" for definida como "absolute", "relative" ou "fixed". O valor padrão é "auto", o que significa que o navegador decide a ordem de sobreposição baseado no fluxo normal do documento. Valores positivos colocam o elemento mais à frente, enquanto valores negativos colocam o elemento mais atrás.

Sem índice z! Mas prof, o que acontece se dois elementos posicionados se sobrepõem sem um z-index especificado? Neste caso, o elemento definido por último no código HTML será mostrado no topo.

Para usar a propriedade "z-index", você precisa primeiro definir a propriedade "position" do elemento como "absolute", "relative" ou "fixed". Em seguida, defina o "z-index" com um número inteiro. Valores positivos colocam o elemento mais à frente, enquanto valores negativos o colocam mais atrás. O valor padrão é "auto", o que significa que o navegador decidirá a ordem de sobreposição baseado no fluxo normal do documento.

Observe que o "z-index" só tem efeito em elementos que estão em posições diferentes. Se dois elementos tiverem a mesma posição, o navegador decidirá qual deles está mais à frente com base na ordem deles no código HTML. Além disso, o "z-index" não funcionará em elementos estáticos (com "position: static").

(FCC – TRT - 4ª REGIÃO (RS) – 2022) Quando um Técnico criou uma página web usando HTML e CSS, um elemento ficou sobreposto ao outro na tela e ele deseja inverter a ordem de sobreposição desses elementos. Para definir ou mudar a ordem de sobreposição dos elementos ele deve utilizar a propriedade CSS

- a) z-index
- b) inverter
- c) position
- d) flex-position
- e) x-order

Comentários: Quando se fala em ordem de sobreposição de elementos HTML estamos falando sobre a propriedade z-index. Lembre-se que A propriedade CSS "z-index" é usada para controlar a ordem de sobreposição de elementos HTML na tela. Elementos com um valor de "z-index" mais alto são exibidos "acima" de elementos com um valor de "z-index" mais baixo. (Gabarito: Letra A)

Overflow

A propriedade CSS "overflow" controla o que acontece quando o conteúdo de um elemento HTML excede suas dimensões definidas, ou seja, **conteúdo é grande demais** para caber em uma área. A propriedade "overflow" pode ter os seguintes valores:

- **visible**: o conteúdo é exibido fora do elemento, sem cortar ou rolar.
- **hidden**: o conteúdo é cortado, ou seja, o que ultrapassar as bordas do elemento não será exibido.
- **scroll**: uma barra de rolagem é adicionada ao elemento, permitindo que o usuário role e veja o conteúdo que está fora das dimensões visíveis.
- **auto**: se o conteúdo ultrapassar as dimensões do elemento, uma barra de rolagem será adicionada. Se o conteúdo couber, nenhuma barra de rolagem será exibida.

Observe que o "overflow" pode ser definido tanto para o **eixo x (horizontal)** quanto para o **eixo y (vertical)**, usando as propriedades "**overflow-x**" e "**overflow-y**". Além disso, é possível usar a propriedade "**overflow: scroll**" para forçar a exibição de uma barra de rolagem, mesmo que o conteúdo não ultrapasse as dimensões do elemento.

A propriedade overflow especifica se deve cortar o conteúdo ou adicionar barras de rolagem quando o conteúdo de um elemento for muito grande para caber na área especificada.

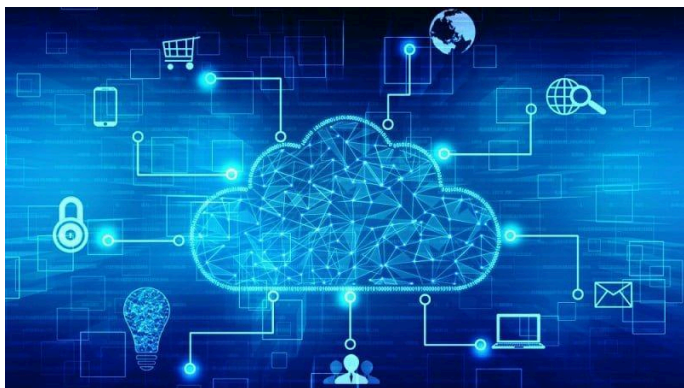
Por padrão, o overflow é **visible**, o que significa que **não é recortado** e é renderizado fora da caixa do elemento. Por outro lado, com o valor **hidden**, o overflow é recortado e o restante do conteúdo é ocultado.

Ademais, definindo o valor como **scroll**, o **overflow é cortado e uma barra de rolagem** é adicionada para rolar dentro da caixa. Observe que isso **adicionará uma barra de rolagem horizontal e verticalmente** (mesmo que você não precise dela)

O valor **auto** é semelhante a **scroll**, mas adiciona barras de rolagem somente quando necessário.

Propriedades de Posição CSS	Descrição
overflow	Especifica o que acontece se o conteúdo ultrapassar a caixa de um elemento
overflow-wrap	Especifica se o navegador pode ou não quebrar linhas com palavras longas, caso elas transbordem seu contêiner
overflow-x	Especifica o que fazer com as bordas esquerda/direita do conteúdo se ele ultrapassar a área de conteúdo do elemento
overflow-y	Especifica o que fazer com as bordas superior/inferior do conteúdo se ele ultrapassar a área de conteúdo do elemento

Layout CSS: Float and clear



As propriedades CSS "float" e "clear" são usadas para controlar o layout de elementos HTML na página. A propriedade "float" define se um elemento deve flutuar à esquerda ou à direita, (a imagem ao lado está a esquerda!) permitindo que outros elementos sejam exibidos ao seu lado. Por exemplo, uma imagem pode ser definida com "float: left" para que o texto ao seu lado seja exibido ao seu redor. Se vários elementos tiverem a

propriedade "float", eles serão organizados lado a lado, na ordem em que aparecem no código HTML.

A propriedade CSS "float" pode ter três valores principais:

- "left": flutua o elemento para a esquerda e o posiciona na linha horizontal, permitindo que outros elementos sejam exibidos ao seu lado, na direita.
- "right": flutua o elemento para a direita e o posiciona na linha horizontal, permitindo que outros elementos sejam exibidos ao seu lado, na esquerda.
- "none": não flutua o elemento e mantém-no no fluxo normal da página.

Além desses valores, a propriedade "float" também pode ter o valor "inherit", que herda o valor da propriedade "float" do elemento pai.

A propriedade "float: left" é comumente utilizada para posicionar elementos HTML na linha horizontalmente, flutuando-os para a esquerda. Quando um elemento é definido com "float: left", ele é retirado do fluxo normal da página e posicionado na esquerda da página, ocupando apenas o espaço necessário para o seu conteúdo. Os elementos subsequentes são então exibidos ao seu lado, na direita. É importante destacar que elementos com "float" não afetam o tamanho de seus elementos pais, por isso é comum utilizar a propriedade "clear" em conjunto para garantir que outros elementos não fiquem sobrepostos aos elementos flutuantes. Assim também ocorre para o "float:right".

A propriedade "clear" define se um elemento deve ser colocado abaixo de elementos flutuantes anteriores. Por exemplo, se você tem uma imagem flutuando à esquerda e deseja que o próximo parágrafo comece abaixo da imagem, você pode definir "clear: left" para o parágrafo. A propriedade "clear" pode ter os valores "left", "right", "both" ou "none". (A imagem agora está à direita!)



Observe que a propriedade "float" é útil para criar layouts simples, mas pode ser difícil de controlar em situações mais complexas. Alternativas mais modernas incluem o uso de [flexbox](#) e [grid](#).

Frisando que a propriedade float pode ter [um dos seguintes valores](#):

- [left](#)- O elemento flutua à esquerda de seu contêiner
- [right](#)- O elemento flutua à direita de seu contêiner
- [none](#)- O elemento não flutua (será exibido apenas onde ocorre no texto). Isso é padrão
- [inherit](#)- O elemento herda o valor float de seu pai

Em seu uso mais simples, a propriedade [float](#) pode ser usada para quebrar texto em torno de imagens.

A propriedade CSS "clear"

A propriedade CSS "clear" é usada para controlar a posição de um elemento em relação a elementos flutuantes anteriores. A propriedade "clear" tem os seguintes valores:

- [left](#): o elemento não pode ser colocado à esquerda de um elemento flutuante.
- [right](#): o elemento não pode ser colocado à direita de um elemento flutuante.
- [both](#): o elemento não pode ser colocado à esquerda ou à direita de um elemento flutuante.
- [none](#): o elemento pode ser colocado ao lado de um elemento flutuante.

Por exemplo, se você tem uma imagem flutuando à esquerda e deseja que o próximo parágrafo comece abaixo da imagem, você pode definir "clear: left" para o parágrafo. Isso garantirá que o parágrafo não seja colocado ao lado da imagem, mas sim abaixo dela.

Por fim, vamos relembrar, [float](#) especifica se um elemento deve flutuar à esquerda, à direita ou não flutuar. Valores possíveis são "left", "right" ou "none".

Por outro lado, clear especifica se um elemento deve ser colocado abaixo de elementos flutuantes anteriores. Valores possíveis são "left", "right", "both" ou "none".

Observe que estas são as [duas principais propriedades relacionadas a "float"](#). Há outras propriedades menos comuns, como "float-offset", que são usadas em situações específicas. É importante ter em mente que o uso de "float" para o layout pode ser limitante em algumas situações e alternativas mais modernas, como o uso de flexbox e grid, devem ser consideradas.

Propriedades de Posição CSS	Descrição
box-sizing	Define como a largura e a altura de um elemento são calculadas: devem incluir preenchimento e bordas ou não
clear	Especifica o que deve acontecer com o elemento que está ao lado de um elemento flutuante

float	Especifica se um elemento deve flutuar para a esquerda, direita ou nada
overflow	Especifica o que acontece se o conteúdo ultrapassar a caixa de um elemento
overflow-x	Especifica o que fazer com as bordas esquerda/direita do conteúdo se ele ultrapassar a área de conteúdo do elemento
overflow-y	Especifica o que fazer com as bordas superior/inferior do conteúdo se ele ultrapassar a área de conteúdo do elemento

Layout CSS: display: inline-block

O valor CSS "display: inline-block" é usado para criar um elemento que se comporta como um elemento em linha (inline), mas que também pode ter uma largura e altura definidas, como um elemento em bloco (block).

display: inline-block é uma propriedade CSS que especifica o comportamento de exibição de um elemento HTML. O valor inline-block cria um contêiner de bloco de nível inline. Os elementos com esse valor são colocados alinhados com o texto e outros elementos, mas se comportam como um elemento de nível de bloco, permitindo que largura, altura e preenchimento sejam definidos. Isso torna o bloco embutido útil para criar elementos flexíveis e de tamanho ajustável que podem ser usados em layouts com muito texto.

"inline-block" é uma opção útil quando você precisa que um **elemento se comporte como um bloco, mas ainda precisa que ele seja exibido na linha**, como quando você precisa alinhar elementos horizontalmente. Além disso, ele também permite que você especifique a largura e a altura de um elemento, bem como ajustes de margem, padding e borda, o que não é possível com elementos "inline".

Normalmente, os elementos em bloco preenchem toda a largura da linha, mas com "display: inline-block", o elemento pode ser definido com uma largura específica, permitindo que outros elementos sejam posicionados ao lado dele na mesma linha. Isso pode ser útil para criar layouts de página com elementos alinhados horizontalmente.

O "display: inline-block" também permite que o elemento tenha margens, preenchimento e bordas, ao contrário dos elementos em linha que não podem ter largura e altura definidas, nem bordas ou margens.

No entanto, é importante lembrar que o "display: inline-block" adiciona um espaçamento em branco à direita do elemento, que pode causar problemas de layout. Para evitar isso, é possível remover o espaço em branco ou definir uma largura menor para o elemento.

Além disso, é importante lembrar que o "display: inline-block" não é a solução ideal para todos os layouts de página e pode ter limitações em relação à flexibilidade do layout. Em vez disso, pode ser necessário combinar várias propriedades CSS para obter o resultado desejado.

Os valores CSS "inline", "inline-block" e "block" são usados para definir **como um elemento HTML é exibido na página**.

- "display: inline" é usado para elementos que devem ser exibidos em uma linha, como texto ou links. Isso significa que eles não podem ter uma largura ou altura definida e não podem ter margens ou preenchimentos em todas as direções. Os elementos em linha são dimensionados automaticamente com base no conteúdo que eles contêm.
- "display: inline-block" é usado para elementos que precisam de uma largura e altura definidas, mas também precisam ser exibidos em uma linha. Isso permite que esses elementos sejam posicionados horizontalmente, permitindo que outros elementos sejam

posicionados ao lado deles na mesma linha. Eles também podem ter margens, preenchimento e bordas definidos.

- "display: block" é usado para elementos que devem ser exibidos em blocos separados, ocupando toda a largura disponível. Isso significa que eles podem ter uma largura e altura definidas e podem ter margens, preenchimento e bordas em todas as direções. Os elementos em bloco são geralmente usados para definir seções maiores de conteúdo na página, como parágrafos, cabeçalhos, listas e divs.

Em resumo, "inline" é usado para elementos que devem ser exibidos em uma linha, "inline-block" é usado para elementos que precisam de uma largura e altura definidas e "block" é usado para elementos que devem ser exibidos em blocos separados, ocupando toda a largura disponível.

Alinhamento

Alinhamento horizontal:

- **text-align:** Alinha o texto dentro de um elemento (esquerda, direita, centralizado ou justificado).
- **margin:** Adiciona espaço ao redor de um elemento, que pode ser usado para centralizar horizontalmente um elemento definindo margin: auto em ambos os lados.
- **display: flex e justify-content:** quando aplicado a um elemento de contêiner, isso cria um layout de flexbox e permite o alinhamento horizontal de elementos filho usando a propriedade de **justify-content** (esquerda, direita, centro, espaço entre, espaço ao redor).

Alinhamento vertical:

- **vertical-align:** Alinha elementos de nível embutido e célula de tabela (topo, meio, fundo).
- **line-height:** especifica a altura de uma linha de texto, que pode ser usada para centralizar o texto verticalmente em um elemento.
- **display: flex and align-items:** quando aplicado a um elemento container, isso cria um layout flexbox e permite o alinhamento vertical de elementos filho usando a propriedade align-items (top, middle, bottom, stretch).
- **display: table-cell e vertical-align:** Definir um container e seus filhos para display: table-cell e vertical-align permite o alinhamento vertical como uma tabela.

Elementos de Alinhamento Central

Existem várias maneiras de alinhar elementos ao centro de uma página ou contêiner usando CSS, incluindo:

Elementos de Alinhamento Central	Descrição
Margens automáticas	Definir margens automáticas para os lados esquerdo e direito de um elemento fará com que ele seja centralizado horizontalmente <code>margin: 0 auto;</code>
Transformação	É possível usar a propriedade "transform" para centralizar vertical e horizontalmente um elemento. <code>transform: translate(-50%, -50%);</code>
Flexbox	Usando a propriedade "display: flex" em um contêiner, é possível centralizar os itens dentro dele usando as propriedades "justify-content" e "align-items". Por exemplo: <code>.container {</code>

	<pre>display: flex; justify-content: center; align-items: center; }</pre>
Grid	Usando a propriedade "display: grid" em um contêiner, é possível centralizar os itens dentro dele usando a propriedade "place-items". Por exemplo: <pre>.container { display: grid; place-items: center; }</pre>

CSS Combinators

Combinadores CSS são símbolos usados para combinar seletores para direcionar elementos específicos em uma página da web. Eles permitem selecionar elementos com base em suas relações com outros elementos. Existem quatro tipos de combinadores:

1. Combinador descendente (espaço): tem como alvo um elemento que é descendente de outro elemento. Exemplo: `div p` visa todos os elementos `p` que são descendentes de um elemento `div`.
2. Combinador filho (`>`): tem como alvo um elemento que é filho direto de outro elemento. Exemplo: `div > p` tem como alvo todos os elementos `p` que são filhos diretos de um elemento `div`.
3. Combinador irmão adjacente (`+`): tem como alvo um elemento que é imediatamente adjacente a outro elemento e tem o mesmo pai. Exemplo: `h1 + p` tem como alvo o primeiro elemento `p` imediatamente após um elemento `h1`.
4. Combinador irmão geral (`~`): tem como alvo um elemento que é irmão de outro elemento e tem o mesmo pai. Exemplo: `h1 ~ p` tem como alvo todos os elementos `p` que são irmãos de um elemento `h1`.

Esses combinadores podem ser usados para aumentar a especificidade dos seletores CSS e para direcionar elementos específicos em uma página de maneira mais precisa. Os combinators podem ser usados em conjunto com seletores de classe, ID e atributo para criar seletores mais específicos e precisos. Eles são uma ferramenta poderosa para selecionar e estilizar elementos em uma página da web.

Seletores CSS	Exemplo	Descrição do Exemplo
<code>element element</code>	<code>div p</code>	Seleciona todos os elementos <code><p></code> dentro dos elementos <code><div></code>
<code>element>element</code>	<code>div > p</code>	Seleciona todos os elementos <code><p></code> onde o pai é um elemento <code><div></code>
<code>element+element</code>	<code>div + p</code>	Seleciona o primeiro elemento <code><p></code> que é colocado imediatamente após os elementos <code><div></code>
<code>element1~element2</code>	<code>p ~ ul</code>	Seleciona todos os elementos <code></code> precedidos por um elemento <code><p></code>

Pseudo-classes

O que são pseudo-classes? Uma pseudoclasse é usada para definir um estado especial de um elemento. Por exemplo, pode ser usado para:

- Estilizar um elemento quando um usuário passa o mouse sobre ele
- Estilizar links visitados e não visitados de forma diferente
- Estilizar um elemento quando ele recebe o foco

Pseudo-classes são pseudo-elementos que permitem selecionar elementos baseados em algum estado específico, como `:hover`, `:active`, `:focus`, etc. Em outras palavras, pseudo-classes permitem selecionar elementos HTML que não são identificados por sua tag, classe ou ID, mas por sua posição ou estado dentro da página.

Aqui estão alguns exemplos de pseudo-classes comuns:

- `:hover`: seleciona o elemento quando o cursor do mouse estiver sobre ele.
- `:active`: seleciona o elemento quando ele estiver ativo, ou seja, quando o usuário clicou ou tocou nele.
- `:focus`: seleciona o elemento quando ele estiver em foco, ou seja, quando ele estiver selecionado para receber entrada de teclado.
- `:first-child`: seleciona o primeiro elemento filho de seu pai.
- `:last-child`: seleciona o último elemento filho de seu pai.

Pseudo-classes são usadas em conjunto com outros seletores, como tags HTML, classes ou ID's, para aumentar a especificidade de um seletor CSS e aplicar estilos a elementos HTML específicos em determinadas situações.

Seletores	Exemplo	Descrição do Exemplo
<code>:active</code>	<code>a:active</code>	Seleciona o link ativo
<code>:checked</code>	<code>input:checked</code>	Seleciona cada elemento <code><input></code> marcado
<code>:disabled</code>	<code>input:disabled</code>	Seleciona cada elemento <code><input></code> desabilitado
<code>:empty</code>	<code>p:empty</code>	Seleciona cada elemento <code><p></code> que não tem filhos
<code>:enabled</code>	<code>input:enabled</code>	Seleciona cada elemento <code><input></code> ativado
<code>:first-child</code>	<code>p:first-child</code>	Seleciona cada elemento <code><p></code> que é o primeiro filho de seu pai
<code>:first-of-type</code>	<code>p:first-of-type</code>	Seleciona cada elemento <code><p></code> que é o primeiro elemento <code><p></code> de seu pai

:focus	input:focus	Seleciona o elemento <input> que tem foco
:hover	a:hover	Seleciona links ao passar o mouse
:in-range	input:in-range	Seleciona elementos <input> com um valor dentro de um intervalo especificado
:invalid	input:invalid	Seleciona todos os elementos <input> com um valor inválido
:lang(language)	p:lang(it)	Seleciona cada elemento <p> com um valor de atributo lang começando com "it"
:last-child	p:last-child	Seleciona cada elemento <p> que é o último filho de seu pai
:last-of-type	p:last-of-type	Seleciona cada elemento <p> que é o último elemento <p> de seu pai
:link	a:link	Seleciona todos os links não visitados
:not(selector)	:not(p)	Seleciona todos os elementos que não são um elemento <p>
:nth-child(n)	p:nth-child(2)	Seleciona cada elemento <p> que é o segundo filho de seu pai
:nth-last-child(n)	p:nth-last-child(2)	Seleciona cada elemento <p> que é o segundo filho de seu pai, contando a partir do último filho
:nth-last-of-type(n)	p:nth-last-of-type(2)	Seleciona cada elemento <p> que é o segundo elemento <p> de seu pai, contando a partir do último filho
:nth-of-type(n)	p:nth-of-type(2)	Seleciona cada elemento <p> que é o segundo elemento <p> de seu pai
:only-of-type	p:only-of-type	Seleciona cada elemento <p> que é o único elemento <p> de seu pai
:only-child	p:only-child	Seleciona cada elemento <p> que é o único filho de seu pai
:optional	input:optional	Seleciona elementos <input> sem atributo "obrigatório"

:out-of-range	input:out-of-range	Seleciona elementos <input> com um valor fora de um intervalo especificado
:read-only	input:read-only	Seleciona elementos <input> com um atributo "readonly" especificado
:read-write	input:read-write	Seleciona elementos <input> sem o atributo "readonly"
:required	input:required	Seleciona elementos <input> com um atributo "obrigatório" especificado
:root	root	Seleciona o elemento raiz do documento
:target	#news:target	Seleciona o elemento #news ativo atual (clicado em uma URL contendo o nome da âncora)
:valid	input:valid	Seleciona todos os elementos <input> com um valor válido
:visited	a:visited	Seleciona todos os links visitados

Opacidade / Transparência

A propriedade `opacity` em CSS é usada para controlar a transparência de um elemento. Ela pode ter um valor entre 0 (totalmente transparente) e 1 (totalmente opaco). Um elemento com `opacity` definido como 0.5 será metade transparente, ou seja, metade visível e metade invisível.

Aqui está um exemplo de como usar a propriedade `opacity` em um elemento:

```
div {  
  opacity: 0.5;  
}
```

É importante lembrar que a opacidade é herdada pelos elementos filhos de um elemento com opacidade definida. Portanto, se um elemento pai tiver `opacity` definido como 0.5, todos os seus elementos filhos também terão `opacity` de 0.5, a menos que sejam sobrescritos com uma opacidade diferente.

A propriedade `opacity` é útil para criar efeitos de transparência e para tornar elementos menos opacos, permitindo que o fundo da página seja visto através deles.

Pseudo-elementos

Pseudo-elementos são elementos fictícios que permitem adicionar conteúdo a elementos HTML sem modificar o conteúdo do documento. Eles são escritos usando dois pontos (:) seguidos pelo nome do pseudo-elemento. Aqui estão alguns exemplos de pseudo-elementos comuns:

- **::before**: adiciona conteúdo antes do conteúdo principal de um elemento.
- **::after**: adiciona conteúdo depois do conteúdo principal de um elemento.
- **::first-letter**: seleciona a primeira letra de um elemento de texto.
- **::first-line**: seleciona a primeira linha de um elemento de texto.

Pseudo-elementos são úteis para adicionar conteúdo a elementos HTML, como adicionar ícones a links, adicionar símbolos a itens de lista, adicionar cabeçalhos a caixas de texto, etc. Eles também são usados para aplicar estilos diferentes a partes específicas de um elemento, como mudar a cor da primeira letra de um parágrafo ou adicionar uma seta antes de um link.

O pseudo-elemento de **::first-line**

O pseudo-elemento **::first-line** é usado para adicionar um estilo especial à primeira linha de um texto. As seguintes propriedades se aplicam ao pseudo-elemento **::first-line**:

- font properties
- color properties
- background properties
- word-spacing
- letter-spacing
- text-decoration
- vertical-align
- text-transform
- line-height
- clear

O pseudo-elemento **::first-letter** é usado para adicionar um estilo especial à primeira letra de um texto. As seguintes propriedades se aplicam ao pseudo-elemento **::first-letter**:

- font properties
- color properties
- background properties
- margin properties
- padding properties
- border properties
- text-decoration
- vertical-align (only if "float" is "none")
- text-transform
- line-height
- float
- clear

Todos os pseudoelementos CSS

Existem **vários pseudoelementos** disponíveis em CSS que permitem estilizar elementos com base em sua posição em relação a outros elementos ou em seu estado. Abaixo está uma lista completa dos pseudoelementos CSS, de acordo com o W3Schools:

Seletores	Exemplo	Descrição do Exemplo
::after	p::after	Inserir conteúdo após cada elemento <p>
::before	p::before	Inserir conteúdo antes de cada elemento <p>
::first-letter	p::first-letter	Seleciona a primeira letra de cada elemento <p>
::first-line	p::first-line	Seleciona a primeira linha de cada elemento <p>
::marker	::marker	Seleciona os marcadores dos itens da lista
::selection	p::selection	Seleciona a parte de um elemento que é selecionado por um usuário

É importante lembrar que os pseudoelementos são diferentes das pseudoclasses, que são usados para estilizar elementos com base em seu estado (por exemplo, :hover para estilizar um elemento quando o cursor do mouse está sobre ele). Os pseudoelementos permitem estilizar partes específicas de um elemento, enquanto as pseudoclasses permitem estilizar o elemento como um todo com base em seu estado.

Barra de Navegação

A barra de navegação (também conhecida como "navbar") é um elemento comum em páginas da web que fornece uma forma de navegar pelo site. É geralmente exibido no topo da página e contém links para as seções ou páginas do site.

A barra de navegação pode ser implementada usando HTML e CSS. Aqui está um exemplo simples de código HTML para uma barra de navegação:

```
<nav>
  <ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#services">Services</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>
```

E aqui está um exemplo de como estilizar a barra de navegação usando CSS

```
nav {
  background-color: #333;
  height: 50px;
  display: flex;
  align-items: center;
}

nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
}

nav li {
  margin-right: 20px;
}

nav a {
  color: #fff;
  text-decoration: none;
  font-size: 16px;
  padding: 10px 20px;
  display: block;
}

nav a:hover {
  background-color: #444;
}
```

}

Essa é apenas uma implementação básica de uma barra de navegação. É possível adicionar muitos outros recursos e efeitos, como dropdowns, estilos de hover, animações, etc. O importante é que a barra de navegação seja fácil de usar e permita que os usuários naveguem facilmente pelo site.

Barra de Navegação Vertical

Uma barra de navegação vertical é uma barra de navegação que é exibida na lateral da página, em vez de no topo. É semelhante a uma barra de navegação horizontal, mas é exibida de forma vertical e pode ser útil quando você deseja economizar espaço na tela ou quando deseja fornecer acesso a muitos links de forma compacta.

Aqui está um exemplo de como criar uma barra de navegação vertical usando HTML e CSS

```
<aside>
  <ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#services">Services</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</aside>
```

```
aside {
  background-color: #333;
  width: 200px;
  height: 100%;
  float: left;
  display: flex;
  flex-direction: column;
  align-items: center;
}

aside ul {
  list-style: none;
  margin: 0;
  padding: 0;
}

aside li {
  margin-bottom: 20px;
}

aside a {
  color: #fff;
  text-decoration: none;
```

```
font-size: 16px;
padding: 20px;
display: block;
width: 100%;
text-align: center;
}

aside a:hover {
  background-color: #444;
}
```

Neste exemplo, estamos usando o elemento `<aside>` para envolver a barra de navegação vertical. Usamos o `display flex` para exibi-lo como um contêiner flexível e especificamos o `flex-direction: column` para que os itens sejam exibidos na vertical. Alinhamos os itens ao centro usando `align-items: center`.

Como com qualquer outra barra de navegação, você pode personalizar a aparência da barra de navegação vertical e adicionar recursos extras, como hover effects, animações, etc.

Barra de Navegação Horizontal

A barra de navegação horizontal é uma parte comum de um layout de website que contém links para as páginas principais do site. É geralmente posicionada na parte superior do site e permite que os usuários naveguem pelo site de forma fácil e conveniente. Em CSS, a barra de navegação horizontal é criada usando listas horizontais e regras de estilo para controlar o posicionamento e a aparência dos elementos.

Itens da Lista Flutuante

Outra maneira de criar uma barra de navegação horizontal é flutuar os elementos `<li>` e especificar um layout para os links de navegação

- **float: left;** - Use **float** para fazer com que os elementos do bloco flutuem um ao lado do outro
- **display: block;** - Permite especificar o preenchimento (altura, largura, margens, etc., se desejar)
- **padding: 8px;** - Especifique algum preenchimento entre cada elemento `<a>`, para torná-los com boa aparência
- **background-color: #dddddd;** - Adicione uma cor de fundo cinza a cada elemento `<a>`

Menus suspensos em CSS

Os menus suspensos são elementos de interface do usuário que permitem a exibição de uma lista de opções ao clicar em um botão ou elemento de interface. Eles podem ser criados com CSS e HTML. Aqui está um exemplo simples de como criar um menu suspenso em CSS:

HTML:

```
<div class="dropdown">
  <button class="dropdown-button">Menu</button>
  <div class="dropdown-content">
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
    <a href="#">Link 3</a>
  </div>
</div>
```

CSS:

```
.dropdown {
  position: relative;
  display: inline-block;
}

.dropdown-content {
  display: none;
  position: absolute;
  background-color: #f9f9f9;
  min-width: 160px;
  box-shadow: 0px 8px 16px 0px rgba(0,0,0,0.2);
  z-index: 1;
}

.dropdown:hover .dropdown-content {
  display: block;
}
```

Neste exemplo, o HTML define uma div com a classe "dropdown", que contém um botão com a classe "dropdown-button" e uma div com a classe "dropdown-content" que irá conter as opções do menu. O CSS define a posição e a exibição do conteúdo do menu, além de estilos como cor de fundo, sombreamento e z-index. Quando o mouse é posicionado sobre a div com classe "dropdown", o conteúdo do menu é exibido.

Galeria de imagens

Uma galeria de imagens pode ser criada usando HTML e CSS. Uma galeria de imagens CSS é uma página web que apresenta uma coleção de imagens de forma organizada e atraente. O CSS (Cascading Style Sheets) é usado para estilizar e dar formato à galeria, incluindo o layout, tamanho das imagens, bordas, espaçamentos, entre outros aspectos visuais.

Sprites de imagem

Sprites de imagem CSS são imagens combinadas que contêm vários ícones ou elementos de imagem usados em uma página web. Ao invés de carregar cada imagem individualmente, o sprite é carregado uma única vez e os elementos são exibidos usando posicionamento CSS. Isso ajuda a otimizar o tempo de carregamento da página, pois o navegador precisa carregar apenas uma imagem.

Seletores de atributos CSS

Os seletores de atributos CSS permitem selecionar elementos HTML baseados em seus atributos ou valores de atributos. Isso significa que você pode estilizar elementos específicos em uma página web com base em suas propriedades, em vez de selecionar elementos pelo tipo ou classe. Aqui estão alguns exemplos comuns de seletores de atributos CSS:

- **Seletor de atributo exato:** `[attribute="value"]` - seleciona elementos com um atributo específico e valor exato. Por exemplo: `input[type="checkbox"]` seleciona todas as caixas de seleção na página.
- **Seletor de atributo contém:** `[attribute*="value"]` - seleciona elementos com um atributo que contém um valor específico. Por exemplo: `a[href*="example.com"]` seleciona todos os links que apontam para o site "example.com".
- **Seletor de atributo começa com:** `[attribute^="value"]` - seleciona elementos com um atributo que começa com um valor específico. Por exemplo: `img[src^="https"]` seleciona todas as imagens que começam com "https".
- **Seletor de atributo termina com:** `[attribute$="value"]` - seleciona elementos com um atributo que termina com um valor específico. Por exemplo: `a[href$=".pdf"]` seleciona todos os links que apontam para arquivos PDF.

Esses são apenas alguns exemplos dos muitos seletores de atributos que podem ser usados com CSS. Eles são úteis para aplicar estilos de forma precisa e personalizada em elementos específicos em uma página web.

Formulários CSS

Formulários são uma parte importante da maioria das páginas web e CSS pode ser usado para estilizá-los de maneiras criativas e atraentes. Aqui estão algumas coisas que você pode fazer com formulários e CSS:

- Alterar o tamanho e a cor dos elementos: Você pode alterar o tamanho das caixas de texto, botões e outros elementos de formulário para se adequar ao seu layout. Além disso, você pode mudar as cores dos elementos para combinar com o resto do seu site.
- Adicionar bordas e sombras: Você pode adicionar bordas e sombras aos elementos de formulário para melhorar sua aparência. Por exemplo, você pode adicionar uma sombra à caixa de texto para que ela pareça estar flutuando sobre a página.
- Modificar o estilo dos botões: Você pode modificar a aparência dos botões de formulário, incluindo suas cores, bordas e fontes. Isso ajuda a destacar os botões e torná-los mais fáceis de clicar.
- Adicionar imagens ao fundo: Você pode adicionar imagens de fundo a elementos de formulário, como caixas de texto e botões, para melhorar sua aparência. Isso também pode ser útil para destacar elementos importantes.
- Criar animações: Você pode criar animações simples ou complexas para elementos de formulário, como quando o usuário clica em um botão ou entra em uma caixa de texto. Isso pode tornar a experiência do usuário mais envolvente e atraente.

CSS é uma ferramenta poderosa para estilizar formulários de maneira criativa e personalizada. Você pode usar as técnicas acima ou combiná-las de várias maneiras para criar formulários únicos e atraentes que funcionem perfeitamente com o resto do seu site.

RESUMO

CSS é a sigla para **Folhas de Estilo em Cascata**. É uma linguagem de estilos usada para descrever a apresentação de um documento escrito em uma linguagem de marcação. É mais comumente usado para estilizar documentos HTML e XML, mas pode ser usado com qualquer documento XML, incluindo SVG e XHTML. O CSS é uma tecnologia-chave usada para criar sites visualmente atraentes e consistentes. Ele é usado para controlar layout, cores, fontes e outros elementos visuais em um site.

```
body {  
  background-color: lightblue;  
}  
  
h1 {  
  color: white;  
  text-align: center;  
}  
  
p {  
  font-family: verdana;  
  font-size: 20px;  
}
```

Existem **três** maneiras principais de incluir uma folha de estilo CSS em um documento HTML:

- **Estilos embutidos**: são estilos aplicados diretamente a um elemento HTML específico usando o atributo "estilo". Por exemplo, `<p style="color: red;">Este é um parágrafo vermelho.</p>`
- **Folha de estilo interna**: são estilos definidos na seção de cabeçalho de um documento HTML usando uma tag `<style>`. Por exemplo,

```
<head>  
<style>  
  p {  
    color: red;  
  }  
</style>  
</head>
```

- **Folha de estilo externa**: são estilos definidos em um arquivo .css separado e vinculados a um documento HTML usando uma tag `<link>` na seção head. Por exemplo,

```
<head>  
<link rel="stylesheet" type="text/css" href="styles.css">  
</head>
```

Sintaxe CSS

A sintaxe CSS consiste em um conjunto de regras que definem como aplicar estilos a elementos em um documento HTML. Essas regras são compostas de seletores, que especificam a quais elementos os estilos devem ser aplicados, e declarações, que definem os próprios estilos. O seletor é o elemento ou grupo de elementos aos quais os estilos serão aplicados. A propriedade é a propriedade CSS, como cor ou tamanho da fonte, que você deseja alterar. O valor é o valor para o qual você deseja definir a propriedade. Várias declarações podem ser adicionadas à mesma regra, separadas por ponto e vírgula:



- O seletor aponta para o elemento HTML que você deseja estilizar.
- O bloco de declaração contém uma ou mais declarações separadas por ponto e vírgula.
- Cada declaração inclui um nome de propriedade CSS e um valor, separados por dois pontos.
- Várias declarações CSS são separadas por ponto e vírgula e os blocos de declaração são cercados por chaves.

CSS também suporta o uso de **cascata e herança**, onde **estilos podem ser herdados** por elementos filhos de seus elementos pais.

Comentário

```
/* Este é um comentário CSS */
p{
  cor vermelha; /* Isso também é um comentário */
}
```

Seletores de CSS

Os seletores CSS são usados para "encontrar" (ou selecionar) os elementos HTML que você deseja estilizar.

Podemos dividir os seletores CSS em cinco categorias:

- Seletores simples (selecionar elementos com base no nome, id, classe)
- Seletores de combinação (selecionam elementos com base em uma relação específica entre eles)
- Seletores de pseudoclasse (selecionar elementos com base em um determinado estado)
- Seletores de pseudoelementos (selecionar e estilizar uma parte de um elemento)
- Seletores de atributo (selecionar elementos com base em um atributo ou valor de atributo)

Todos os seletores simples de CSS

Seletor	Exemplo	Descrição do exemplo
#id	#firstname	Seleciona o elemento com id="firstname"
.class	.intro	Seleciona todos os elementos com class="intro"
element.class	p.intro	Seleciona apenas elementos <p> com class="intro"
*	*	Seleciona todos os elementos
element	p	Seleciona todos os elementos <p>
element,element,..	div, p	Seleciona todos os elementos <div> e todos os elementos <p>

Cores CSS

As cores são especificadas usando nomes de cores predefinidos ou valores **RGB**, **HEX**, **HSL**, **RGBA**, **HSLA**. Em **CSS**, as cores podem ser especificadas usando vários métodos diferentes. Valor RGB: As cores podem ser especificadas usando o modelo de cores RGB (vermelho, verde, azul), onde cada cor é especificada por um valor entre 0 e 255. Por exemplo, a cor vermelha pode ser especificada como `rgb(255, 0, 0)`.

EXEMPLOS DE CORES EM CSS	<H1 STYLE="BACKGROUND-COLOR:RGB(255, 99, 71);">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:#FF6347;">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:HSL(9, 100%, 64%);">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:RGBA(255, 99, 71, 0.5);">...</H1>
	<H1 STYLE="BACKGROUND-COLOR:HSLA(9, 100%, 64%, 0.5);">...</H1>

CORES EM CSS

UM VALOR DE COR RGB REPRESENTA AS FONTES DE LUZ VERMELHA, VERDE E AZUL.

UMA COR HEXADECIMAL É ESPECIFICADA COM: #RRGGBB, ONDE OS INTEIROS HEXADECIMAIS RR (VERMELHO), GG (VERDE) E BB (AZUL)

HSL SIGNIFICA MATIZ, SATURAÇÃO E LEVEZA: HSL (MATIZ , SATURAÇÃO , LUMINOSIDADE)

Fundos CSS

CSS permite que você defina planos de fundo para elementos usando várias propriedades.

Propriedade	Descrição
background-color	Define a cor de fundo de um elemento. Você pode usar nomes de cores, valores rgb, hex, hsl para definir uma cor de fundo.
background-image	Define uma imagem como fundo de um elemento. A imagem é especificada por um valor de url. Você também pode usar a palavra-chave nenhum para remover qualquer imagem de plano de fundo.
background-repeat	Define como a imagem de fundo é repetida. O valor padrão é repetir, o que significa que a imagem será repetida horizontal e verticalmente. Você também pode usar repeat-x, repeat-y, no-repeat para controlar a repetição da imagem.
background-position	Define a posição da imagem de fundo. Você pode usar valores como centro, superior, inferior, esquerda, direita ou valores de comprimento específicos, como 10px 20px.
background-attachment	Define se a imagem de fundo rola com o resto do conteúdo (scroll) ou é fixa no lugar (fixed).
background-clip	Define a área onde a imagem de fundo deve ser pintada. O valor padrão é border-box, o que significa que a imagem será pintada dentro das bordas do elemento.
background-size	Define o tamanho da imagem de fundo. Você pode usar valores como conter ou cobrir para dimensionar a imagem ou valores de comprimento como 50% ou 200px para definir um tamanho específico.

É importante observar que a ordem dos valores na propriedade abreviada é:
plano de fundo: cor de fundo - imagem de fundo repetição de fundo posição de fundo

PLANO DE FUNDO: COR DE FUNDO - IMAGEM DE FUNDO - REPETIÇÃO DE FUNDO - POSIÇÃO DE FUNDO

Imagem de Fundo CSS

A propriedade `background-image` em CSS é usada para definir uma imagem como plano de fundo de um elemento. A imagem é especificada por um valor de URL, que pode ser um caminho relativo ou absoluto. Por exemplo:

```
body {  
  background-image: url("my-image.jpg");  
}
```

Propriedades de Borda do CSS

As propriedades de borda do CSS permitem que você especifique o estilo, a largura e a cor da borda de um elemento. Elas permitem controlar a aparência das bordas de um elemento HTML. Algumas das principais propriedades de borda incluem:

- `border-width`: controla a largura da borda.
- `border-style`: controla o estilo da borda (por exemplo, sólido, tracejado, etc.).
- `border-color`: controla a cor da borda.
- `border-radius`: controla a curvatura das bordas (para criar bordas arredondadas).
- `border`: é uma propriedade curta que permite definir a largura, estilo e cor da borda de uma só vez.

Além disso, também é possível controlar as bordas individuais de cada lado de um elemento utilizando as propriedades `border-top`, `border-right`, `border-bottom` e `border-left`. A propriedade `border-style` especifica que tipo de borda exibir. Os seguintes valores são permitidos:

Estilo de Borda CSS	Descrição
<code>dotted</code>	Define uma borda pontilhada
<code>dashed</code>	Define uma borda tracejada
<code>solid</code>	Define uma borda sólida
<code>double</code>	Define uma borda dupla
<code>groove</code>	Define uma borda ranhurada 3D. O efeito depende do valor da cor da borda
<code>ridge</code>	Define uma borda ondulada 3D. O efeito depende do valor da cor da borda

inset	Define uma borda de inserção 3D. O efeito depende do valor da cor da borda
outset	Define uma borda inicial 3D. O efeito depende do valor da cor da borda
none	Não define borda
hidden	Define uma borda oculta

Margens CSS

As margens são usadas para **criar espaço ao redor dos elementos**, fora de qualquer borda definida. Elas **permitem controlar o espaço fora de um elemento HTML**. Consiste na área transparente ao redor de um elemento e o separa dos elementos vizinhos.

A propriedade principal para controlar margens é a **margin**.

```
margin: top right bottom left;
```

Preenchimento CSS

O preenchimento CSS controla o espaço dentro de um elemento HTML. O preenchimento é a área preenchida dentro de um elemento, ocupando o espaço entre o seu conteúdo e sua borda.

A propriedade principal para controlar o preenchimento é a **padding**. A sintaxe geral é:

```
padding: top right bottom left;
```

Altura, largura e largura máxima do CSS

A altura, largura e largura máxima são propriedades CSS que controlam o tamanho de um elemento HTML.

A propriedade **height** define a altura de um elemento, a propriedade **width** define a largura de um elemento e a propriedade **max-width** define a largura máxima de um elemento. Vejamos a sintaxe para definir altura, largura e largura máxima, respectivamente:

```
/* Sintaxe para definir altura, largura e largura máxima, respectivamente */  
height: value;  
width: value;  
max-width: value;
```

Modelo de Caixa CSS

O modelo de caixa CSS é o modelo que define como o **conteúdo, borda, preenchimento e margem** de um elemento HTML **são exibidos e gerenciados na página**. O modelo de caixa CSS é composto pelas seguintes partes:

- **Conteúdo**: é o conteúdo real dentro do elemento, como texto, imagens, etc.
- **Preenchimento**: é o espaço entre o conteúdo e a borda, controlado pela propriedade padding.
- **Borda**: é uma linha ao redor do preenchimento e do conteúdo, controlada pelas propriedades border-width, border-style e border-color.
- **Margem**: é o espaço fora da borda, controlado pela propriedade margin.

Texto CSS

A propriedade color é usada para definir a cor do texto. A cor é especificada por:

um nome de cor - como "vermelho"

um valor HEX - como "#ff0000"

um valor RGB - como "rgb(255,0,0)"

Consulte Valores de cores CSS para obter uma lista completa de possíveis valores de cores.

O CSS oferece uma ampla gama de propriedades para formatar o texto, incluindo:

Propriedade	Descrição
font-family	Define a fonte para o texto.
font-size	Define o tamanho da fonte.
font-weight	Define o peso da fonte (normal, bold, etc.).
line-height	Define a altura da linha.
letter-spacing	Define o espaçamento entre letras.
word-spacing	Define o espaçamento entre palavras.
text-align	Define o alinhamento do texto (esquerda, direita, centro, justificado).
text-decoration	Define a decoração do texto (sublinhado, riscado, etc.).
text-transform	Define a transformação do texto (maiúsculas, minúsculas, capitalização).
color	Define a cor do texto.
text-shadow	Define uma sombra para o texto.

Fontes CSS

A propriedade **font** é uma **propriedade abreviada** que permite **especificar várias propriedades de fonte em um único valor**, como **tamanho, estilo, variante e peso**. Isso facilita a escrita de regras de estilo e economiza tempo. A ordem das propriedades especificadas na propriedade abreviada é **font-style, font-variant, font-weight, font-size, line-height e font-family**. Em CSS existem cinco famílias de fontes genéricas:

- As fontes **serifadas** têm um pequeno traço nas bordas de cada letra. Eles criam uma sensação de formalidade e elegância.
- As fontes **sem serifa** têm linhas limpas (sem pequenos traços anexados). Eles criam um visual moderno e minimalista.
- Fontes **monoespaçadas** - aqui todas as letras têm a mesma largura fixa. Eles criam uma aparência mecânica.
- As fontes **cursivas** imitam a caligrafia humana.
- Fontes **fantasia** são fontes decorativas/divertidas.

Ícones de CSS

Os ícones podem ser facilmente adicionados à sua página HTML, usando uma biblioteca de ícones.

- **Fontes de ícones:** Utilizando fontes de ícones como Font Awesome, Glyphicons, etc.
- **Imagens:** Adicionando imagens através de URLs ou arquivos locais.
- **SVG:** Inserindo imagens SVG diretamente no HTML ou através de arquivos externos.
- **CSS pseudo-elementos:** Utilizando pseudo-elementos como **:before** ou **:after** para criar ícones usando CSS.

Links CSS

Style a link with a color

This is a link

Com CSS, os links podem ser estilizados de várias maneiras diferentes. Os links podem ser estilizados com qualquer propriedade CSS (por exemplo **color, font-family, background, etc.**). Veja na imagem um link com a cor hotpink.

```
a {  
  color: hotpink;  
}
```

Os quatro estados dos links são:

- **a:link**- um link normal não visitado
- **a:visited**- um link que o usuário visitou
- **a:hover**- um link quando o usuário passa o mouse sobre ele
- **a:active**- um link no momento em que é clicado

Tabelas CSS

A aparência de uma tabela HTML pode ser muito melhorada com CSS! 😊 As tabelas em CSS podem ser estilizadas usando os seguintes conceitos:

- **CSS table properties:** Propriedades como "border-collapse", "border-spacing", "width", "height", etc. podem ser usadas para controlar o aspecto de uma tabela.
- **CSS table-cell properties:** Propriedades como "text-align", "vertical-align", "padding", etc. podem ser usadas para controlar o aspecto das células da tabela.
- **CSS pseudo-classes:** Pseudo-classes como ":hover" podem ser usadas para mudar o estilo da tabela quando o usuário passa o mouse sobre ela.
- **CSS media queries:** Media queries podem ser usadas para criar tabelas responsivas, que se adaptam a diferentes tamanhos de tela.

Layout CSS - A propriedade de exibição

A propriedade de exibição **display** especifica se/como um elemento é exibido. Cada elemento HTML tem um valor de exibição padrão, dependendo do tipo de elemento. O valor de exibição padrão para a maioria dos elementos é block ou inline.

Elementos de nível de bloco

Um elemento de nível de bloco sempre começa em uma nova linha e ocupa toda a largura disponível (estende-se para a esquerda e para a direita o máximo possível).

O elemento `<div>` é um elemento de nível de bloco. Exemplos de elementos de nível de bloco:

ELEMENTOS DE NÍVEL DE BLOCO						
<code><DIV></code>	<code><H1> - <H6></code>	<code><P></code>	<code><FORMA></code>	<code><CABEÇALHO></code>	<code><RODAPÉ></code>	<code><SEÇÃO></code>

Elementos embutidos

Um elemento inline não começa em uma nova linha e ocupa apenas a largura necessária. Exemplos de elementos embutidos:

ELEMENTOS EMBUTIDOS		
<code></code>	<code><A></code>	<code></code>

`display: none;` é comumente usado com JavaScript para **ocultar e mostrar elementos sem excluí-los e recriá-los**. `display: none` é uma propriedade de CSS que **especifica que um elemento**

HTML não deve ser exibido na página. Esse valor oculta completamente o elemento, incluindo todo o espaço que ele ocuparia.

A propriedade de posição

A propriedade CSS "position" é usada para definir como um elemento HTML é posicionado em relação ao seu elemento pai ou à página (estático, relativo, fixo, absoluto ou fixo).

. As opções de posição incluem:

- **static**: o elemento é posicionado de acordo com o fluxo normal do documento (padrão)
- **relative**: o elemento é posicionado em relação a sua posição normal, permitindo o uso de "top", "right", "bottom" e "left" para mover o elemento a partir da sua posição original
- **fixed**: o elemento é posicionado em relação à janela do navegador e não se move quando a página é rolada.
- **absolute**: o elemento é posicionado em relação ao primeiro elemento pai com posição não-estática (se não houver, será posicionado em relação à janela do navegador)
- **sticky**: o elemento é posicionado de forma semelhante ao "relative" até que ele atinja uma certa posição na tela, a partir daí ele se comporta como "fixed".

Os elementos são então posicionados usando as propriedades **superior**, **inferior**, **esquerda** e **direita**. No entanto, essas propriedades não funcionarão, a menos que a propriedade **position** seja definida primeiro. Eles também funcionam de forma diferente, dependendo do valor da posição.

Posição	Descrição
<code>position: static;</code>	Os elementos HTML são posicionados estáticos por padrão. Os elementos posicionados estáticos não são afetados pelas propriedades superior, inferior, esquerda e direita.
<code>position: relative;</code>	Um elemento com <code>position: relative;</code> é posicionado em relação à sua posição normal. Definir as propriedades superior, direita, inferior e esquerda de um elemento relativamente posicionado fará com que ele seja ajustado para longe de sua posição normal. Outros conteúdos não serão ajustados para caber em qualquer lacuna deixada pelo elemento.
<code>position: fixed;</code>	É posicionado em relação à viewport, o que significa que ele sempre permanece no mesmo lugar, mesmo que a página seja rolada. As propriedades superior, direita, inferior e esquerda são usadas para posicionar o elemento. Um elemento fixo não deixa uma lacuna na página onde normalmente estaria localizado.
<code>position: absolute;</code>	Um elemento com <code>position: absolute;</code> é posicionado em relação ao ancestral posicionado mais próximo (em vez de posicionado em relação à viewport, como fixo).

A propriedade z-index

A propriedade CSS "z-index" é usada para **controlar a ordem de sobreposição de elementos HTML** na tela. Elementos com um valor de "z-index" **mais alto são exibidos "acima"** de elementos com um valor de "z-index" mais baixo. A propriedade "z-index" só tem efeito se a propriedade "position" for definida como "absolute", "relative" ou "fixed". O valor padrão é "auto", o que significa que o navegador decide a ordem de sobreposição baseado no fluxo normal do documento. Valores positivos colocam o elemento mais à frente, enquanto valores negativos colocam o elemento mais atrás.

Overflow

A propriedade CSS "overflow" controla o que acontece quando o conteúdo de um elemento HTML excede suas dimensões definidas, ou seja, **conteúdo é grande demais** para caber em uma área. A propriedade "overflow" pode ter os seguintes valores:

- **visible**: o conteúdo é exibido fora do elemento, sem cortar ou rolar.
- **hidden**: o conteúdo é cortado, ou seja, o que ultrapassar as bordas do elemento não será exibido.
- **scroll**: uma barra de rolagem é adicionada ao elemento, permitindo que o usuário role e veja o conteúdo que está fora das dimensões visíveis.
- **auto**: se o conteúdo ultrapassar as dimensões do elemento, uma barra de rolagem será adicionada. Se o conteúdo couber, nenhuma barra de rolagem será exibida.

Seletores de atributos CSS

Os seletores de atributos CSS permitem selecionar elementos HTML baseados em seus atributos ou valores de atributos. Isso significa que você pode estilizar elementos específicos em uma página web com base em suas propriedades, em vez de selecionar elementos pelo tipo ou classe. Aqui estão alguns exemplos comuns de seletores de atributos CSS:

- **Seletor de atributo exato**: [attribute="value"] - seleciona elementos com um atributo específico e valor exato. Por exemplo: input[type="checkbox"] seleciona todas as caixas de seleção na página.
- **Seletor de atributo contém**: [attribute*="value"] - seleciona elementos com um atributo que contém um valor específico. Por exemplo: a[href*="example.com"] seleciona todos os links que apontam para o site "example.com".
- **Seletor de atributo começa com**: [attribute^="value"] - seleciona elementos com um atributo que começa com um valor específico. Por exemplo: img[src^="https"] seleciona todas as imagens que começam com "https".
- **Seletor de atributo termina com**: [attribute\$="value"] - seleciona elementos com um atributo que termina com um valor específico. Por exemplo: a[href\$=".pdf"] seleciona todos os links que apontam para arquivos PDF.

REFERÊNCIAS

<https://www.w3schools.com/css/default.asp>

QUESTÕES COMENTADAS

Questões CESPE

1. (CESPE – SEPLAN RR– 2023) Com relação a programação e desenvolvimento de sistemas, julgue o item a seguir.

CSS é um código separado da HTML que pode afetar a aparência das tags em uma única página ou em todo um site da Web.

Comentários:

Correto Pessoal! CSS é uma sigla para Cascading Style Sheets e é usado para definir o estilo visual de um documento HTML. Ele permite que você defina a aparência e formatação de elementos HTML, como fontes, cores, espaçamento, tamanhos de elementos, etc. A vantagem de se separar o HTML e o CSS é que você pode mudar a aparência de todo o site, aplicando alterações em um único arquivo CSS, em vez de ter que mudar o HTML em cada página individual. Isso facilita a manutenção e otimização do site.

Gabarito: Correto

2. (CESPE –TCE RJ– 2022) Quanto ao desenvolvimento de sistemas web, julgue o item seguinte.

A versão 3 do CSS trouxe para as bordas das páginas web as opções de definição de cor, arredondamento de cantos e multiplicidade de linhas.

Comentários:

Pessoal, na verdade, a versão 3 do CSS não trouxe essas especificações para as bordas das páginas da web. Elas já estavam disponíveis nas versões anteriores do CSS. As propriedades CSS como border-color, border-radius e border-style são amplamente utilizadas para definir a aparência das bordas de elementos HTML. A versão 3 do CSS trouxe muitos novos recursos e melhorias em comparação com as versões anteriores, incluindo novos seletores, animações, fontes, modelos de layout, etc. Além disso, o CSS3 também trouxe melhorias na performance e no suporte a diferentes dispositivos, como dispositivos móveis.

Gabarito: Errado

3. (CESPE – DP DF – 2022) Julgue o item a seguir, acerca de CSS3, JMS, JSON e JUnit.

Quando se utiliza CSS3, para que os flex items sejam apresentados na mesma ordem em que aparecem no HTML, é necessário atribuir o valor 1 à propriedade order.

Comentários:

Não, não é necessário atribuir o valor 1 à propriedade order para que os itens flexíveis sejam apresentados na mesma ordem em que aparecem no HTML. Por padrão, os itens flexíveis são exibidos na ordem em que aparecem no HTML, a menos que sejam especificados de outra forma. A propriedade order é usada para controlar a ordem de apresentação dos itens flexíveis dentro de um container flexível. Por padrão, todos os itens flexíveis têm order: 0, o que significa que serão exibidos na ordem em que aparecem no HTML. Se você deseja mudar a ordem de apresentação de um item flexível, basta atribuir um valor diferente à propriedade order desse item. Por exemplo, um item com order: -1 será exibido antes de um item com order: 0 no container flexível.

Gabarito: Errado

4. (CESPE – PETROBRAS – 2022)

Julgue o próximo item que trata de CSS, JavaScript e Net Core.

No código abaixo, escrito na linguagem CSS, red é um valor do tipo palavra-chave, enquanto #f00 é um valor do tipo notação funcional.

```
p {  
  color: red;  
  background-color: #f00;  
}
```

Comentários:

Pessoal, a primeira parte está correta, a questão erra ao dizer: "enquanto #f00 é um valor do tipo notação funcional, quando na verdade é um valor hexadecimal. A propriedade color está definida como a palavra-chave red, enquanto a propriedade background-color está definida como a notação hexadecimal #f00. As palavras-chave são nomes pré-definidos para cores comuns, enquanto as notações funcionais são códigos numéricos que representam cores. Além da notação hexadecimal, existem outras notações funcionais, como RGB e HSL.

Gabarito: Errado

5. (CESPE – ALECE – 2021) Web designers costumam usar CSS e HTML na construção de um sítio eletrônico, já que, juntos, eles definirão como as páginas serão exibidas nos navegadores, conforme exemplificado no código a seguir.

```
<html>  
<head>  
  
<style type="text/css">  
body {background-color: yellow}  
h1 {background-color: #00ff00}  
h2 {background-color: transparent}  
h3 {background-color: rgb(250,0,0)}  
p {background-color: rgb(250,0,255)}
```

```
</style>

</head>

<body>

<h1>Este é título 1</h1>
<h2>Este é título 2</h2>
<h3>Este é título 3</h3>
<p>Este é um parágrafo</p>

</body>
</html>
```

Considerando-se o código anterior, o comando da cor verde é

- a) background-color: rgb(250,0,0).
- b) background-color: yellow.
- c) background-color: #00ff00.
- d) background-color: transparente.
- e) background-color: rgb(250,0,255).

Comentários:

E aí? O que acharam? Não conseguiu lembrar as cores? Pessoal, ultimamente vimos as bancas cobrando um conhecimento cada vez mais aprofundado. As cores são "simples" para lembrar: a sequência é RGB (red, green, blue) Assim, Vermelho = "#FF0000", Verde = "#00FF00" e Azul = "#0000FF". Além disso temos o Preto = "#000000" e o Branco = "#FFFFFF" sendo as cores mais cobradas. Dessa forma, nosso gabarito é a alternativa c) background-color: #00ff00.

Gabarito: Letra C

6. (CESPE – TJ PA – 2020) Assinale a opção que indica a propriedade usada no CSS3 para definir o alinhamento de um elemento inline com relação ao seu elemento-pai.

- a) alignment-baseline
- b) alignment-adjust
- c) background-image
- d) background-position
- e) line-boxes

Comentários:

A propriedade alignment-baseline especifica como um elemento de nível embutido é alinhado em relação ao seu pai. Isto é, a qual das linhas de base do pai o ponto de alinhamento deste elemento está alinhado. Ao contrário da propriedade 'dominant-baseline', a propriedade 'alignment-baseline' não tem efeito em suas linhas de base dominantes filhas.

7. (CESPE – ME– 2020) Acerca de desenvolvimento de sistemas web, julgue o item a seguir.

No CSS 3, o uso da propriedade box-shadow e da função linear-gradient possibilita a criação de formas, cores e efeitos das páginas diretamente no código, de modo que os resultados aparecem diretamente nos navegadores.

Comentários:

Pessoal, a questão está correta! As propriedades box-shadow e linear-gradient são poderosas ferramentas para designers e desenvolvedores na criação de interfaces web atraentes e funcionais. A propriedade box-shadow permite adicionar sombras aos elementos da página, criando profundidade e uma aparência mais tridimensional. Você pode especificar a direção, a intensidade e a cor da sombra para se adequar ao seu projeto. Já a função linear-gradient permite criar gradientes lineares de cor, que são transições suaves entre duas ou mais cores. Você pode usar gradientes para enriquecer a aparência de elementos de interface, como botões e planos de fundo. Ambas as propriedades são fáceis de usar e permitem a criação de efeitos visuais avançados sem a necessidade de imagens adicionais. Além disso, eles são compatíveis com a maioria dos navegadores modernos, o que significa que seus designs serão consistentes em uma ampla variedade de dispositivos.

Gabarito: Correto

8. (CESPE – MPC TCE-PA – 2019) O uso de CSS permite a criação de padrões para a publicação de páginas na Internet, com possibilidade de redução da quantidade de informações em cada página. Em relação a esse assunto, a propriedade que deve ser configurada quando é necessário posicionar-se uma legenda de uma tabela é

- a) outline.
- b) padding-top.
- c) caption-side.
- d) table-layout.
- e) text-combine-upright.

Comentários:

Pessoal, a propriedade CSS "caption-side" permite posicionar uma legenda de uma tabela em relação aos dados da tabela. Os valores possíveis para esta propriedade são: "top", "bottom", "initial" e "inherit". Vejamos resumidamente as demais propriedades. a) outline é uma propriedade CSS que permite adicionar uma linha ao redor de um elemento HTML, fornecendo uma marca visual que destaca o elemento. b) padding-top é uma propriedade CSS que permite especificar a quantidade de espaço acima do conteúdo de um elemento HTML. d) table-layout é uma propriedade CSS que controla como os dados de uma tabela são distribuídos na tela. e) text-combine-upright é uma propriedade CSS que permite combinar vários caracteres em uma unidade, permitindo a exibição de texto longo em áreas pequenas.

Gabarito: Letra C

9. (CESPE – TJ AM– 2019) Acerca do desenvolvimento web mediante o uso do HTML 5, do JavaScript, do XML e do CSS, julgue o item subsequente.

O CSS permite anexar estilos a elementos estruturados, em documentos HTML ou em aplicativos XML. Com o CSS 3, é possível flutuar o <nav>, colocar bordas no <header> e no <footer>, além de incluir margens e deslocamentos no <article>.

Comentários:

Pessoal, a questão está perfeita! De fato, o CSS permite anexar estilos a elementos estruturados, em documentos HTML ou em aplicativos XML. Com o CSS 3, é possível flutuar o <nav>, colocar bordas no <header> e no <footer>, além de incluir margens e deslocamentos no <article>. O CSS é uma linguagem de folha de estilo usada para controlar a apresentação de elementos HTML na web. Ele permite que você dê estilo e formato aos seus elementos de HTML, como alterar a cor de fundo, a cor da fonte, a largura, a altura, a margem, o alinhamento, entre outros aspectos. Com o CSS 3, é possível ainda adicionar efeitos visuais como sombras, rotações, animações, entre outros. Além disso, ele permite que você aplique estilos a elementos específicos com base em sua class, ID ou outro atributo HTML.

Gabarito: Correto

10. (CESPE – TJ AM – 2019) Com relação a desenvolvimento em Java para Web, julgue o item que se segue.

De acordo com o conceito de herança em CSS, as propriedades de página com especificidade mais alta têm prioridade sobre as propriedades com especificidade mais baixa.

Comentários:

A especificidade é a maneira de como os navegadores definem quais valores de propriedades são os mais relevantes para o elemento a ser utilizado. A especificidade é baseada apenas nas regras impostas na composição de diferentes tipos de seletores. Ela determina a prioridade de uma regra de estilo sobre outras regras de estilo aplicadas ao mesmo elemento. As regras de estilo com a especificidade mais alta têm prioridade sobre as regras de estilo com a especificidade mais baixa.

Gabarito: Correto

Questões FCC

1. (FCC – TJ-CE – 2022) Para colocar a cor de fundo vermelha apenas dos campos (inputs) do tipo text de um formulário, utiliza-se a instrução CSS

- a) `input [type=text] {background-color: #00ff00}`
- b) `input.type [text] {background-color: #ff0000}`
- c) `input [type=text] {background-color: #ff0000}`
- d) `input.type[text] {background: rgb(0,255,0)}`
- e) `input [type('text')] {background-color:rgb(255,0,0)}`

Comentários:

Para isso podemos usar seletores de atributos: selecione elementos com base em seus atributos ou valores de atributos. Por exemplo, `input[type="text"] { }` seleciona todos os elementos `<input>` com um atributo de tipo "text". No caso da questão, devemos usar o `input [type=text] {background-color: #ff0000}`.

Gabarito: Letra C

2. (FCC – TRT - 4ª REGIÃO (RS) – 2022) Quando um Técnico criou uma página web usando HTML e CSS, um elemento ficou sobreposto ao outro na tela e ele deseja inverter a ordem de sobreposição desses elementos. Para definir ou mudar a ordem de sobreposição dos elementos ele deve utilizar a propriedade CSS

- a) z-index
- b) inverter
- c) position
- d) flex-position
- e) x-order

Comentários:

Ei pessoal, lembrem-se do que eu disse, sempre que se fala em sobreposição de elementos, devemos lembrar do z-index que é propriedade CSS usada para definir ou mudar a ordem de sobreposição dos elementos. Então, vocês vão lembrar que quando se fala em ordem de sobreposição de elementos HTML estamos falando sobre a propriedade z-index. Lembre-se que a propriedade CSS "z-index" é usada para **controlar a ordem de sobreposição de elementos HTML** na tela. Elementos com um valor de "z-index" mais alto são exibidos "acima" de elementos com um valor de "z-index" mais baixo.

Gabarito: Letra A

3. (FCC – PGE AM – 2022) Na utilização externa de CSS em páginas HTML, deve-se indicar um arquivo CSS que será usado na definição dos estilos dos elementos da página. Supondo que o arquivo a ser utilizado é o `estilos.css`, a instrução correta é

- a) `<@import type="style/css" src="estilos.css">`
- b) `<link rel="stylesheet" href="estilos.css">`
- c) `<style type="css" src="estilos.css"></style>`
- d) `<link type="stylesheet" src="estilos.css">`
- e) `<@require file="estilos.css" type="style/css">`

Comentários:

A forma correta de incluir um arquivo CSS externo em uma página HTML é `<link rel="stylesheet" href="estilos.css">`. A tag `<link>` é usada para vincular o arquivo CSS ao HTML, e o atributo `rel` especifica o tipo de relação entre o HTML e o CSS (no caso, "stylesheet"). O atributo `href` especifica a localização do arquivo CSS a ser vinculado. Ao incluir o arquivo CSS externo, você pode reutilizar o mesmo estilo em várias páginas HTML. Além disso, isso ajuda a manter o código HTML mais limpo, pois você pode manter todo o código CSS em um único arquivo externo.

Gabarito: Letra B

4. (FCC – MANAUSPREV – 2021) No formulário de contato de uma página web do site do Governo de XX, um desenvolvedor de software precisa especificar que todos os campos do tipo texto (text) devem ser mostrados com fonte azul. Para estabelecer este estilo usando a linguagem CSS, deve-se utilizar o comando

- a) `input [type=text] {color:blue}`
- b) `input type='text' {color:blue}`
- c) `input.text {color:blue}`
- d) `input!text {color:#0000ff}`
- e) `input#text {color:#blue}`

Comentários:

Nessa questão também temos a aplicação de seletores! Nesse caso um desenvolvedor de software precisa especificar que todos os campos do tipo texto (text), ou seja, `input [type=text]`, devem ser mostrados com fonte azul... eis a questão... qual é o correto? Tanto `{color:blue}` quanto `{color:#0000ff}` estão corretos. O principal, nessa questão é saber a sintaxe correta: `input [type=text]`. Portanto, temos aí nosso gabarito!

Gabarito: Letra A

5. (FCC– TJ-SC – 2021) Considere o fragmento de código HTML5, abaixo:

```
<body>
  <div class="topo">
    <div class="logo"></div>
    <div class="slogan"></div>
  </div>
</body>
```

Considere, também, o fragmento de código CSS, abaixo, para o código HTML5 apresentado.

```
div.topo{width:100%;height:150px;background-color:yellow; Imagem associada para  
resolução da questão }  
div.logo{background-color:blue; flex:1}  
div.slogan{background-color:green; flex:1}
```

Para que as divisões logo e slogan apareçam uma do lado da outra, com tamanhos iguais, preenchendo toda a divisão topo, a lacuna I deve ser corretamente preenchida por

- a) display:inline
- b) float:left
- c) display:flex
- d) position:relative
- e) display:block

Comentários:

Display: flex é um valor de CSS que transforma um elemento HTML em um container flexível. Isso significa que os elementos filhos dentro desse container podem ser alinhados em diferentes direções e dimensionados de acordo com a largura ou altura disponíveis. Assim, podemos ter por exemplo `.flex-container { display: flex; }`

Gabarito: Letra C

6. (FCC – AL-AP – 2020) Segundo os padrões de acessibilidade na web definidos para sites do Governo Eletrônico, códigos CSS devem estar sempre em arquivos externos a serem chamados pelas páginas do sítio/portal. Por exemplo, a instrução `<link rel="stylesheet" type="text/css" href="aleap.css"/>` pode ser também escrita na forma:

- a) `<style rel="link" type="text/css"> @include url("aleap.css"); </style>`
- b) `<link lang="css" type="stylesheet" href="aleap.css"/>`
- c) `<style rel="stylesheet" type="text/css"> @import url("aleap.css"); </style>`
- d) `<link rel="external" type="text/css" src="aleap.css"/>`
- e) `<style rel="stylesheet" type="text/css" url("aleap.css")/>`

Comentários:

Sabemos que há três formas de inserir CSS: Estilos embutidos, Folha de estilo interna e Folha de estilo externa. Ainda, o uso de uma folha de estilo externa é considerado uma prática recomendada, portanto, vamos lembrar como é inserir CSS usando tags! As tags são `<style>` e `<link>`. Já que a banca deu a link no enunciado, vamos procurar pela tag Style, certo? Das opções, a correta é semelhante ao nosso exemplo `<style type="text/css"> @import url("aleap.css"); </style>`. Portanto, nosso gabarito é a letra C.

Gabarito: Letra C

7. (FCC– AL-AP – 2020) Em um contêiner criado pela tag div há vários contêineres menores. Para que estes contêineres internos sejam posicionados um ao lado do outro horizontalmente, pode ser utilizada, em todos eles, a instrução CSS:
- a) align:inline
 - b) float:left
 - c) display:left
 - d) position:inline
 - e) aling:inline-block

Comentários:

A propriedade "float: left" é comumente utilizada para posicionar elementos HTML na linha horizontalmente, flutuando-os para a esquerda. Quando um elemento é definido com "float: left", ele é retirado do fluxo normal da página e posicionado na esquerda da página, ocupando apenas o espaço necessário para o seu conteúdo. Os elementos subsequentes são então exibidos ao seu lado, na direita. É importante destacar que elementos com "float" não afetam o tamanho de seus elementos pais, por isso é comum utilizar a propriedade "clear" em conjunto para garantir que outros elementos não fiquem sobrepostos aos elementos flutuantes.

Gabarito: Letra B

8. (FCC –TJ-MA– 2019) A propriedade display na linguagem CSS3 especifica como um elemento será exibido no layout da página web. Comparado ao display:block, a principal diferença é que o display:inline-block
- a) exibe parte do elemento da página na horizontal e parte na vertical.
 - b) não adiciona quebra de linha após o elemento, permitindo que este se ajuste ao lado de outros elementos.
 - c) não permite definir uma largura e altura para o elemento.
 - d) não permite definir as margens e os paddings do elemento.
 - e) adiciona quebra de linha após o elemento, não permitindo que este se ajuste ao lado de outros elementos.

Comentários:

Display: inline-block é uma propriedade CSS que especifica o comportamento de exibição de um elemento HTML. O valor inline-block cria um contêiner de bloco de nível inline. Os elementos com esse valor são colocados alinhados com o texto e outros elementos, mas se comportam como um elemento de nível de bloco, permitindo que largura, altura e preenchimento sejam definidos. Isso torna o bloco embutido útil para criar elementos flexíveis e de tamanho ajustável que podem ser usados em layouts com muito texto.

Gabarito: Letra B

9. (FCC –SANASA Campinas– 2019) Um Desenvolvedor de software precisa inserir uma instrução no cabeçalho de uma página HTML que fará referência a um arquivo chamado a001.css a ser aplicado apenas quando a página for aberta em dispositivos com tela de até 600 pixels. A instrução correta que deverá ser inserida é

- a) <@import URL(a001.css) only screen and (max-width: 600px)>
- b) <link rel="media" device="only screen with (max-width: 600px)" href="a001.css">
- c) <link rel="stylesheet" media="screen and (max-width: 600px)" href="a001.css">
- d) <inport file="a001.css" media="screen and (max-width: 600px)">
- e) <style>@media only screen and (min-width: 600px) URL(a001.css) <style>

Comentários:

A instrução correta a ser inserida no cabeçalho da página HTML é a seguinte: <link rel="stylesheet" media="screen and (max-width: 600px)" href="a001.css">. Aqui, a propriedade "media" é utilizada para especificar que o arquivo CSS "a001.css" deve ser aplicado apenas quando a tela do dispositivo tiver até 600 pixels de largura. Isso é feito usando a expressão "screen and (max-width: 600px)". Esse recurso permite que você aplique folhas de estilo diferentes dependendo das características do dispositivo ou do tamanho da tela.

Gabarito: Letra C

10.(FCC– Prefeitura de Manaus - AM – 2019) Para que, ao se posicionar o ponteiro do mouse sobre cada um dos links da página, a cor da letra do link mude para vermelha, deve-se utilizar para a página web a configuração CSS

- a) a:hover {color: #00FF00}.
- b) a:over {font-color: #0000FF}.
- c) a:hover {font-color: #FF0000}.
- d) a:over {color: #00FF00}.
- e) a:hover {color: #FF0000}

Comentários:

Você pode usar a pseudo-classe ":hover" do CSS para alterar a cor da letra dos links quando o ponteiro do mouse estiver posicionado sobre eles. Nosso gabarito é a alternativa E) a:hover {color: #FF0000}. Essa configuração CSS alterará a cor da letra dos links para vermelho quando o ponteiro do mouse estiver posicionado sobre eles. A pseudo-classe ":hover" é usada para selecionar um elemento quando o ponteiro do mouse estiver sobre ele, permitindo que você aplique estilos diferentes a um elemento quando ele estiver em um determinado estado. Além disso, você pode usar outras pseudo-classes, como ":active" e ":visited", para selecionar links em estados diferentes.

Gabarito: Letra E

11.(FCC –TRF - 4ª REGIÃO – 2019) As unidades de medida usadas em CSS para ajudar na responsividade dos sites, que especificam medidas relativas a 1% da largura e 1% da altura da janela do navegador (viewport), são, respectivamente,

- a) px e pt.
- b) cm e pc.
- c) vw e vh.
- d) em e ex.

e) em e p

Comentários:

As unidades de medida usadas em CSS para especificar medidas relativas a 1% da largura e 1% da altura da janela do navegador (viewport) são, respectivamente, "vw" (viewport width) e "vh" (viewport height). "vw" representa 1% da largura da janela do navegador e "vh" representa 1% da altura da janela do navegador. Isso significa que, por exemplo, se você definir a largura de um elemento como "50vw", ele terá 50% da largura da janela do navegador. E se você definir a altura de um elemento como "30vh", ele terá 30% da altura da janela do navegador. Essas unidades de medida são úteis para ajudar a tornar o site mais responsivo, já que elas permitem que você especifique medidas relativas ao tamanho da janela do navegador, em vez de medidas absolutas em pixels. Isso significa que o site será automaticamente ajustado a diferentes tamanhos de tela, sem a necessidade de ajustes adicionais.

Gabarito: Letra C

12.(FCC –SEFAZ-BA – 2019) Em uma página web criada com HTML5 e CSS3 há 3 contêineres com nome de classe caixa, criados com a tag div. Criou-se para estes contêineres a seguinte configuração CSS.

```
div.caixa{  
    width:300px;  
    height:200px;  
    border:solid #c0c0c0 1px;  
}
```

Para que os contêineres sejam posicionados um ao lado do outro horizontalmente deve-se adicionar ao bloco CSS acima a instrução

- a) position: side -by-side;
- b) box-position: left;
- c) position: relative;
- d) align: side-by-side;
- e) float: left;

Comentários:

Neste caso, deve-se usar a propriedade float: left. Assim, a configuração CSS ficaria da seguinte forma:

```
div.caixa{  
    width:300px;  
    height: 200px;  
    border:solid #c0c0c0 1px;  
    float: left;  
}
```

Com isso, os contêineres com a classe "caixa" serão posicionados um ao lado do outro horizontalmente, criando uma linha. Se você quiser que eles apareçam na próxima linha, basta

adicionar uma quebra de linha (
) ou outro contêiner com clear: both; depois dos três contêineres com a classe "caixa".

Gabarito: Letra E

13.(FCC– AFAP – 2019) No CSS3 podem ser usadas diversas unidades de medida para definir o tamanho pelo qual os elementos são renderizados na página web quando aberta na janela do navegador. Algumas dessas unidades de medida são relativas e adequam o tamanho do elemento proporcionalmente ao tamanho da janela. Duas dessas unidades de medida são descritas abaixo, em inglês.

I. Relative to 1% of the width of the viewport (the browser window size).

II . Relative to 1% of the height of the viewport (the browser window size).

I e II referem-se, respectivamente, às unidades de medida

- a) pw e ph.
- b) ex e ey.
- c) vx e vy.
- d) pc e pt.
- e) vw e vh.

Comentários:

I e II referem-se, respectivamente, às unidades de medida vw (viewport width) e vh (viewport height) em CSS. As unidades de medida vw e vh são usadas em CSS para definir o tamanho de elementos de forma relativa ao tamanho da janela do navegador. A unidade vw representa 1% da largura da janela do navegador, enquanto a unidade vh representa 1% da altura da janela do navegador. Essas unidades são úteis para criar layouts responsivos, que se adaptam automaticamente a diferentes tamanhos de tela. Por exemplo, você pode definir a largura de um elemento como 50vw para que ele ocupe 50% da largura da janela do navegador.

Gabarito: Letra E

14.(FCC– DPE-AM– 2018) A instrução CSS3 div + p {background-color: blue;} aplica a cor de fundo azul ao elemento <p> colocado

- a) imediatamente após o elemento <div>.
- b) dentro do elemento <div>.
- c) próximo a um elemento <div>.
- d) imediatamente antes do elemento <div>.
- e) como filho do elemento <div>.

Comentários:

A instrução CSS3 div + p {background-color: blue;} aplica a cor de fundo azul ao elemento <p> colocado imediatamente após a tag <div>, na mesma hierarquia de elementos HTML, ou seja, nosso gabarito é a letra a. Esse tipo de seletor, "+", é conhecido como seletor adjacente. Com

ele, você pode selecionar um elemento que está imediatamente adjacente a outro elemento, especificado antes do seletor adjacente. Nesse caso, o elemento <p> está sendo selecionado porque está imediatamente após a tag <div>.

Gabarito: Letra A

15.(FCC–DPE-AM – 2018) Um Técnico Programador usou a referência abaixo para um arquivo CSS em uma página web construída com HTML.

```
<link type="text/css" href="som.css">
```

Para que esse arquivo CSS seja usado por leitores de tela que leem a página em voz alta, deve ser acrescentado o atributo

- a) media-type="aural"
- b) speaker="true"
- c) media="speech"
- d) media="aural-synthesizer"
- e) accessible="true"

Comentários:

O atributo "media" especifica para qual tipo de dispositivo ou mídia o arquivo CSS deve ser aplicado. Especificando "media=speech" indica que o arquivo CSS deve ser aplicado apenas quando a página é lida em voz alta por um leitor de tela. <link type="text/css" href="som.css" media="speech">.

Gabarito: Letra C

16.(FCC –DPE-AM – 2018) A largura do contêiner <div> definida no fragmento CSS3 abaixo é de 360px, pois o padding adiciona 30px de margem interna à esquerda e 30px à direita.

```
div {  
    width: 300px;  
    padding: 30px;  
}
```

Para que o valor definido no padding não seja acrescentado aos 300px de largura do contêiner <div>, deve-se incluir no fragmento de código a instrução

- a) weight-width: fixed;
- b) max-width:300px;
- c) width-dimension: fixed;
- d) box-sizing: border-box;
- e) padding-render: none;

Comentários:

A instrução "box-sizing: border-box;" garante que o padding e o border não serão acrescentados ao tamanho da largura do contêiner <div>, e sim incluídos dentro da área definida para o tamanho do contêiner. Dessa forma, a largura do contêiner <div> permanecerá com 360px, independentemente das margens internas adicionadas pelo padding.

Gabarito: Letra D

17.(FCC– DPE-AM – 2018) A instrução CSS3 margin: 25px 50px 75px; significa que a margem

- a) inferior tem 0px.
- b) direita e a esquerda têm 50px.
- c) superior tem 75px.
- d) direita tem 75px.
- e) esquerda tem 25px.

Comentários:

A margem superior do elemento será de 25 pixels, a margem direita será de 50 pixels, a margem inferior será de 75 pixels e a margem esquerda também será de 50 pixels. Vamos relembrar a propriedade margin? "margin" é uma propriedade CSS que define a margem externa de um elemento HTML. É usado para criar espaços entre elementos e suas bordas. A margem pode ser definida para todos os lados de uma vez (margin: 10px), ou individualmente para cada lado (margin-top, margin-right, margin-bottom, margin-left). Você pode usar valores positivos ou negativos para criar sobreposições de elementos. A propriedade margin é amplamente utilizada na maioria dos estilos CSS para garantir uma melhor distribuição dos elementos na página.

Gabarito: Letra B

18.(FCC – TJ MA– 2019) Considere a página web abaixo.

```
<!DOCTYPE html>
<html>
<head>
<style>
..!.. {
  color: #0000ff;
  font-size:300%;
}
</style>
</head>
<body>
<h1 class="tit">História do Tribunal de Justiça</h1>
<p>A história do Tribunal de Justiça do Maranhão reflete a própria
evolução da Justiça maranhense, que...</p>
</body>
</html>
```

A instrução CSS que deve ser colocada na lacuna I para que somente na primeira letra do título seja aplicada a cor e o tamanho da fonte é:

- a) h1#tit::first-character
- b) h1.tit::first-letter
- c) h1.tit:initial-letter
- d) h1#tit::first-letter
- e) h1.tit::captive-letter

Comentários:

Pessoal, o correto é h1.tit::first-letter. A propriedade first-letter em CSS é usada para selecionar e estilizar somente a primeira letra de um elemento, como um título ou um parágrafo. É possível usar a propriedade para aplicar uma cor diferente, um tamanho de fonte diferente, como é o caso da questão, ou uma formatação de fonte diferente apenas na primeira letra de um título.

Gabarito: Letra B

19.(FCC – ATTIFM Pref Manaus – 2019) Um programador desenvolveu a página web abaixo.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.caixa {
  box-sizing: border-box;
  width: 50%;
  border: 5px solid #000;
  float: left;
  text-align:center
}
</style>
</head>
<body>
  <div class="caixa">Caixa do lado esquerdo</div>
  <div class="caixa">Caixa do lado direito</div>
</body>
</html>
```

Se o programador trocar o valor da propriedade box-sizing de border-box para content-box os contêineres com class="caixa" provavelmente

- a) não terão bordas, permanecendo apenas seus conteúdos.
- b) diminuirão de tamanho e haverá uma margem entre eles.
- c) permanecerão um ao lado do outro, um à esquerda e um à direita.
- d) serão colocados um dentro do outro.
- e) serão posicionados um abaixo do outro.

Comentários:

Pessoal, caso o programador trocar o valor da propriedade box-sizing de border-box para content-box os contêineres com class="caixa" provavelmente serão posicionados um abaixo do outro

Isso ocorre porque, ao alterar o valor da propriedade box-sizing de border-box para content-box, o tamanho do elemento não incluirá mais o tamanho da borda e do preenchimento, o que pode fazer com que o elemento não caiba mais ao lado do outro. Além disso, a propriedade float pode ter seu comportamento afetado pela mudança na propriedade box-sizing.

Gabarito: Letra E

20. (FCC –SANASA – 2019) Em uma página web um Analista de TI criou um contêiner com outros 3 contêineres em seu interior, como mostra a imagem abaixo.



Os códigos CSS e HTML são mostrados abaixo.

```
.container {  
  ...I...  
  border:solid #c0c0c0; 1px  
}  
.container > div {  
  background-color: #c0c0c0;  
  margin: 10px;  
  padding: 20px;  
  font-size: 30px;  
}
```

```
<div class="container">  
  <div>Água</div>  
  <div>Esgoto</div>  
  <div>Outros</div>  
</div>
```

Para conseguir a disposição dos contêineres mostrada na figura, a lacuna I deve ser corretamente preenchida por

- a) float:left;
- b) display: flex;
- c) position:horizontal;
- d) align:side-by-side;
- e) position:flexible;

Comentários:

Vamos relembrar propriedade display: flex! Display: flex é um valor de CSS que transforma um elemento HTML em um container flexível. Isso significa que os elementos filhos dentro desse container podem ser alinhados em diferentes direções e dimensionados de acordo com a largura ou altura disponíveis. Assim, podemos ter por exemplo `.flex-container { display: flex; }` No caso da questão, o preenchimento da lacuna com `"display: flex;"` vai definir o comportamento dos elementos filhos do contêiner como flexíveis, permitindo que eles sejam dispostos em uma linha ao invés de um abaixo do outro.

Gabarito: Letra B

21. (FCC –Pref Manaus – 2019) Quando se está desenvolvendo um site responsivo utilizando HTML5, em todas as páginas do site é aconselhado utilizar uma tag que fornecerá instruções ao navegador sobre como controlar as dimensões e escalas da página. Trata-se da tag

- a) `<style content="width=device-auto, initial-scale=0, final-scale=100"/>`
- b) `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
- c) `<layout size="auto" inicial-scale="0" final-scale="100" content="all-content">`
- d) `<style content="with=auto, height=auto, scale=responsive"/>`
- e) `<meta name="responsive" content="width=device-width, max-scale=100%">`

Comentários:

Pessoal, a instrução sobre como controlar as dimensões e escalas da página é: `<meta name="viewport" content="width=device-width, initial-scale=1.0">` A tag `<meta name="viewport" content="width=device-width, initial-scale=1.0">` é utilizada para fornecer instruções ao navegador sobre como controlar as dimensões e escalas de uma página em um site responsivo. Ela especifica a largura do "viewport" para ser igual à largura do dispositivo, ou seja, o navegador deve exibir a página com o tamanho apropriado para o dispositivo em que está sendo visualizada. Além disso, a escala inicial é definida como 1.0, ou seja, a página será exibida na sua escala original. Essa tag é usada como uma boa prática em todas as páginas do site para garantir que a página seja exibida corretamente em diferentes tipos de dispositivos.

Gabarito: Letra B

22. (FCC – METRO SP – 2019) Em uma página web há um contêiner chamado trem, que possui a seguinte configuração CSS.

```
div.trem {  
  width: 200px;  
  height: 200px;  
  background-color: red;  
  animation-name: metro;  
  animation-duration: 4s;  
}
```

```
..l.. metro {
```

```
from {background-color: red;}  
to {background-color: blue;}  
}
```

Para que a animação chamada metro mude a cor de fundo do contêiner de vermelho para azul, a lacuna I deve ser corretamente preenchida por

- a) @media animation
- b) @function
- c) @keyframes
- d) @transient
- e) @animation

Comentários:

Para que a animação chamada metro mude a cor de fundo do contêiner de vermelho para azul, a lacuna I deve ser corretamente preenchida por @keyframes. @keyframes é uma regra de animação CSS que permite definir a sequência de estilos para uma animação. Com ela, é possível especificar diferentes estilos para diferentes momentos da animação, como, por exemplo, uma transição suave de uma cor para outra. A regra @keyframes é referenciada na propriedade "animation-name" em um elemento HTML, especificando qual animação será aplicada ao elemento. Vejamos rapidamente as demais regras. @media é uma regra CSS utilizada para aplicar estilos diferentes para diferentes tipos de dispositivos, como por exemplo, mostrando um estilo diferente em tela cheia do que em dispositivos móveis. Já @function é uma regra CSS utilizada para definir funções customizadas que podem ser reutilizadas em outras partes do código CSS. Por fim, @transient e @animation não são diretivas reconhecidas pelo CSS. 😊

Gabarito: Letra C

23. (FCC – Pref Manaus– 2019) Considere o fragmento de uma página web criada com HTML5 e CSS3.

```
<head>  
<style>  
  [title~=carro] {  
    border: 5px solid #FF0000;  
  }  
</style>  
</head>  
<body>  
    
    
    
    
    
</body>
```

A configuração CSS3 de borda

- a) não será aplicada, pois não é possível aplicar uma configuração CSS usando o atributo title s.
- b) será aplicada a apenas à imagem carro3.gif, que possuir o atributo title="carro".
- c) será aplicada a todos os elementos img cujo atributo title contenha a palavra carro.
- d) será aplicada a todos os elementos img cujo atributo title contenha uma string terminada pela palavra carro.
- e) será aplicada somente aos elementos cujo atributo title contenha uma string iniciada pela palavra carro.

Comentários:

A configuração CSS3 de borda será aplicada a todos os elementos img cujo atributo title contenha a palavra "carro", pois o seletor utilizado na regra CSS é [title~=carro]. Isso significa que todos os elementos img que tenham um atributo title cujo valor contenha a palavra "carro" serão selecionados e a borda será aplicada a eles.

Gabarito: Letra C

Questões FGV

1. (FGV– TRT - 13ª Região (PB) – 2022) Numa codificação CSS, a propriedade z-index refere-se
- a) à ordem de um elemento numa pilha.
 - b) à ordenação crescente de uma lista de elementos.
 - c) à ordenação decrescente de uma lista de elementos.
 - d) ao valor zero em medidas de altura.
 - e) ao valor zero em medidas de largura.

Comentários:

Numa codificação CSS, a propriedade z-index refere-se à profundidade ou o nível de sobreposição de um elemento em relação a outros elementos numa pilha. A propriedade z-index é usada para controlar a ordem de superposição de elementos na página, onde elementos com um valor z-index maior ficam acima de elementos com um valor z-index menor. Isso significa que, se dois elementos são sobrepostos, o elemento com o z-index maior será mostrado na frente do elemento com o z-index menor. Valores negativos de z-index são permitidos, permitindo que elementos estejam abaixo da linha de base do documento.

Gabarito: Letra A

2. (FGV –SEFAZ-AM– 2022) Maria escolheu utilizar uma folha de estilo em cascata (CSS3) para controlar a aparência das páginas do seu site de Internet.

A sintaxe do seletor CSS que Maria deve usar para aplicar um determinado estilo somente aos elementos que estiverem diretamente dentro de elementos , é

- a) ul.li
- b) ul > li
- c) ul + li
- d) ul ~ li
- e) ul , li

Comentários:

A sintaxe do seletor CSS que Maria deve usar para aplicar um determinado estilo somente aos elementos que estiverem diretamente dentro de elementos , é ul > li. Esta sintaxe define um seletor aninhado que aplica o estilo apenas aos elementos que são filhos diretos de elementos . ul > li { /* estilo a ser aplicado */ }. Por fim, vejamos as demais alternativas. a) O seletor "ul.li" casa elementos com a classe "li" que são descendentes de um elemento de lista não ordenada (ul). c) O seletor "ul + li" corresponde ao primeiro elemento da lista de irmãos (li) que segue imediatamente um elemento da lista não ordenada (ul). d) O seletor "ul ~ li" corresponde a todos os elementos de lista de itens irmãos (li) que vêm após um elemento de lista não ordenada (ul). e) O seletor "ul, li" corresponde a qualquer elemento de lista não ordenada (ul) e a qualquer elemento de lista (li). Este seletor é usado para selecionar vários elementos e aplicar estilos a todos eles.

3. (FGV– MPE-SC –2022) Considere o conteúdo de uma página web simplificada, exibido a seguir.

```
<!DOCTYPE html>  
<html> <head> <XXXXXXX rel="stylesheet" href="mystyle.css"> </head> <body> <h1>This  
is a heading</h1> </body> </html>
```

O elemento que deve substituir a string XXXXXXXX, na quarta linha, para especificar corretamente a referência a uma folha de estilos externa (external style sheet) é:

- a) import
- b) include
- c) link
- d) ref
- e) url

Comentários:

O elemento correto é o <link> para especificar a referência a uma folha de estilos externa (external style sheet). A tag <link> é usada para vincular uma folha de estilo externa chamada mystyle.css à página. O atributo href especifica o caminho para o arquivo CSS, enquanto o atributo rel indica que o link é para uma folha de estilo. Além disso, podemos usar a tag <style> no cabeçalho da página.

4. (FGV – TRT - 16ª REGIÃO (MA) – 2022) No contexto do HTML e CSS, analise o código a seguir numa página Web.

```
p.centro {  
  text-align: center;  
  color: blue;  
}
```

Sobre esse trecho, é correto afirmar que a cor e o alinhamento serão aplicados

- a) à classe denominada "p.centro".
- b) à classe denominada "p".
- c) aos elementos com a tag <p> que tenham a classe "centro".
- d) aos elementos com id = "p" que tenham a classe "centro".
- e) aos elementos que tenham a classe "p.centro".

Comentários:

A cor e o alinhamento serão aplicados a todos os elementos <p> que tenham a classe "centro". Os seletores em CSS são a maneira de selecionar elementos HTML aos quais se deseja aplicar um estilo. Eles permitem que você especifique uma regra de estilo para um elemento específico, como por exemplo, todos os elementos <p>, ou para uma classe de elementos, como todos os elementos com class="destaque".

Gabarito: Letra C

5. (FGV–MPE-GO – 2022) No contexto do CSS, assinale o seletor (selector) que seleciona para aplicação os elementos com o atributo class=xpto.

- a) #xpto { ... }
- b) (xpto) { ... }
- c) *xpto { ... }
- d) .xpto { ... }
- e) xpto { ... }

Comentários:

Os seletores de classe selecionam elementos HTML que tenham uma classe específica, utilizando o seletor ".". Por exemplo: ".minha-classe" seleciona todos os elementos com a classe "minha-classe". ".xpto" seleciona para aplicação os elementos com o atributo class=xpto.

Gabarito: Letra D

6. (FGV– PC-AM –2022) No contexto da linguagem de estilo CSS, considere os seletores utilizados no trecho a seguir.

```
* { font-family: Tahoma, Verdana, sans-serif;
}
h1 {
  text-align: center;
  color: red;
}
#xpto { color: blue;
}
```

Assinale o alcance de cada um dos três seletores, na ordem.

- a) Elementos <*>; elementos <h1>; elementos com name="xpto".
- b) Todos os elementos da página; elementos da classe h1; elementos com id="#xpto".
- c) Todos os elementos da página; elementos <h>; elementos com id="xpto".
- d) Todos os elementos da página; elementos da classe h1; elementos com id="#xpto".
- e) Todos os elementos da página; elementos <h1>; elementos com id="xpto".

Comentários:

O seletor "*" seleciona todos os elementos da página, ou seja, todos os elementos HTML presentes na página terão o estilo aplicado. O seletor "h1" seleciona somente os elementos h1 da página, ou seja, somente os elementos h1 terão o estilo aplicado. O seletor "#xpto" seleciona somente os elementos com ID igual a "xpto", ou seja, somente os elementos com ID "xpto" terão o estilo aplicado.

Gabarito: Letra E

7. (FGV –SEMSA Manaus – 2022) Dentre os formatos de seletores que podem ser definidos no CSS3, a expressão

E[xxx="yyy"]

casaria com o elemento E,

- a) cuja classe "xxx" é sinônima da classe "yyy".
- b) cujo valor do atributo "xxx" contém em qualquer posição a string "yyy".
- c) cujo valor do atributo "xxx" é igual a "yyy".
- d) cujo valor do atributo "xxx" inicia com a string "yyy".
- e) cujo valor do atributo "xxx" termina com a string "yyy".

Comentários:

O seletor de atributo "E[xxx="yyy"]" é um seletor CSS que permite selecionar elementos HTML com base em valores específicos de seus atributos. "E" representa o **nome do elemento** que desejamos selecionar. Já, "xxx" representa o **nome do atributo** que desejamos selecionar. Por fim, "yyy" representa o **valor do atributo** que desejamos selecionar. Portanto, a expressão "E[xxx="yyy"]" seleciona o elemento "E" cujo valor do atributo "xxx" é igual a "yyy". Esse é um seletor de atributo que permite selecionar elementos com base em valores específicos de seus atributos. Além do seletor "=", há outros seletores de atributo que podem ser usados, incluindo "^=", "*=", "|=" e "~=". Esses seletores permitem selecionar elementos com base em valores parciais ou que começam ou terminam com um determinado valor.

Gabarito: Letra C

8. (FGV –Banestes – 2021) No contexto do uso de CSS para aplicar estilos a elementos HTML numa página web, o seletor universal, que referencia todos os elementos, é identificado pelo símbolo:

- a) #
- b) %
- c) &
- d) *
- e) @

Comentários:

O seletor "*" seleciona todos os elementos da página, ou seja, todos os elementos HTML presentes na página terão o estilo aplicado. O seletor universal seleciona todos os elementos em uma página HTML, independentemente de sua tag HTML ou atributos. O seletor universal (*) é uma das ferramentas mais básicas e poderosas em CSS. Além de selecionar todos os elementos em uma página HTML, ele também pode ser usado como um seletor de contexto para limitar a escopo de outros seletores. Por exemplo, a seguinte regra CSS aplicaria a cor vermelha apenas aos elementos filhos diretos de um elemento com a classe "container". O seletor universal também é útil para redefinir estilos padrão de elementos HTML para um estilo consistente em toda a página.

Gabarito: Letra D

9. (FGV –IMBEL –2021) No contexto do CSS numa página Web, assinale o script que estabelece que todos os elementos com id="p" serão exibidos de acordo com os atributos especificados.

- a) p { font-size: 20px; color: blue}
- b) .p { font-size: 20px; color: blue}
- c) #p { font-size: 20px; color: blue}
- d) <p> { font-size: 20px; color: blue}
- e) :p { font-size: 20px; color: blue}

Comentários:

O seletor CSS que estabelece que todos os elementos com o ID "p" serão exibidos de acordo com os atributos especificados é o #p. O seletor de ID é identificado pelo símbolo de hashtag (#) seguido pelo nome do ID. Neste caso, o ID é "p".

Gabarito: Letra C

10.(FGV–TJ-RO – 2021) No contexto do CSS, considere os seguintes seletores.

h1, xxxx .xxxx, #d345 #xxxx xxxx:hover

A lista que indica corretamente a natureza do objeto correspondente ao símbolo "xxxx" em cada linha, respectivamente, é:

- a) classe xxxx / tag / classe xxxx / classe xxxx
- b) classe xxxx / classe xxxx / classe xxxx / tag
- c) tag / classe xxxx / id "xxxx" / tag
- d) tag / classe xxxx / id "xxxx" / classe xxxx
- e) tag / tag / classe xxxx / id "xxxx"

Comentários:

Na lista dos seletores CSS apresentados, a natureza do objeto correspondente ao símbolo "xxxx" em cada linha, respectivamente, é:

1. h1 - Este é um seletor de tag HTML. O elemento h1 representa um título de nível 1.

2. .xxxx - Este é um seletor de classe. As classes são atribuídas a elementos HTML específicos e permitem que você aplique estilos de forma personalizada a esses elementos.
3. #d345 - Este é um seletor de ID. Os IDs são atribuídos a um elemento HTML específico e são usados para aplicar estilos exclusivos a esse elemento.
4. #xxxx - Este é também um seletor de ID.
5. xxxx: hover - Este é um seletor de pseudo-classe. As pseudo-classes são usadas para aplicar estilos em resposta a um determinado estado ou interação do usuário, como o mouse pairando sobre um elemento (:hover).

Gabarito: Letra C

11.(FGV– FUNSAÚDE – CE – 2021) No contexto do CSS3, assinale o script que define corretamente a formatação para todos os elementos h1 de um documento.

- a) .h1 { color: white; text-align: center; }
- b) h1 { color= white, background= yellow }
- c) #h1 { background: yellow, family-font: georgia }
- d) h1 { background-color: yellow; font-family: verdana; }
- e) \$h1 { text-align= center; background-color= yellow }

Comentários:

O script CSS3 que define corretamente a formatação para todos os elementos h1 de um documento é o h1 { background-color: yellow; font-family: verdana; }. Ela define a cor de fundo como amarela para todos os elementos h1 no documento, e define a fonte Verdana como fonte para esses elementos h1. O seletor de tag HTML é identificado pelo nome da tag (h1). O conteúdo entre as chaves representa a formatação específica que será aplicada a todos os elementos h1 no documento.

Gabarito: Letra D

12.(FGV – FunSaúde CE– 2021) No contexto da formatação de páginas Web, assinale o papel da propriedade padding no CSS em relação a um box.

- a) Definir a altura e a largura.
- b) Definir a espessura da área que separa as bordas do preenchimento.
- c) Definir a espessura das margens externas.
- d) Definir a espessura e o estilo da borda.
- e) Definir a unidade utilizada para estabelecer as dimensões.

Comentários:

A propriedade "padding" no CSS tem como papel definir a espessura da área que separa as bordas do preenchimento de um box ou elemento HTML. Ela adiciona uma margem interna em volta do conteúdo de um elemento, deixando um espaço entre o conteúdo e a borda. A propriedade "padding" é útil para controlar o espaço entre o conteúdo de um elemento e suas

bordas, permitindo que você personalize a aparência de um elemento sem afetar seu tamanho real. Além disso, ela é útil para controlar a distância entre elementos adjacentes na página.

Gabarito: Letra B

13.(FGV –IMBEL – 2021) Analise o trecho de código CSS3 a seguir.

```
p {  
  
    color: red;  
  
    text-align: center;  
}
```

Assinale o efeito causado por essa declaração.

- a) Estilizar todos os elementos <p>.
- b) Definir uma classe CSS intitulada p.
- c) Definir uma variável CSS intitulada p.
- d) Estilizar todos os elementos com id=p.
- e) Um erro, pois falta definir o seletor da regra.

Comentários:

A declaração citada na questão especifica uma regra CSS que se aplica a todos os elementos "p" da página. Ela define que o texto dentro desses elementos deve ser exibido em vermelho e centralizado (text-align: center). Assim, todos os elementos "p" na página serão estilizados com as propriedades especificadas na regra CSS, incluindo a cor vermelha para o texto e o alinhamento central.

Gabarito: Letra A

14.(FGV –IMBEL – 2021) Com relação às regras escritas em CSS3, o papel da propriedade padding em um box é definir

- a) o padrão de preenchimento.
- b) a densidade da cor de fundo.
- c) o espaçamento entre as letras no texto.
- d) a espessura da área que separa os parágrafos.
- e) a espessura da área que separa a borda do conteúdo.

Comentários:

A propriedade "padding" em CSS define a espessura da área que separa a borda de um elemento (box) do seu conteúdo. O padding é a área vazia dentro da borda e ao redor do conteúdo do elemento. A propriedade padding pode ser especificada para cada um dos quatro lados de um elemento (superior, direito, inferior e esquerdo) ou pode ser especificada para todos os lados de uma só vez.

Gabarito: Letra E

15. (FGV –IMBEL – 2021) No contexto do CSS numa página Web, assinale o script que estabelece que todos os elementos <p> serão exibidos de acordo com a definição dos estilos.

- a) p { font-size: 20px; color: blue}
- b) .p { font-size: 20px; color: blue}
- c) #p { font-size: 20px; color: blue}
- d) <p> { font-size: 20px; color: blue}
- e) :p { font-size: 20px; color: blue}

Comentários:

Pessoal, para que todos os elementos <p> serão exibidos de acordo com a definição dos estilos, devemos usar p { font-size: 20px; color: blue}, sem nenhum seletor (".", #, ou < >). A regra CSS {font-size: 20px; color: blue} define que todos os elementos <p> deverão ter uma fonte de tamanho 20 pixels e cor azul. Todos os elementos <p> na página seguirão essa definição de estilo.

Gabarito: Letra A

16. (FGV –IMBEL – 2021) No contexto do CSS numa página Web, assinale o script que estabelece que todos os elementos com id="p" serão exibidos de acordo com os atributos especificados.

- a) p { font-size: 20px; color: blue}
- b) .p { font-size: 20px; color: blue}
- c) #p { font-size: 20px; color: blue}
- d) <p> { font-size: 20px; color: blue}
- e) :p { font-size: 20px; color: blue}

Comentários:

O script que estabelece que todos os elementos com id="p" serão exibidos de acordo com os atributos especificados é: #p { font-size: 20px; color: blue}. Pessoal, esse é um seletor "#" muito utilizado.

Gabarito: Letra C

LISTA DE QUESTÕES

Questões CESPE

1. (CESPE – SEPLAN RR– 2023) Com relação a programação e desenvolvimento de sistemas, julgue o item a seguir.

CSS é um código separado da HTML que pode afetar a aparência das tags em uma única página ou em todo um site da Web.

2. (CESPE –TCE RJ– 2022) Quanto ao desenvolvimento de sistemas web, julgue o item seguinte.

A versão 3 do CSS trouxe para as bordas das páginas web as opções de definição de cor, arredondamento de cantos e multiplicidade de linhas.

3. (CESPE – DP DF – 2022) Julgue o item a seguir, acerca de CSS3, JMS, JSON e JUnit.

Quando se utiliza CSS3, para que os flex items sejam apresentados na mesma ordem em que aparecem no HTML, é necessário atribuir o valor 1 à propriedade order.

4. (CESPE – PETROBRAS – 2022)

Julgue o próximo item que trata de CSS, JavaScript e Net Core.

No código abaixo, escrito na linguagem CSS, red é um valor do tipo palavra-chave, enquanto #f00 é um valor do tipo notação funcional.

```
p {  
color: red;  
background-color: #f00;  
}
```

5. (CESPE – ALECE – 2021) Web designers costumam usar CSS e HTML na construção de um sítio eletrônico, já que, juntos, eles definirão como as páginas serão exibidas nos navegadores, conforme exemplificado no código a seguir.

```
<html>  
<head>
```

```
<style type="text/css">
body {background-color: yellow}
h1 {background-color: #00ff00}
h2 {background-color: transparent}
h3 {background-color: rgb(250,0,0)}
p {background-color: rgb(250,0,255)}
</style>

</head>

<body>

<h1>Este é título 1</h1>
<h2>Este é título 2</h2>
<h3>Este é título 3</h3>
<p>Este é um parágrafo</p>

</body>
</html>
```

Considerando-se o código anterior, o comando da cor verde é

- a) background-color: rgb(250,0,0).
 - b) background-color: yellow.
 - c) background-color: #00ff00.
 - d) background-color: transparente.
 - e) background-color: rgb(250,0,255).
6. (CESPE – TJ PA – 2020) Assinale a opção que indica a propriedade usada no CSS3 para definir o alinhamento de um elemento inline com relação ao seu elemento-pai.
- a) alignment-baseline
 - b) alignment-adjust
 - c) background-image
 - d) background-position
 - e) line-boxes
7. (CESPE – ME– 2020) Acerca de desenvolvimento de sistemas web, julgue o item a seguir.

No CSS 3, o uso da propriedade box-shadow e da função linear-gradient possibilita a criação de formas, cores e efeitos das páginas diretamente no código, de modo que os resultados aparecem diretamente nos navegadores.

8. (CESPE – MPC TCE-PA – 2019) O uso de CSS permite a criação de padrões para a publicação de páginas na Internet, com possibilidade de redução da quantidade de informações em cada página. Em relação a esse assunto, a propriedade que deve ser configurada quando é necessário posicionar-se uma legenda de uma tabela é

- a) outline.
- b) padding-top.
- c) caption-side.
- d) table-layout.
- e) text-combine-upright.

9. (CESPE – TJ AM– 2019) Acerca do desenvolvimento web mediante o uso do HTML 5, do JavaScript, do XML e do CSS, julgue o item subsequente.

O CSS permite anexar estilos a elementos estruturados, em documentos HTML ou em aplicativos XML. Com o CSS 3, é possível flutuar o <nav>, colocar bordas no <header> e no <footer>, além de incluir margens e deslocamentos no <article>.

10. (CESPE – TJ AM – 2019) Com relação a desenvolvimento em Java para Web, julgue o item que se segue.

De acordo com o conceito de herança em CSS, as propriedades de página com especificidade mais alta têm prioridade sobre as propriedades com especificidade mais baixa.

Questões FCC

1. **(FCC – TJ-CE – 2022)** Para colocar a cor de fundo vermelha apenas dos campos (inputs) do tipo text de um formulário, utiliza-se a instrução CSS
 - a) `input [type=text] {background-color: #00ff00}`
 - b) `input.type [text] {background-color: #ff0000}`
 - c) `input [type=text] {background-color: #ff0000}`
 - d) `input.type|text {background: rgb(0,255,0)}`
 - e) `input [type('text')] {background-color:rgb(255,0,0)}`

2. **(FCC – TRT - 4ª REGIÃO (RS) – 2022)** Quando um Técnico criou uma página web usando HTML e CSS, um elemento ficou sobreposto ao outro na tela e ele deseja inverter a ordem de sobreposição desses elementos. Para definir ou mudar a ordem de sobreposição dos elementos ele deve utilizar a propriedade CSS
 - a) z-index
 - b) inverter
 - c) position
 - d) flex-position
 - e) x-order

3. **(FCC – PGE AM – 2022)** Na utilização externa de CSS em páginas HTML, deve-se indicar um arquivo CSS que será usado na definição dos estilos dos elementos da página. Supondo que o arquivo a ser utilizado é o estilos.css, a instrução correta é
 - a) `<@import type="style/css" src="estilos.css">`
 - b) `<link rel="stylesheet" href="estilos.css">`
 - c) `<style type="css" src="estilos.css"></style>`
 - d) `<link type="stylesheet" src="estilos.css">`
 - e) `<@require file="estilos.css" type="style/css">`

4. **(FCC – MANAUSPREV – 2021)** No formulário de contato de uma página web do site do Governo de XX, um desenvolvedor de software precisa especificar que todos os campos do tipo texto (text) devem ser mostrados com fonte azul. Para estabelecer este estilo usando a linguagem CSS, deve-se utilizar o comando
 - a) `input [type=text] {color:blue}`
 - b) `input type='text' {color:blue}`
 - c) `input.text {color:blue}`

- d) `input!text {color:#0000ff}`
- e) `input#text {color:#blue}`

5. (FCC– TJ-SC – 2021) Considere o fragmento de código HTML5, abaixo:

```
<body>
  <div class="topo">
    <div class="logo"></div>
    <div class="slogan"></div>
  </div>
</body>
```

Considere, também, o fragmento de código CSS, abaixo, para o código HTML5 apresentado.

```
div.topo{width:100%;height:150px;background-color:yellow; Imagem associada para
resolução da questão }
div.logo{background-color:blue; flex:1}
div.slogan{background-color:green; flex:1}
```

Para que as divisões logo e slogan apareçam uma do lado da outra, com tamanhos iguais, preenchendo toda a divisão topo, a lacuna I deve ser corretamente preenchida por

- a) `display:inline`
- b) `float:left`
- c) `display:flex`
- d) `position:relative`
- e) `display:block`

6. (FCC – AL-AP – 2020) Segundo os padrões de acessibilidade na web definidos para sites do Governo Eletrônico, códigos CSS devem estar sempre em arquivos externos a serem chamados pelas páginas do site/portal. Por exemplo, a instrução `<link rel="stylesheet" type="text/css" href="aleap.css"/>` pode ser também escrita na forma:

- a) `<style rel="link" type="text/css"> @include url("aleap.css"); </style>`
- b) `<link lang="css" type="stylesheet" href="aleap.css"/>`
- c) `<style rel="stylesheet" type="text/css"> @import url("aleap.css"); </style>`
- d) `<link rel="external" type="text/css" src="aleap.css"/>`
- e) `<style rel="stylesheet" type="text/css" url("aleap.css")/>`

7. (FCC– AL-AP – 2020) Em um contêiner criado pela tag `div` há vários contêineres menores. Para que estes contêineres internos sejam posicionados um ao lado do outro horizontalmente, pode ser utilizada, em todos eles, a instrução CSS:

- a) align:inline
- b) float:left
- c) display:left
- d) position:inline
- e) aling:inline-block

8. (FCC – TJ-MA – 2019) A propriedade display na linguagem CSS3 especifica como um elemento será exibido no layout da página web. Comparado ao display:block, a principal diferença é que o display:inline-block

- a) exibe parte do elemento da página na horizontal e parte na vertical.
- b) não adiciona quebra de linha após o elemento, permitindo que este se ajuste ao lado de outros elementos.
- c) não permite definir uma largura e altura para o elemento.
- d) não permite definir as margens e os paddings do elemento.
- e) adiciona quebra de linha após o elemento, não permitindo que este se ajuste ao lado de outros elementos.

9. (FCC – SANASA Campinas – 2019) Um Desenvolvedor de software precisa inserir uma instrução no cabeçalho de uma página HTML que fará referência a um arquivo chamado a001.css a ser aplicado apenas quando a página for aberta em dispositivos com tela de até 600 pixels. A instrução correta que deverá ser inserida é

- a) <@import URL(a001.css) only screen and (max-width: 600px)>
- b) <link rel="media" device="only screen with (max-width: 600px)" href="a001.css">
- c) <link rel="stylesheet" media="screen and (max-width: 600px)" href="a001.css">
- d) <inport file="a001.css" media="screen and (max-width: 600px)">
- e) <style>@media only screen and (min-width: 600px) URL(a001.css) </style>

10. (FCC – Prefeitura de Manaus - AM – 2019) Para que, ao se posicionar o ponteiro do mouse sobre cada um dos links da página, a cor da letra do link mude para vermelha, deve-se utilizar para a página web a configuração CSS

- a) a:hover {color: #00FF00}.
- b) a:over {font-color: #0000FF}.
- c) a:hover {font-color: #FF0000}.
- d) a:over {color: #00FF00}.
- e) a:hover {color: #FF0000}

11. (FCC –TRF - 4ª REGIÃO – 2019) As unidades de medida usadas em CSS para ajudar na responsividade dos sites, que especificam medidas relativas a 1% da largura e 1% da altura da janela do navegador (viewport), são, respectivamente,

- a) px e pt.
- b) cm e pc.
- c) vw e vh.
- d) em e ex.
- e) em e p

12. (FCC –SEFAZ-BA – 2019) Em uma página web criada com HTML5 e CSS3 há 3 contêineres com nome de classe caixa, criados com a tag div. Criou-se para estes contêineres a seguinte configuração CSS.

```
div.caixa{  
    width:300px;  
    height:200px;  
    border:solid #c0c0c0 1px;  
}
```

Para que os contêineres sejam posicionados um ao lado do outro horizontalmente deve-se adicionar ao bloco CSS acima a instrução

- a) position: side -by-side;
- b) box-position: left;
- c) position: relative;
- d) align: side-by-side;
- e) float: left;

13. (FCC– AFAP – 2019) No CSS3 podem ser usadas diversas unidades de medida para definir o tamanho pelo qual os elementos são renderizados na página web quando aberta na janela do navegador. Algumas dessas unidades de medida são relativas e adequam o tamanho do elemento proporcionalmente ao tamanho da janela. Duas dessas unidades de medida são descritas abaixo, em inglês.

I. Relative to 1% of the width of the viewport (the browser window size).

II . Relative to 1% of the height of the viewport (the browser window size).

I e II referem-se, respectivamente, às unidades de medida

- a) pw e ph.

- b) ex e ey.
- c) vx e vy.
- d) pc e pt.
- e) vw e vh.

14.(FCC–DPE-AM–2018) A instrução CSS3 `div + p {background-color: blue;}` aplica a cor de fundo azul ao elemento `<p>` colocado

- a) imediatamente após o elemento `<div>`.
- b) dentro do elemento `<div>`.
- c) próximo a um elemento `<div>`.
- d) imediatamente antes do elemento `<div>`.
- e) como filho do elemento `<div>`.

15. (FCC–DPE-AM – 2018) Um Técnico Programador usou a referência abaixo para um arquivo CSS em uma página web construída com HTML.

```
<link type="text/css" href="som.css">
```

Para que esse arquivo CSS seja usado por leitores de tela que leem a página em voz alta, deve ser acrescentado o atributo

- a) `media-type="aural"`
- b) `speaker="true"`
- c) `media="speech"`
- d) `media="aural-synthesizer"`
- e) `accessible="true"`

16. (FCC –DPE-AM – 2018) A largura do contêiner `<div>` definida no fragmento CSS3 abaixo é de 360px, pois o padding adiciona 30px de margem interna à esquerda e 30px à direita.

```
div {  
    width: 300px;  
    padding: 30px;  
}
```

Para que o valor definido no padding não seja acrescentado aos 300px de largura do contêiner `<div>`, deve-se incluir no fragmento de código a instrução

- a) `weight-width: fixed;`

- b) max-width:300px;
- c) width-dimension: fixed;
- d) box-sizing: border-box;
- e) padding-render: none;

17. (FCC– DPE-AM – 2018) A instrução CSS3 margin: 25px 50px 75px; significa que a margem

- a) inferior tem 0px.
- b) direita e a esquerda têm 50px.
- c) superior tem 75px.
- d) direita tem 75px.
- e) esquerda tem 25px.

18. (FCC – TJ MA– 2019) Considere a página web abaixo.

```
<!DOCTYPE html>
<html>
<head>
<style>
..l.. {
  color: #0000ff;
  font-size:300%;
}
</style>
</head>
<body>
<h1 class="tit">História do Tribunal de Justiça</h1>
<p>A história do Tribunal de Justiça do Maranhão reflete a própria
evolução da Justiça maranhense, que...</p>
</body>
</html>
```

A instrução CSS que deve ser colocada na lacuna I para que somente na primeira letra do título seja aplicada a cor e o tamanho da fonte é:

- a) h1#tit::first-character
- b) h1.tit::first-letter
- c) h1.tit:initial-letter
- d) h1#tit::first-letter
- e) h1.tit::captive-letter

19. (FCC – ATTIFM Pref Manaus – 2019) Um programador desenvolveu a página web abaixo.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.caixa {
  box-sizing: border-box;
  width: 50%;
  border: 5px solid #000;
  float: left;
  text-align:center
}
</style>
</head>
<body>
  <div class="caixa">Caixa do lado esquerdo</div>
  <div class="caixa">Caixa do lado direito</div>
</body>
</html>
```

- Se o programador trocar o valor da propriedade box-sizing de border-box para content-box os contêineres com class="caixa" provavelmente
 - a) não terão bordas, permanecendo apenas seus conteúdos.
 - b) diminuirão de tamanho e haverá uma margem entre eles.
 - c) permanecerão um ao lado do outro, um à esquerda e um à direita.
 - d) serão colocados um dentro do outro.
 - e) serão posicionados um abaixo do outro.

20. (FCC – SANASA – 2019) Em uma página web um Analista de TI criou um contêiner com outros 3 contêineres em seu interior, como mostra a imagem abaixo.



Os códigos CSS e HTML são mostrados abaixo.

```
.container {
```

```
...l...  
border:solid #c0c0c0; 1px  
}  
.container > div {  
background-color: #c0c0c0;  
margin: 10px;  
padding: 20px;  
font-size: 30px;  
}
```

```
<div class="container">  
  <div>Água</div>  
  <div>Esgoto</div>  
  <div>Outros</div>  
</div>
```

Para conseguir a disposição dos contêineres mostrada na figura, a lacuna I deve ser corretamente preenchida por

- a) float:left;
- b) display: flex;
- c) position:horizontal;
- d) align:side-by-side;
- e) position:flexible;

21. (FCC –Pref Manaus – 2019) Quando se está desenvolvendo um site responsivo utilizando HTML5, em todas as páginas do site é aconselhado utilizar uma tag que fornecerá instruções ao navegador sobre como controlar as dimensões e escalas da página. Trata-se da tag

- a) <style content="width=device-auto, initial-scale=0, final-scale=100"/>
- b) <meta name="viewport" content="width=device-width, initial-scale=1.0">
- c) <layout size="auto" inicial-scale="0" final-scale="100" content="all-content">
- d) <style content="with=auto, height=auto, scale=responsive"/>
- e) <meta name="responsive" content="width=device-width, max-scale=100%">

22. (FCC – METRO SP – 2019) Em uma página web há um contêiner chamado trem, que possui a seguinte configuração CSS.

```
div.trem {  
width: 200px;  
height: 200px;  
background-color: red;
```

```
animation-name: metro;  
animation-duration: 4s;  
}
```

```
..l.. metro {  
  from {background-color: red;}  
  to {background-color: blue;}  
}
```

Para que a animação chamada metro mude a cor de fundo do contêiner de vermelho para azul, a lacuna I deve ser corretamente preenchida por

- a) @media animation
- b) @function
- c) @keyframes
- d) @transient
- e) @animation

23.(FCC – Pref Manaus– 2019) Considere o fragmento de uma página web criada com HTML5 e CSS3.

```
<head>  
<style>  
  [title~=carro] {  
    border: 5px solid #FF0000;  
  }  
</style>  
</head>  
<body>  
    
    
    
    
    
</body>
```

A configuração CSS3 de borda

- a) não será aplicada, pois não é possível aplicar uma configuração CSS usando o atributo title s.

- b) será aplicada a apenas à imagem carro3.gif, que possuir o atributo title="carro".
- c) será aplicada a todos os elementos img cujo atributo title contenha a palavra carro.
- d) será aplicada a todos os elementos img cujo atributo title contenha uma string terminada pela palavra carro.
- e) será aplicada somente aos elementos cujo atributo title contenha uma string iniciada pela palavra carro.

Questões FGV

1. (FGV– TRT - 13ª Região (PB) – 2022) Numa codificação CSS, a propriedade z-index refere-se

- a) à ordem de um elemento numa pilha.
- b) à ordenação crescente de uma lista de elementos.
- c) à ordenação decrescente de uma lista de elementos.
- d) ao valor zero em medidas de altura.
- e) ao valor zero em medidas de largura.

2. (FGV –SEFAZ-AM– 2022) Maria escolheu utilizar uma folha de estilo em cascata (CSS3) para controlar a aparência das páginas do seu site de Internet.

A sintaxe do seletor CSS que Maria deve usar para aplicar um determinado estilo somente aos elementos que estiverem diretamente dentro de elementos , é

- a) ul.li
- b) ul > li
- c) ul + li
- d) ul ~ li
- e) ul , li

3. (FGV– MPE-SC –2022) Considere o conteúdo de uma página web simplificada, exibido a seguir.

```
<!DOCTYPE html>
<html> <head> <XXXXXXX rel="stylesheet" href="mystyle.css"> </head> <body> <h1>This
is a heading</h1> </body> </html>
```

O elemento que deve substituir a string XXXXXXXX, na quarta linha, para especificar corretamente a referência a uma folha de estilos externa (external style sheet) é:

- a) import
- b) include
- c) link
- d) ref
- e) url

4. (FGV – TRT - 16ª REGIÃO (MA) – 2022) No contexto do HTML e CSS, analise o código a seguir numa página Web.

```
p.centro {  
  text-align: center;  
  color: blue;  
}
```

Sobre esse trecho, é correto afirmar que a cor e o alinhamento serão aplicados

- a) à classe denominada "p.centro".
- b) à classe denominada "p".
- c) aos elementos com a tag <p> que tenham a classe "centro".
- d) aos elementos com id = "p" que tenham a classe "centro".
- e) aos elementos que tenham a classe "p.centro".

5. (FGV-MPE-GO – 2022) No contexto do CSS, assinale o seletor (selector) que seleciona para aplicação os elementos com o atributo class=xpto.

- a) #xpto { ... }
- b) (xpto) { ... }
- c) *xpto { ... }
- d) .xpto { ... }
- e) xpto { ... }

6. (FGV- PC-AM –2022) No contexto da linguagem de estilo CSS, considere os seletores utilizados no trecho a seguir.

```
* { font-family: Tahoma, Verdana, sans-serif;  
}  
h1 {  
  text-align: center;  
  color: red;  
}  
#xpto { color: blue;  
}
```

Assinale o alcance de cada um dos três seletores, na ordem.

- a) Elementos <*>; elementos <h1>; elementos com name="xpto".
- b) Todos os elementos da página; elementos da classe h1; elementos com id="#xpto".
- c) Todos os elementos da página; elementos <h>; elementos com id="xpto".
- d) Todos os elementos da página; elementos da classe h1; elementos com id="#xpto"
- e) Todos os elementos da página; elementos <h1>; elementos com id= "xpto".

7. (FGV –SEMSA Manaus – 2022) Dentre os formatos de seletores que podem ser definidos no CSS3, a expressão

`E[xxx="yyy"]`

casaria com o elemento E,

- a) cuja classe "xxx" é sinônima da classe "yyy".
 - b) cujo valor do atributo "xxx" contém em qualquer posição a string "yyy".
 - c) cujo valor do atributo "xxx" é igual a "yyy".
 - d) cujo valor do atributo "xxx" inicia com a string "yyy".
 - e) cujo valor do atributo "xxx" termina com a string "yyy".
8. (FGV –Banestes – 2021) No contexto do uso de CSS para aplicar estilos a elementos HTML numa página web, o seletor universal, que referencia todos os elementos, é identificado pelo símbolo:

- a) #
- b) %
- c) &
- d) *
- e) @

9. (FGV –IMBEL –2021) No contexto do CSS numa página Web, assinale o script que estabelece que todos os elementos com `id="p"` serão exibidos de acordo com os atributos especificados.

- a) `p { font-size: 20px; color: blue }`
- b) `.p { font-size: 20px; color: blue }`
- c) `#p { font-size: 20px; color: blue }`
- d) `<p> { font-size: 20px; color: blue }`
- e) `:p { font-size: 20px; color: blue }`

10. (FGV –TJ-RO – 2021) No contexto do CSS, considere os seguintes seletores.

`h1, xxxx .xxxx, #d345 #xxxx xxxx:hover`

A lista que indica corretamente a natureza do objeto correspondente ao símbolo "xxxx" em cada linha, respectivamente, é:

- a) classe xxxx / tag / classe xxxx / classe xxxx
- b) classe xxxx / classe xxxx / classe xxxx / tag

- c) tag / classe xxxx / id "xxxx" / tag
- d) tag / classe xxxx / id "xxxx" / classe xxxx
- e) tag / tag / classe xxxx / id "xxxx"

11. (FGV– FUNSAÚDE – CE – 2021) No contexto do CSS3, assinale o script que define corretamente a formatação para todos os elementos h1 de um documento.

- a) .h1 { color: white; text-align: center; }
- b) h1 { color= white, background= yellow }
- c) #h1 { background: yellow, family-font: georgia }
- d) h1 { background-color: yellow; font-family: verdana; }
- e) \$h1 { text-align= center; background-color= yellow }

12. (FGV – FunSaúde CE– 2021) No contexto da formatação de páginas Web, assinale o papel da propriedade padding no CSS em relação a um box.

- a) Definir a altura e a largura.
- b) Definir a espessura da área que separa as bordas do preenchimento.
- c) Definir a espessura das margens externas.
- d) Definir a espessura e o estilo da borda.
- e) Definir a unidade utilizada para estabelecer as dimensões.

13. (FGV –IMBEL – 2021) Analise o trecho de código CSS3 a seguir.

```
p {  
  
color: red;  
  
text-align: center;  
}
```

Assinale o efeito causado por essa declaração.

- a) Estilizar todos os elementos <p>.
- b) Definir uma classe CSS intitulada p.
- c) Definir uma variável CSS intitulada p.
- d) Estilizar todos os elementos com id=p.
- e) Um erro, pois falta definir o seletor da regra.

14. (FGV –IMBEL – 2021) Com relação às regras escritas em CSS3, o papel da propriedade padding em um box é definir

- a) o padrão de preenchimento.
- b) a densidade da cor de fundo.
- c) o espaçamento entre as letras no texto.
- d) a espessura da área que separa os parágrafos.
- e) a espessura da área que separa a borda do conteúdo.

15. (FGV –IMBEL – 2021) No contexto do CSS numa página Web, assinale o script que estabelece que todos os elementos <p> serão exibidos de acordo com a definição dos estilos.

- a) p { font-size: 20px; color: blue}
- b) .p { font-size: 20px; color: blue}
- c) #p { font-size: 20px; color: blue}
- d) <p> { font-size: 20px; color: blue}
- e) :p { font-size: 20px; color: blue}

16. (FGV –IMBEL – 2021) No contexto do CSS numa página Web, assinale o script que estabelece que todos os elementos com id="p" serão exibidos de acordo com os atributos especificados.

- a) p { font-size: 20px; color: blue}
- b) .p { font-size: 20px; color: blue}
- c) #p { font-size: 20px; color: blue}
- d) <p> { font-size: 20px; color: blue}
- e) :p { font-size: 20px; color: blue}

GABARITO

CESPE

1. CORRETO
2. ERRADO
3. ERRADO
4. ERRADO
5. LETRA C
6. LETRA A
7. CORRETO
8. LETRA C
9. CORRETO
10. CORRETO

FCC

1. LETRA C
2. LETRA A
3. LETRA B
4. LETRA A
5. LETRA C

6. LETRA C
7. LETRA B
8. LETRA B
9. LETRA C
10. LETRA E
11. LETRA C
12. LETRA E
13. LETRA E
14. LETRA A
15. LETRA C
16. LETRA D
17. LETRA B
18. LETRA B
19. LETRA E
20. LETRA B
21. LETRA B
22. LETRA C
23. LETRA C

FGV

1. LETRA A
2. LETRA B
3. LETRA C
4. LETRA C
5. LETRA D
6. LETRA E
7. LETRA C
8. LETRA D
9. LETRA C
10. LETRA C
11. LETRA D
12. LETRA B
13. LETRA A
14. LETRA E
15. LETRA A
16. LETRA C

BOOTSTRAP

Conceitos Básicos



O Bootstrap é o framework CSS gratuito mais famoso do mundo. Ele foi lançado em 2011 pela equipe de design da Twitter e rapidamente se tornou uma das ferramentas mais populares para desenvolvimento web. Devido a sua facilidade de uso, ampla gama de recursos e vasta comunidade de desenvolvedores, o Bootstrap é amplamente utilizado por designers e desenvolvedores para criar páginas web responsivas e aplicativos móveis. Além disso, é regularmente atualizado e melhorado, mantendo-se como um dos frameworks CSS mais populares e amplamente utilizados

atualmente.

Com ele é possível criar sites rápidos e responsivos. O bootstrap é um kit de ferramentas de front-end poderoso, extensível e repleto de recursos. É possível criar e personalizar com Sass, utilizar componentes e Grid system pré-criados além de dar vida aos projetos com plug-ins **JavaScript** poderosos.

Paolla, mas vamos por partes? o que é Sass? Sass (Syntactically Awesome Style Sheets - tradução livre: Folhas de estilo **sintaticamente impressionantes**, gostaram disso? Eu gostei! 😊) é uma linguagem de estilo para o desenvolvimento web que é baseada em CSS (Cascading Style Sheets). Sass **adiciona recursos ao CSS**, como variáveis, aninhamento de regras, herança, mixins, entre outros, tornando o processo de estilo mais poderoso, organizado e fácil de manter.

Ao utilizar Sass, os desenvolvedores podem escrever código CSS de forma **mais modular e organizada**, o que é especialmente **útil em projetos grandes e complexos**. Sass também **permite a automação de tarefas repetitivas**, como a geração de cores a partir de variáveis, o que ajuda a tornar o processo de desenvolvimento mais rápido e eficiente.

Em resumo, Sass é uma extensão para o CSS que permite aos desenvolvedores escrever código de estilo mais poderoso, organizado e eficiente, tornando o processo de desenvolvimento de sites e aplicativos web mais fácil e eficiente.

Uau! Incrível, não é? mas vamos voltar o foco para o **Bootstrap**? Bora 😁! O **Bootstrap** é altamente personalizável, permitindo que os desenvolvedores criem designs exclusivos a partir das ferramentas pré-construídas.

Para começar a usar o Bootstrap, podemos fazer o download do arquivo CSS ou adicioná-lo ao seu projeto usando um gerenciador de pacotes, como o npm. Em seguida, você pode adicionar o arquivo CSS ao seu HTML e começar a usar as classes fornecidas pelo Bootstrap para estilizar seus elementos.

```
<!DOCTYPE html>
<html>
  <head>
    <!-- Adicione o CSS do Bootstrap -->
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">
  </head>
  <body>
    <!-- Adicione o conteúdo HTML aqui -->

    <!-- Adicione o JavaScript do Bootstrap (opcional) -->
    <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></scr
ipt>
  </body>
</html>
```

Este é um exemplo básico de uma estrutura HTML que inclui o Bootstrap. Primeiro, você inclui o arquivo CSS do Bootstrap na seção "head" do seu documento HTML, usando a tag "link". Em seguida, você adiciona o conteúdo HTML que você deseja exibir na página. Finalmente, você pode incluir o arquivo JavaScript do Bootstrap (opcional) na seção "body" do seu documento HTML, usando a tag "script".

Uma vez que o CSS do Bootstrap está adicionado ao seu HTML, você pode começar a usar as classes fornecidas pelo Bootstrap para estilizar seus elementos. Por exemplo, você pode usar a classe container para criar um container responsivo que se ajusta a diferentes tamanhos de tela:

```
<div class="container">
  <!-- Adicione o conteúdo do container aqui -->
</div>
```

Existem muitas outras classes fornecidas pelo Bootstrap que você pode usar para estilizar seus elementos. Além disso, você pode personalizar ainda mais o Bootstrap usando Sass e criar seu próprio estilo a partir dos componentes pré-construídos.

Certo, e depois Paolla, o que devo fazer?

Layout

Breakpoints

Breakpoints ou Pontos de interrupção são **larguras personalizáveis** que determinam **como seu layout responsivo se comporta** em dispositivos ou tamanhos de viewport no Bootstrap.

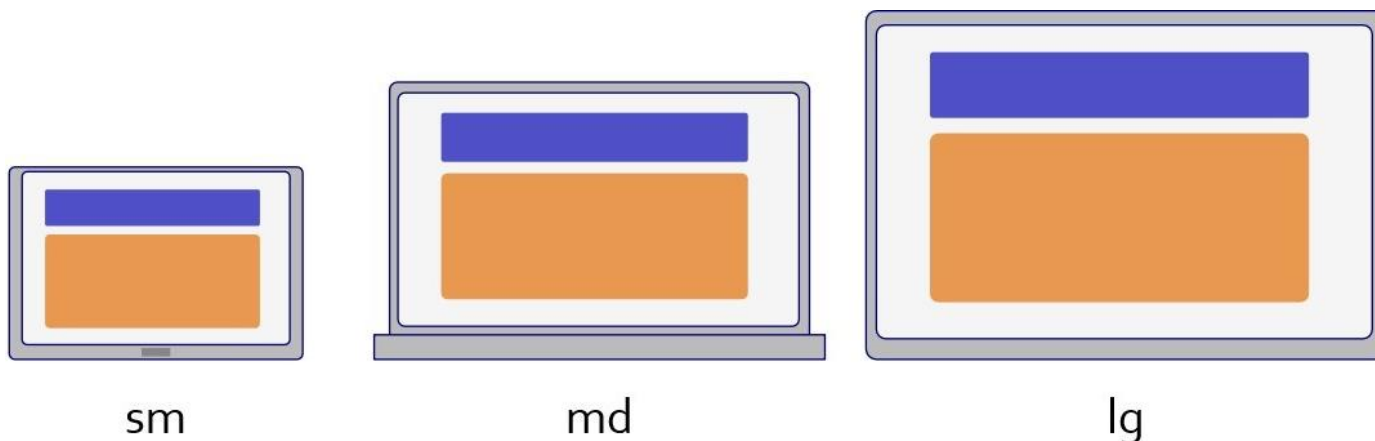
Os pontos de interrupção são os blocos de construção do design responsivo. É possível utilizá-los para controlar quando seu layout pode ser adaptado em uma determinada janela de visualização ou tamanho de dispositivo.

Use consultas de mídia para arquitetar seu CSS por ponto de interrupção. As **consultas de mídia** são um **recurso do CSS que permite aplicar estilos condicionalmente** com base em um conjunto de parâmetros do navegador e do sistema operacional. Usamos mais comumente min-width em nossas consultas de mídia.

Mobile first, design responsivo é o objetivo. O CSS do Bootstrap visa aplicar o mínimo de estilos **para fazer um layout funcionar no menor ponto de interrupção** e, em seguida, camadas em estilos para ajustar esse design para dispositivos maiores. Isso **otimiza seu CSS**, melhora o tempo de renderização e oferece uma ótima experiência para seus visitantes.

O Bootstrap inclui **seis pontos de interrupção padrão**, às vezes chamados de níveis de grade, para construção responsiva. Esses pontos de interrupção podem ser personalizados se você estiver usando arquivos Sass de origem.

Breakpoint	Infixo de classe	Dimensões
X-Small	Nenhum	<576px
Small	sm	≥576px
Medium	md	≥768px
Large	lg	≥992px
Extra large	xl	≥1200 px
Extra extra large	xxl	≥1400px



Cada ponto de interrupção foi escolhido para acomodar confortavelmente contêineres cujas larguras são **múltiplas de 12**. Os pontos de interrupção (**Breakpoints**) também representam um subconjunto de tamanhos de dispositivos comuns e dimensões de janela de visualização — eles não visam especificamente cada caso de uso ou dispositivo. Em vez disso, **os intervalos fornecem uma base forte e consistente para construir para praticamente qualquer dispositivo**. Interessante, não é? Bora ver mais um pouquinho!

Esses pontos de interrupção **são personalizáveis via Sass** - você os encontrará em um mapa Sass na folha de estilo `_variables.scss`.

Containers

Os contêineres são um bloco de construção fundamental do Bootstrap que contém, preenche e alinha seu conteúdo em um determinado dispositivo ou viewport.

Os contêineres são os elementos de layout **mais básico** no Bootstrap e são necessários ao usar o Grid system padrão. Os contêineres são usados para **conter, preencher e (às vezes) centralizar o conteúdo dentro deles**. Embora os contêineres possam ser aninhados, **a maioria dos layouts não requer um contêiner aninhado**. O uso de contêineres é uma parte importante do Bootstrap, pois eles ajudam a manter o layout do seu site organizado e consistente, independentemente do tamanho da tela do usuário.

O Bootstrap vem com três contêineres diferentes:

- **.container**, que define um max-width em cada breakpoint responsivo.
- **.container-fluid**, que está width: 100% em todos os breakpoints.
- **.container-{breakpoint}**, que é width: 100% até o breakpoint especificado.

Antes de adentrarmos a especificação dos containers, vejamos as opções de grade. O sistema de grade do Bootstrap pode se adaptar a todos os **seis pontos de interrupção padrão** e a **quaisquer pontos de interrupção personalizados**. As **seis camadas de grade** padrão são as seguintes:

- Extra small (xs)
- Small (sm)
- Medium (md)
- Large (lg)
- Extra large (xl)
- Extra extra large (xxl)

A tabela ilustra como cada contêiner max-width, ou seja, o tamanho máximo do contêiner, se compara ao `.container` original e ao `.container-fluid` em cada breakpoint.

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
<code>.container</code>	100%	540px	720px	960px	1140px	1320 px
<code>.container-sm</code>	100%	540px	720px	960px	1140px	1320 px
<code>.container-md</code>	100%	100%	720px	960px	1140px	1320 px
<code>.container-lg</code>	100%	100%	100%	960px	1140px	1320 px
<code>.container-xl</code>	100%	100%	100%	100%	1140px	1320 px
<code>.container-xxl</code>	100%	100%	100%	100%	100%	1320 px
<code>.container-fluid</code>	100%	100%	100%	100%	100%	100%

Ainda ficou difícil entender? Então vamos lá. A tabela de breakpoints do Bootstrap ajuda a compreender **como o layout do seu site será exibido em diferentes tamanhos de tela**. Ela ilustra a comparação entre o tamanho máximo de cada contêiner e o tamanho da tela em cada breakpoint.

O `.container` original, como mencionado anteriormente, define um **max-width** em cada breakpoint responsivo. Isso significa que, mesmo que a tela seja muito larga, o contêiner **nunca ultrapassará o tamanho máximo definido em cada breakpoint**. Esse tamanho máximo é calculado a partir de uma fórmula que considera a largura da tela, o espaçamento lateral e outros fatores.

Já o `.container-fluid`, como mencionado, **está sempre com width: 100%**, ou seja, ele sempre ocupa a largura total da tela, independentemente do breakpoint. Por outro lado, o `.container-{breakpoint}` é usado para definir um tamanho máximo em um breakpoint específico. Por exemplo, o `.container-md` faria com que o contêiner tenha largura máxima de **992px** em tela com largura entre **768px e 992px**.

A tabela de breakpoints do Bootstrap permite que você visualize como cada contêiner se compara aos demais em relação ao tamanho máximo em cada breakpoint, o que pode ser útil para planejar e ajustar o layout do seu site de maneira mais eficiente.

Vamos adiante! Além do contêiner padrão, temos os **Contêineres responsivos**. Os contêineres responsivos permitem que você especifique uma classe com 100% de largura até que o ponto de interrupção especificado seja atingido, após o qual aplicamos max-widths para cada um dos pontos de interrupção mais altos. Por exemplo, **.container-sm** é **100% de largura** para iniciar até que o ponto de interrupção sm seja atingido, onde será dimensionado com **md, lg, xle, xxl**.

Por outro lado, podemos utilizar Fluid containers para um **contêiner de largura total, abrangendo toda a largura da viewport**.

```
<div class="container-fluid">  
  ...  
</div>
```

Sass

Conforme mostrado acima, o Bootstrap gera uma série de classes de contêiner predefinidas para ajudá-lo a criar os layouts desejados. Você pode personalizar essas classes de contêiner predefinidas modificando o mapa Sass (encontrado em `_variables.scss`) que as alimenta.

O arquivo `_variables.scss` contém as variáveis que alimentam as classes de contêiner e, ao modificá-las, você pode alterar o comportamento dos contêineres conforme as suas necessidades. Além disso, o Sass permite o uso de mixins e heranças de estilo, o que facilita a customização das classes de contêiner de forma mais precisa e eficiente. É importante lembrar que, ao modificar o mapa Sass, você precisa recompilar o CSS para que as alterações tenham efeito na sua página web.

Grid system

O **sistema grid Bootstrap** (grid system ou sistema de grade) usa vários containers, linhas e colunas para arranjar e alinhar conteúdo. Ele é feito com flexbox e é, totalmente, responsivo. O **sistema de grades** do Bootstrap utiliza um layout de **12 colunas**. Isso significa que você pode **dividir sua página em até 12 colunas de largura igual**, permitindo que você organize o conteúdo de sua página de forma **flexível e responsiva**. Cada coluna é representada por uma classe CSS, como **".col-lg-4"**, que indica que a coluna ocupará 4 das 12 colunas disponíveis na tela larga (lg). Isso permite que você crie layouts que se adaptem automaticamente a diferentes tamanhos de tela, oferecendo uma excelente experiência de usuário em todos os dispositivos.

(CEBRASPE Órgão: STM - 2018) O sistema de grades do Bootstrap baseia-se em um leiaute de 12 colunas.

Comentários: Exatamente, pessoal! o Grid System (sistema de grades) do bootstrap possui doze colunas! permitindo que você organize o conteúdo de sua página de forma **flexível e responsiva**. Cada coluna é representada por uma classe CSS, como **".col-lg-4"**, que indica que a coluna ocupará 4 das 12 colunas disponíveis na tela larga (lg). (Gabarito Correto)

As **classes de grade** são formatadas como **.col-{tamanho da tela}-{número de colunas}**, onde o tamanho da tela é um dos breakpoints responsivos (xs, sm, md, lg, xl) e o **número de colunas** representa o número de colunas que o elemento deve ocupar na grade.

Além disso, o **Bootstrap** oferece classes de grade para definir a largura de um elemento na grade, deslocamento de grade, posicionamento e alinhamento. Isso permite **criar layouts complexos sem a necessidade de escrever CSS adicional**. O Grid system do Bootstrap é **altamente personalizável** e é possível modificar as configurações do grid usando o mapa Sass para atender às suas necessidades.

Propriedades	Descrição
Grid system suporta seis breakpoints responsivos	Os breakpoints são baseados em min-width consultas de mídia, o que significa que afetam esse ponto de interrupção e todos os que estão acima dele (por exemplo, .col-sm-4 aplica-se a sm, md, lg, xl e xxl). Isso significa que você pode controlar o dimensionamento e o comportamento do contêiner e da coluna em cada ponto de interrupção.
Os contêineres centralizam e preenchem horizontalmente seu conteúdo	Use .container para uma largura de pixel responsiva, .container-fluid para width: 100% todas as viewports e dispositivos, ou um contêiner responsivo (por exemplo, .container-md) para uma combinação de larguras fluidas e de pixel.
Linhas são wrappers para colunas	Cada coluna tem uma horizontal padding (chamada de calha) para controlar o espaço entre elas. Isso padding é então neutralizado nas linhas com margens negativas para garantir que o conteúdo em suas colunas esteja visualmente alinhado no lado esquerdo. As linhas também oferecem suporte a classes modificadoras para aplicar uniformemente o dimensionamento de colunas e classes de medianiz para alterar o espaçamento de seu conteúdo.

As colunas são incrivelmente flexíveis	Existem 12 colunas de modelo disponíveis por linha, permitindo que você crie diferentes combinações de elementos que abrangem qualquer número de colunas. As classes de coluna indicam o número de colunas de modelo a serem expandidas (por exemplo, col-4abrange quatro). widths são definidos em porcentagens para que você sempre tenha o mesmo tamanho relativo.
Gutters são responsivas e personalizáveis	As classes de gutter estão disponíveis em todos os breakpoints, com os mesmos tamanhos dos espaçamentos de margem. É possível mudar os gutters horizontais com as classes .gx-, gutters verticais com .gy- ou todos os gutters com as classes .g-*. Também está disponível a classe .g-0 para remover os gutters completamente.
Variáveis Sass, mapas e mixins alimentam o Grid system	Se você não quiser usar as classes de grade pré-definidas, é possível criar suas próprias classes usando o código Sass que alimenta o Grid system. Além disso, também estão incluídas propriedades CSS personalizadas que permitem a você aproveitar as variáveis Sass, o que oferece ainda mais flexibilidade. Em outras palavras, você pode personalizar e adaptar o sistema de grade do Bootstrap de acordo com as suas necessidades específicas.

Vamos ver tudo sobre Gutters na seção específica, mas para fins didáticos, **Gutters** são espaços vazios ou **margens que existem ao redor de um elemento**, geralmente entre colunas em um sistema de grade. Eles ajudam a **separar visualmente** os elementos uns dos outros e fornecem espaço adicional para ajustes visuais. No contexto do Bootstrap, as **classes gutter** são usadas para **adicionar espaços horizontais e verticais entre colunas em um sistema de grade**. Estas classes podem ser personalizadas para ajustar o tamanho dos espaços entre colunas, ou removidas completamente se não forem necessárias.

[definir título]

Para grades que são as mesmas desde o menor dos dispositivos até o maior, use as classes **.col** e **.col-***. Especifique uma classe numerada quando precisar de uma coluna de tamanho específico; caso contrário, sintase à vontade para manter **.col**. Veja o exemplo retirado diretamente da documentação:

```
<div class="container">  
  <div class="row">  
    <div class="col">col</div>
```

```
<div class="col">col</div>
<div class="col">col</div>
<div class="col">col</div>
</div>
<div class="row">
  <div class="col-8">col-8</div>
  <div class="col-4">col-4</div>
</div>
</div>
```

Usando um único conjunto de classes **.col-sm-***, você pode **criar um sistema de grade básico** que começa **empilhado e se torna horizontal** no pequeno ponto de interrupção (sm).

```
<div class="container">
<div class="row">
  <div class="col-sm-8">col-sm-8</div>
  <div class="col-sm-4">col-sm-4</div>
</div>
<div class="row">
  <div class="col-sm">col-sm</div>
  <div class="col-sm">col-sm</div>
  <div class="col-sm">col-sm</div>
</div>
</div>
```

Para criar um layout de grade responsivo usando o Bootstrap deve-se adicionar uma classe "row" a um elemento div e, em seguida, adicionar classes "col-*" para criar as colunas.

```
<div class="container">
<div class="row">
  <div class="col-sm-4">col-sm-4</div>
  <!-- Adicione o conteúdo da primeira coluna aqui -->
</div>
<div class="col-sm-4">col-sm-4</div>
  <!-- Adicione o conteúdo da segunda coluna aqui -->
</div>
<div class="col-sm-4">col-sm-4</div>
  <!-- Adicione o conteúdo da terceira coluna aqui -->
</div>
</div>
```

Neste exemplo, criamos uma div com a classe "container", que fornece um contêiner centralizado para o nosso conteúdo. Em seguida, adicionamos uma div com a classe "row", que define uma linha de grade. Finalmente, adicionamos três colunas usando classes "col-sm-4", que indicam que cada coluna ocupará 4 das 12 colunas disponíveis na tela pequena (sm). Lembre-se de que, para

criar um layout de grade responsivo, você pode usar diferentes classes "col-*" para ajustar o número de colunas que cada coluna ocupa em diferentes tamanhos de tela.

Misturar e combinar

Não quer que suas colunas simplesmente se empilhem em algumas camadas de grade? Use uma combinação de classes diferentes para cada camada, conforme necessário. Veja o exemplo abaixo para ter uma ideia melhor de como tudo funciona.

```
<div class="container">
<!-- Empilhe as colunas no dispositivo móvel fazendo uma largura total e a outra meia largura -->
<div class="row">
<div class="col-md-8">.col-md-8</div>
<div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>

<!-- As colunas começam com 50% de largura em dispositivos móveis e aumentam até 33,3% de largura em computadores -->
<div class="row">
<div class="col-6 col-md-4">.col-6 .col-md-4</div>
<div class="col-6 col-md-4">.col-6 .col-md-4</div>
<div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>

<!-- As colunas têm sempre 50% de largura, em dispositivos móveis e computadores -->
<div class="row">
<div class="col-6">.col-6</div>
<div class="col-6">.col-6</div>
</div>
</div>
```

Vamos entender esse exemplo? Esse é um exemplo de código HTML que usa o Bootstrap para criar um sistema de grades responsivo. O elemento div com a classe "container" é usado para criar um contêiner centralizado que envolve o conteúdo da página. Isso garante que o conteúdo esteja centralizado em relação à largura da tela, independentemente do tamanho da tela.

Em seguida, há três exemplos de como criar grades usando classes Bootstrap. No primeiro exemplo, há uma grade com duas colunas. A primeira coluna ocupa 8 de 12 colunas na largura (ou seja, 2/3 da largura da tela) quando a tela for maior que o "breakpoint" definido pela classe "col-md-". A segunda coluna ocupa metade da largura da tela (6 de 12 colunas) quando a tela for menor que o "breakpoint" definido pela classe "col-md-".

No segundo exemplo, há uma grade com três colunas. Quando a tela for menor que o "breakpoint" definido pela classe "col-md-", as colunas ocuparão metade da largura da tela. Quando a tela for maior que o "breakpoint" definido pela classe "col-md-", as colunas ocuparão

1/3 da largura da tela. No terceiro exemplo, há uma grade com duas colunas que sempre ocupam metade da largura da tela, independentemente do tamanho da tela.

Colunas de linha

Use as classes **.row-cols-*** responsivas para definir rapidamente o número de colunas que melhor renderizam seu conteúdo e layout. Considerando que as classes **.col-*** normais se aplicam às colunas individuais (por exemplo, **.col-md-4**), as classes de colunas de linha são definidas no pai **.row** como padrão para colunas contidas. Com **.row-cols-auto** você pode **dar às colunas sua largura natural**.

Use essas classes de colunas de linha para criar rapidamente layouts de grade básicos ou para controlar seus layouts de cartão e substituir quando necessário no nível da coluna.

```
<div class="container">
  <div class="row row-cols-2">
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
    <div class="col">Column</div>
  </div>
</div>
```

Você também pode usar o mixin Sass que o acompanha, `row-cols()`:

```
.element {
  // Three columns to start
  @include row-cols(3);

  // Five columns from medium breakpoint up
  @include media-breakpoint-up(md) {
    @include row-cols(5);
  }
}
```

(FGV - Auditor do Estado (CGE SC) - 2023) A maneira correta de criar um layout de grid responsivo usando o Bootstrap é usar a classe

- a) "row" em um elemento div e adicionar classes "col-*" para criar as colunas.
- b) "grid" em um elemento div e adicionar classes "row-*" para criar as linhas.
- c) "responsive " em um elemento div e adicionar classes "col-*" para criar as colunas.
- d) "flex" em um elemento div e adicionar classes "grid-*" para criar as linhas.
- e) "container" em um elemento div e adicionar as classes "row" e "col-*" para criar o layout.

Comentários:

Pessoal, a maneira correta de criar um layout de grid responsivo usando o Bootstrap é usar a classe `.row` para criar uma linha de grade e, em seguida, adicionar classes de coluna para especificar a largura de cada coluna dentro da linha. As classes de coluna são representadas por `.col-{breakpoint}-{largura}`, onde o breakpoint é o tamanho da tela (por exemplo, sm, md, lg) e a largura é o número de colunas que a coluna ocupará dentro do grid (de 1 a 12). Por exemplo, `.col-sm-6` significa que a coluna terá largura de 6 de 12 colunas na tela de tamanho pequeno. Portanto, o correto é a letra a) "row" em um elemento div e adicionar classes "col-*" para criar as colunas. (Gabarito: Letra A)

Columns

As colunas se baseiam na arquitetura flexbox da grade, ou seja, Flexbox significa que temos opções para alterar colunas individuais e modificar grupos de colunas no nível da linha. Você escolhe como as colunas **crecem, diminuem ou mudam**.

Ao criar layouts de grade, todo o conteúdo vai em colunas. A hierarquia da grade do Bootstrap vai do contêiner para a linha, para a coluna e para o seu conteúdo. Em raras ocasiões, você pode combinar conteúdo e coluna, mas saiba que pode haver consequências não intencionais.

O Bootstrap inclui classes predefinidas para criar layouts rápidos e responsivos. Com seis pontos de interrupção e uma dúzia de colunas em cada camada de grade, temos dezenas de classes já construídas para você criar os layouts desejados. Isso pode ser desativado via Sass, se desejar.

Gutters

Os gutters são o espaçamento entre as colunas, usados para espaçar e alinhar de forma responsiva o conteúdo no sistema de grade do Bootstrap. Os gutters são as lacunas entre o conteúdo das colunas, criadas por um espaçamento horizontal. Definimos o padding à direita e à esquerda em cada coluna e usamos uma margem negativa para compensar isso no início e no final de cada linha para alinhar o conteúdo.

Os gutters começam com 1,5 rem (24 px) de largura. Isso nos permite corresponder nossa grade à escala de espaçadores de margem e padding. Eles podem ser ajustados de forma responsiva. Use classes específicas de ponto de quebra para modificar os gutters horizontais, verticais e todos.

Utilitários

Bordas

Os utilitários de borda (border utilities) no Bootstrap são um conjunto de classes CSS que podem ser aplicadas a elementos de página para adicionar bordas, remover bordas ou ajustar a aparência de bordas existentes. Essas classes podem ser usadas para adicionar detalhes visuais a elementos na página ou para melhorar o espaçamento e o alinhamento de elementos. As classes de utilidade de borda mais comuns são:

Propriedades	Descrição
<code>.border-top</code> , <code>.border-bottom</code> , <code>.border-start</code> , <code>.border-end</code>	Adiciona uma borda apenas em um dos lados do elemento;
<code>.border-2</code> , <code>.border-3</code> , <code>.border-4</code>	Adiciona uma borda mais grossa com a espessura especificada em pixels;
<code>.border-rounded</code>	Adiciona uma borda arredondada;
<code>.border-top-rounded</code> , <code>.border-bottom-rounded</code> , <code>.border-start-rounded</code> , <code>.border-end-rounded</code>	Adiciona uma borda arredondada apenas em um dos lados do elemento.

Essas classes de utilidade de borda podem ser combinadas com outras classes do Bootstrap, como classes de cor e espaçamento, para personalizar a aparência dos elementos na página.

Você deve ter percebido que, tanto `text-end` quanto `text-right` são classes que podem ser usadas para **alinhar texto à direita** em um elemento HTML. No entanto, a diferença entre elas está na **maneira como o alinhamento é tratado em diferentes contextos de layout**.

`text-end` é uma classe de alinhamento de texto do **Bootstrap 5** que alinha o texto à direita no contexto do **layout flex**. Ou seja, se o elemento pai que contém o texto estiver configurado como um contêiner flex (usando a classe `d-flex`), o texto será **alinhado à direita em relação ao eixo principal do contêiner flex**. Isso significa que, se o eixo principal estiver definido como `row`, o texto será alinhado à direita da coluna, e se o eixo principal estiver definido como `column`, o texto será alinhado à direita da linha.

text-right, por outro lado, é uma classe de alinhamento de texto que pode ser usada **em qualquer contexto de layout**, não apenas no contexto flex. Quando aplicada a um elemento de texto, essa classe alinha o texto à direita em relação à borda direita do elemento pai.

Clearfix

O utilitário Clearfix é uma classe do Bootstrap usada para resolver o problema de elementos flutuantes. Quando elementos flutuantes são usados na página, eles podem causar problemas de layout, como elementos que não são exibidos onde deveriam estar ou elementos que se sobrepõem a outros elementos. O utilitário Clearfix resolve esse problema permitindo que os elementos "limpem" (ou removam) a propriedade de flutuação.

A classe de utilitário Clearfix é definida como **.clearfix** e é aplicada a um elemento contêiner que contém elementos flutuantes. Ela adiciona um pseudo-elemento **::after** ao elemento contêiner e define suas propriedades de exibição como block e sua altura como 0. Isso cria um elemento vazio com uma altura definida, que **"limpa" as propriedades de flutuação dos elementos flutuantes** dentro do elemento contêiner.

Flex

O sistema de flexbox é um recurso do Bootstrap que permite criar layouts flexíveis e responsivos. Alguns dos utilitários Bootstrap relacionados ao flexbox são:

- **display**: define o comportamento de exibição do elemento. As opções são: flex, inline-flex, none.
- **flex**: define o tamanho, crescimento e encolhimento do item flexível. As opções são: flex-grow, flex-shrink, flex-basis.
- **justify-content**: alinha os itens no eixo principal (horizontal). As opções são: start, end, center, between, around.
- **align-items**: alinha os itens no eixo transversal (vertical). As opções são: start, end, center, baseline, stretch.
- **align-self**: alinha um item específico no eixo transversal. As opções são as mesmas do align-items.
- **flex-wrap**: define se os itens devem ser dispostos em uma ou várias linhas. As opções são: nowrap, wrap, wrap-reverse.
- **order**: define a ordem em que os itens devem ser exibidos.

Esses utilitários podem ser aplicados diretamente nos elementos HTML ou combinados com as classes de grid do Bootstrap para criar layouts mais complexos e personalizados.

Alinhamento de Texto

Para alinhamento à esquerda, direita e central, estão disponíveis classes responsivas que usam os mesmos breakpoints da largura da janela de visualização do sistema de grade.

```
<p class="text-left">Texto alinhado à esquerda em todos os tamanhos de viewport.</p>
<p class="text-center">Centralize o texto alinhado em todos os tamanhos de viewport.</p>
<p class="text-right">Texto alinhado à direita em todos os tamanhos de viewport.</p>

<p class="text-sm-left">Texto alinhado à esquerda em viewports de tamanho SM (pequeno) ou maior.</p>
<p class="text-md-left">Texto alinhado à esquerda em viewports de tamanho MD (médio) ou maior.</p>
<p class="text-lg-left">Texto alinhado à esquerda em viewports de tamanho LG (grande) ou maior.</p>
<p class="text-xl-left">Texto alinhado à esquerda em viewports de tamanho XL (extragrande) ou maior. </p>
```

Classes de alinhamento do Bootstrap 5

O Bootstrap 5 oferece uma variedade de classes de alinhamento que podem ser usadas para alinhar conteúdo na página. Aqui estão as principais classes de alinhamento no Bootstrap 5:

Propriedades	Descrição
text-start	Alinha o conteúdo à esquerda (início) do elemento pai. É a opção padrão de alinhamento de texto em um elemento html.
text-center	Alinha o conteúdo ao centro do elemento pai.
text-end	Alinha o conteúdo à direita (fim) do elemento pai, no contexto flex.
align-baseline	Alinha o conteúdo em relação à linha de base do elemento pai.
align-top	Alinha o conteúdo ao topo do elemento pai.
align-middle	Alinha o conteúdo no centro vertical do elemento pai.
align-bottom	Alinha o conteúdo na parte inferior do elemento pai.
align-text-top	Alinha o texto ao topo do elemento pai.
align-text-bottom	Alinha o texto na parte inferior do elemento pai.

Além dessas classes, há também classes de alinhamento específicas para layout flex, como justify-content-start, justify-content-center e justify-content-end, que alinham os elementos filhos horizontalmente no contêiner flex. As classes de alinhamento flex também incluem align-items-start, align-items-center e align-items-end, que alinham os elementos filhos verticalmente no contêiner flex. Todas essas classes de alinhamento podem ser usadas em conjunto com outras

classes do Bootstrap para criar layouts responsivos e visualmente agradáveis. Vejamos de forma objetiva algumas classes de alinhamento flex do Bootstrap

Propriedades	Descrição
<code>justify-content-start</code>	Alinha os elementos filhos do contêiner flex ao início (lado esquerdo) do contêiner. É a opção padrão de alinhamento.
<code>justify-content-center</code>	Centraliza os elementos filhos do contêiner flex horizontalmente.
<code>justify-content-end</code>	Alinha os elementos filhos do contêiner flex ao final (lado direito) do contêiner.
<code>align-items-start</code>	Alinha os elementos filhos do contêiner flex ao início (topo) do contêiner.
<code>align-items-center</code>	Centraliza os elementos filhos do contêiner flex verticalmente.
<code>align-items-end</code>	Usada para alinhar elementos filhos verticalmente na parte inferior do contêiner pai. É usada em contêineres flex e pode ser aplicada ao elemento pai ou aos elementos filhos dentro dele.
<code>align-self-stretch</code>	Usada para esticar um elemento filho dentro do contêiner pai para ocupar todo o espaço disponível na direção vertical. É útil quando se deseja que um elemento se estenda para o tamanho do contêiner, mesmo que o conteúdo interno não o justifique.

Essas classes podem ser usadas em conjunto com outras classes do Bootstrap para criar layouts responsivos e visualmente agradáveis. Elas são particularmente úteis para organizar elementos em colunas e linhas, criando um fluxo de conteúdo claro e fácil de seguir. Além disso, as classes de alinhamento flex permitem que o conteúdo se adapte a diferentes tamanhos de tela, tornando o layout da página mais flexível e acessível.

Classes de posicionamento

As classes de posicionamento do Bootstrap permitem posicionar elementos em relação a um determinado ponto na tela. Algumas das principais classes são:

Propriedades	Descrição
<code>position-static</code>	Posição padrão de um elemento e não modifica seu posicionamento.
<code>position-relative</code>	Posiciona um elemento em relação à sua posição normal.

position-absolute	Posiciona um elemento em relação ao elemento pai mais próximo que não tenha posição estática.
position-fixed	Posiciona um elemento em relação à janela do navegador, o que significa que ele permanece na mesma posição, mesmo que a página seja rolada.
position-sticky	Posiciona um elemento com base em sua posição na página, mas fica "grudado" quando atinge uma determinada posição de rolagem.
float-start	Posiciona um elemento na esquerda em relação ao fluxo de conteúdo normal.
float-end	Posiciona um elemento na direita em relação ao fluxo de conteúdo normal.
float-none	Define que o elemento não deve ser posicionado com base no fluxo de conteúdo normal.
float-right	É uma classe de posicionamento do Bootstrap que é usada para mover um elemento para a direita em relação ao contêiner pai. Ela pode ser usada em qualquer elemento que tenha largura definida, como imagens, botões e divs.
float-left	Posiciona um elemento na esquerda em relação ao fluxo de conteúdo normal.
clearfix	Classe de utilidade usada para limpar os floats que foram aplicados a outros elementos, a fim de evitar que os elementos subsequentes sejam posicionados incorretamente.

No entanto, é importante notar que, com a introdução do sistema de grade flexível (flexbox) no Bootstrap 5, o uso de classes de float é menos necessário para o layout de páginas responsivas. Em vez disso, o sistema de grade flexível oferece mais poder e flexibilidade na criação de layouts de página responsivos.

Classes de cor

O Bootstrap 5 fornece um conjunto de classes de cor que permitem definir rapidamente as cores dos elementos na página. Essas classes podem ser usadas para definir a cor do texto, de fundo, de borda, de gradiente e de foco de elementos HTML, senão, vejamos.

Propriedades	Descrição
--------------	-----------

<code>.text-primary</code>	Define a cor do texto para a cor primária do tema;
<code>.text-secondary</code>	Define a cor do texto para a cor secundária do tema;
<code>.text-success</code>	Define a cor do texto para a cor verde usada para indicar sucesso;
<code>.text-danger</code>	Define a cor do texto para a cor vermelha usada para indicar perigo ou erro;
<code>.text-warning</code>	Define a cor do texto para a cor amarela usada para indicar aviso ou precaução;
<code>.text-info</code>	Define a cor do texto para a cor azul clara usada para fornecer informações;
<code>.text-light</code>	Define a cor do texto para uma cor clara, geralmente usada em um fundo escuro;
<code>.text-dark</code>	Define a cor do texto para uma cor escura, geralmente usada em um fundo claro.
<code>text-muted</code>	É uma classe de cor do Bootstrap que é usada para alterar o tom do texto para um cinza mais suave. Geralmente é usada para adicionar um texto de descrição ou informação adicional que não é tão importante quanto o conteúdo principal da página.
<code>.text-white</code>	Define a cor do texto como branco.

Além das classes de cor do texto, o Bootstrap também oferece classes de cor de fundo, como:

Propriedades	Descrição
<code>.bg-primary</code>	Define a cor de fundo para a cor primária do tema;
<code>.bg-secondary</code>	Define a cor de fundo para a cor secundária do tema;
<code>.bg-success</code>	Define a cor de fundo para a cor verde usada para indicar sucesso;
<code>.bg-danger</code>	Define a cor de fundo para a cor vermelha usada para indicar perigo ou erro;
<code>.bg-warning</code>	Define a cor de fundo para a cor amarela usada para indicar aviso ou precaução;
<code>.bg-info</code>	Define a cor de fundo para a cor azul clara usada para fornecer informações;

.bg-light	Define a cor de fundo para uma cor clara, geralmente usada em um fundo escuro;
.bg-dark	Define a cor de fundo para uma cor escura, geralmente usada em um fundo claro.

Componentes

Formulários

Um formulário Bootstrap é criado usando classes CSS pré-definidas, que permitem ao desenvolvedor aplicar estilos e funcionalidades com facilidade. Por exemplo, é possível usar classes para definir campos de entrada, botões de envio e elementos de seleção de data, além de definir o layout geral do formulário.

Além disso, o Bootstrap Form é responsivo por padrão, o que significa que ele se ajusta automaticamente ao tamanho da tela em que está sendo exibido. Isso é especialmente importante em dispositivos móveis, onde os usuários esperam que a interface do aplicativo seja fácil de usar em qualquer tamanho de tela.

Outra vantagem do Bootstrap Form é que ele é **altamente personalizável**. Embora o framework forneça estilos padrão, é possível personalizar o aspecto visual do formulário para que ele se adapte à identidade visual de um site ou aplicativo. Isso pode ser feito usando classes CSS personalizadas ou criando um arquivo de estilo separado.

O Bootstrap possui várias classes para estilizar os controles de formulário em seus projetos. Algumas das **principais classes** de controle de formulário do Bootstrap incluem:

- **.form-control**: essa classe é aplicada a inputs, selects e textareas para estilizá-los com um estilo padrão do Bootstrap. A classe aplica bordas, sombras e tamanho padrão para esses elementos.
- **.form-check**: essa classe é aplicada a elementos de checkbox e radio buttons para estilizá-los com um estilo padrão do Bootstrap. A classe adiciona bordas e um ícone personalizado para esses elementos.
- **.form-group**: essa classe é aplicada a grupos de elementos de formulário para criar uma separação visual entre eles. A classe adiciona margens e espaços entre os elementos do grupo.
- **.form-text**: essa classe é aplicada a elementos de texto associados a campos de formulário para estilizá-los com um estilo padrão do Bootstrap. A classe adiciona uma fonte menor e uma cor de texto cinza claro para esses elementos.
- **.form-label**: essa classe é aplicada a elementos de rótulo (label) para estilizá-los com um estilo padrão do Bootstrap. A classe adiciona uma fonte em negrito e uma cor de texto preta para esses elementos.

- **.form-inline**: essa é uma classe Bootstrap utilizada para criar formulários horizontais. Com essa classe, os campos de entrada (inputs), selects e textareas são exibidos lado a lado, em linha, em vez de um em cima do outro. Isso pode ser útil para formulários com poucos campos ou quando se deseja economizar espaço na tela.

Essas são apenas algumas das classes disponíveis para estilizar controles de formulário do Bootstrap. Existem outras classes disponíveis que podem ser usadas para estilizar diferentes tipos de elementos de formulário, como botões de envio e elementos de seleção de data.

Caixas de Seleção (Checkboxes) e Radio Buttons

No Bootstrap, as caixas de seleção (checkboxes) e os radio buttons podem ser estilizados com a classe **.form-check**. Essa classe adiciona estilos ao elemento HTML para deixá-lo mais visualmente atraente e fácil de usar.

Para criar uma caixa de seleção, pode-se utilizar o elemento HTML `<input>` com o atributo `type="checkbox"` e a classe **.form-check-input**. Em seguida, deve-se criar um rótulo associado a essa caixa de seleção, utilizando o elemento HTML `<label>` com a classe **.form-check-label**. O atributo `for` desse rótulo deve apontar para o atributo `id` do elemento `<input>`. Exemplo:

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="exampleCheckbox">
  <label class="form-check-label" for="exampleCheckbox">
    Exemplo de caixa de seleção
  </label>
</div>
```

Para criar um radio button, pode-se utilizar o elemento HTML `<input>` com o atributo `type="radio"` e a classe **.form-check-input**. O atributo `name` desse elemento deve ser o mesmo para todos os radio buttons do grupo. Em seguida, deve-se criar um rótulo associado a esse radio button, utilizando o elemento HTML `<label>` com a classe **.form-check-label**. O atributo `for` desse rótulo deve apontar para o atributo `id` do elemento `<input>`. Exemplo:

```
<div class="form-check">
  <input class="form-check-input" type="radio" name="exampleRadio"
  id="exampleRadio1">
  <label class="form-check-label" for="exampleRadio1">
    Exemplo de radio button 1
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="radio" name="exampleRadio"
  id="exampleRadio2">
  <label class="form-check-label" for="exampleRadio2">
```

```
Exemplo de radio button 2
</label>
</div>
```

É importante lembrar que, para que a caixa de seleção ou o radio button sejam enviados com o formulário, é necessário que o atributo name esteja definido. Além disso, pode-se utilizar outras classes e propriedades CSS para personalizar ainda mais o estilo desses elementos.

.form-check e .form-check-inline

As classes **.form-check** e **.form-check-inline** são utilizadas para estilizar grupos de caixas de seleção (checkboxes) e radio buttons no Bootstrap. A classe **.form-check** é utilizada para criar um grupo de caixas de seleção ou radio buttons em bloco, onde cada opção é exibida em uma linha separada. A classe **.form-check** é aplicada ao contêiner que envolve as caixas de seleção ou radio buttons e o rótulo (label) associado a elas. Exemplo:

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="exampleCheckbox1">
  <label class="form-check-label" for="exampleCheckbox1">
    Exemplo de caixa de seleção 1
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="exampleCheckbox2">
  <label class="form-check-label" for="exampleCheckbox2">
    Exemplo de caixa de seleção 2
  </label>
</div>
```

A classe **.form-check-inline** é utilizada para criar um grupo de caixas de seleção ou radio buttons em linha, onde cada opção é exibida ao lado da outra. A classe **.form-check-inline** é aplicada ao contêiner que envolve as caixas de seleção ou radio buttons e o rótulo (label) associado a elas. Exemplo:

```
<div class="form-check-inline">
  <input class="form-check-input" type="radio" name="exampleRadio"
id="exampleRadio1">
  <label class="form-check-label" for="exampleRadio1">
    Exemplo de radio button 1
  </label>
</div>
<div class="form-check-inline">
  <input class="form-check-input" type="radio" name="exampleRadio"
id="exampleRadio2">
  <label class="form-check-label" for="exampleRadio2">
```

```
Exemplo de radio button 2  
</label>  
</div>
```

É importante lembrar que a classe `.form-check-input` deve ser aplicada ao elemento `<input>` e a classe `.form-check-label` deve ser aplicada ao elemento `<label>` para que a estilização do Bootstrap funcione corretamente. Além disso, pode-se utilizar outras classes e propriedades CSS para personalizar ainda mais o estilo desses elementos.

Outras classes form

Classe	Descrição
<code>form-check-input</code>	É utilizada para estilizar o elemento <code><input></code> que representa a caixa de seleção (checkbox) ou radio button em um grupo de formulário no Bootstrap. Essa classe é necessária para que a estilização de caixas de seleção e radio buttons funcione corretamente no Bootstrap. O elemento <code><input></code> é estilizado com uma aparência de caixa de seleção ou radio button, dependendo do valor do atributo <code>type</code> . Essa classe também adiciona algumas propriedades CSS para garantir que o elemento fique alinhado corretamente com o rótulo (label) associado a ele.
<code>form-group</code>	É utilizada para agrupar elementos relacionados em um formulário, como campos de entrada (inputs), rótulos (labels), botões (buttons), etc. Quando a classe <code>.form-group</code> é aplicada a um contêiner que envolve esses elementos, o Bootstrap adiciona margens (espaçamentos) e outras propriedades CSS para melhorar a legibilidade e o espaçamento dos elementos.
<code>form-row</code>	É utilizada para agrupar elementos de formulário em uma linha, em um layout de grade responsivo. Quando a classe <code>.form-row</code> é aplicada a um contêiner que envolve elementos de formulário, o Bootstrap adiciona propriedades CSS para que esses elementos sejam exibidos em uma grade, com colunas que se ajustam automaticamente de acordo com o tamanho da tela. Isso pode ser útil para criar layouts responsivos de formulários que se adaptem a diferentes tamanhos de tela, como em dispositivos móveis.

Em resumo, o **Bootstrap Form** é uma ferramenta poderosa e útil para qualquer desenvolvedor que precise criar formulários elegantes e responsivos em seus projetos web. Com uma ampla gama de recursos e opções de personalização, é possível criar formulários que sejam tanto visualmente atraentes quanto fáceis de usar.

Carousel

O **Carousel** é um componente do Bootstrap que permite **exibir uma sequência de imagens**, textos ou outros conteúdos de forma rotativa em uma página web. O componente é conhecido também como slider ou slideshow.

O Carousel é um componente interativo, que permite que o usuário avance ou retroceda a sequência de conteúdos exibidos. Além disso, ele é responsivo, o que significa que se adapta automaticamente a diferentes tamanhos de tela, desde dispositivos móveis até monitores de desktop.

Para utilizar o **Carousel** no Bootstrap, é necessário **criar uma estrutura HTML** com alguns elementos específicos, como por exemplo:

- Um elemento com a classe **.carousel**, que é o contêiner principal do Carousel.
- Um elemento com a classe **.carousel-item** para cada item da sequência de conteúdos. Cada item pode conter uma imagem, um texto ou outros elementos HTML.
- Botões para avançar e retroceder a sequência de conteúdos, que são representados pelos elementos `<a>` com as classes **.carousel-control-prev** e **.carousel-control-next**.
- Indicadores da sequência de conteúdos, que são representados pelos elementos `<li>` com a classe **.active** e a classe `data-target="#carouselExampleIndicators" data-slide-to="0"`. O valor `data-target` deve ser igual ao ID do elemento com a classe **.carousel**, e o valor `data-slide-to` deve ser um número que representa a posição do item na sequência.

Antes de partir para os exemplos, devemos saber que:

- Em navegadores nos quais a API de visibilidade de página é compatível, o carrossel evitará deslizamentos quando a página da Web não estiver visível para o usuário (como quando a guia do navegador estiver inativa, a janela do navegador estiver minimizada etc.).
- Esteja ciente de que os **carrosséis aninhados não são suportados** e os carrosséis geralmente **não são compatíveis com os padrões de acessibilidade**.
- Por fim, se você estiver criando nosso JavaScript a partir do código-fonte, ele requer **util.js**.

Exemplo básico de uso do Carousel no Bootstrap:

```
<div id="carouselExampleSlidesOnly" class="carousel slide" data-ride="carousel">
  <div class="carousel-inner">
    <div class="carousel-item active">
      
```

```
</div>
<div class="carousel-item">
  
</div>
<div class="carousel-item">
  
</div>
</div>
</div>
```

Veja que nesse exemplo cada `carousel-item` está dentro de um `div`. A classe `carousel-item` é uma classe do Bootstrap que é usada em conjunto com a classe `carousel-inner` para criar um carrossel de imagens. Vamos entender melhor esse exemplo?

```
<div id="carouselExampleSlidesOnly" class="carousel slide" data-ride="carousel">
```

Este elemento `div` envolve todo o carrossel de imagens e possui as classes `carousel` e `slide`, indicando que este é um carrossel de imagens que desliza. O `id` do elemento é `carouselExampleSlidesOnly`, que pode ser usado para se referir a ele por meio de JavaScript ou CSS. A opção `data-ride` define que o carrossel será iniciado automaticamente ao carregar a página.

```
<div class="carousel-inner">
```

Este elemento `div` envolve todos os itens do carrossel e possui a classe `carousel-inner`.

```
<div class="carousel-item active">
  
</div>
```

Este elemento `div` com a classe `carousel-item` é usado para definir cada imagem no carrossel. A primeira imagem é marcada com a classe `active` para indicar que é a imagem inicial a ser exibida no carrossel. A imagem em si é definida usando a tag `img`, com as classes `d-block` e `w-100` para garantir que a imagem ocupe todo o espaço disponível no carrossel.

```
<div class="carousel-item">
  
</div>
<div class="carousel-item">
  
</div>
```

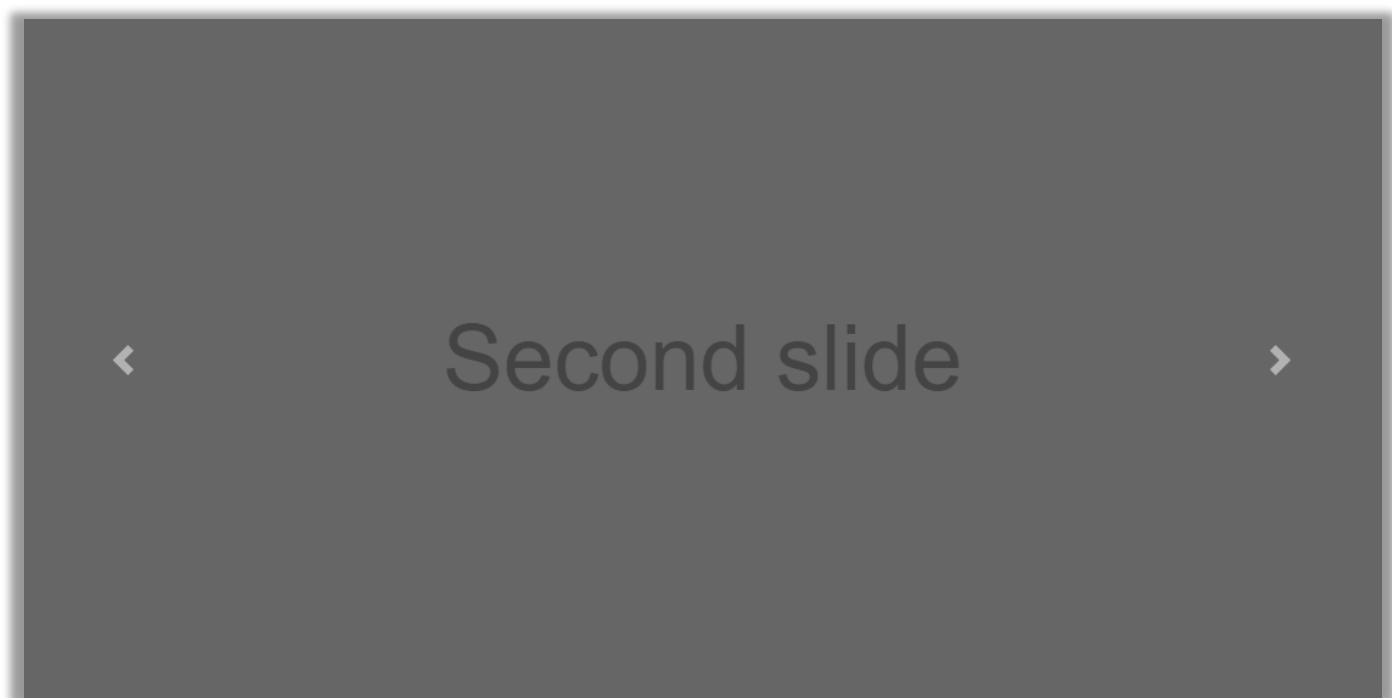
Esses elementos `div` são usados para definir as outras imagens no carrossel. Cada imagem é definida da mesma maneira que a primeira imagem, mas sem a classe `active`.

Em resumo, este código HTML cria um carrossel de imagens no Bootstrap, com três imagens que deslizam automaticamente. As imagens podem ser controladas manualmente usando os botões "próximo" e "anterior" ou clicando nas bolinhas de indicador abaixo do carrossel.

Agora vejamos um exemplo com três imagens, que são exibidas em sequência e podem ser navegadas com os botões de avançar (carousel-control-next) e retroceder(carousel-control-prev) ou com os indicadores de posição (carouselExampleIndicators).

```
<div id="carouselExampleIndicators" class="carousel slide" data-ride="carousel">
  <ol class="carousel-indicators">
    <li data-target="#carouselExampleIndicators" data-slide-to="0" class="active"></li>
    <li data-target="#carouselExampleIndicators" data-slide-to="1"></li>
    <li data-target="#carouselExampleIndicators" data-slide-to="2"></li>
  </ol>
  <div class="carousel-inner">
    <div class="carousel-item active">
      
    </div>
    <div class="carousel-item">
      
    </div>
    <div class="carousel-item">
      
    </div>
  </div>
  <a class="carousel-control-prev" href="#carouselExampleIndicators" role="button" data-slide="prev">
    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
    <span class="sr-only">Anterior</span>
  </a>
  <a class="carousel-control-next" href="#carouselExampleIndicators" role="button" data-slide="next">
    <span class="carousel-control-next-icon" aria-hidden="true"></span>
    <span class="sr-only">Próximo</span>
  </a>
</div>
```

A imagem abaixo apresenta um carrousel com controles



A imagem abaixo apresenta um carousel com indicadores



O Carousel do Bootstrap é composto por vários elementos que trabalham juntos para criar a funcionalidade desejada. Alguns desses elementos são:

Classe	Descrição
--------	-----------

<code>.carousel</code>	é a classe que define o contêiner principal do Carousel, que envolve todos os outros elementos do componente.
<code>.carousel-inner</code>	é a classe que envolve todos os <code>.carousel-item</code> e é responsável por exibir um único item do Carousel de cada vez.
<code>.carousel-item</code>	é a classe que define cada item individual do Carousel, que pode incluir uma imagem, um texto ou outros elementos HTML.
<code>.carousel-control-prev</code> e <code>.carousel-control-next</code>	são as classes usadas para criar os botões de controle de avanço e retrocesso do Carousel. Esses botões permitem que o usuário navegue para o próximo ou para o item anterior na sequência.
<code>.carousel-indicators</code>	é a classe usada para criar os indicadores de posição do Carousel, que permitem ao usuário saber em que item da sequência ele está.

Métodos	Descrição
<code>.carousel(options)</code>	Inicializa o carrossel com opções opcionais object e começa a percorrer os itens.
<code>.carousel('cycle')</code>	Percorre os itens do carrossel da esquerda para a direita.
<code>.carousel('pause')</code>	Impede que o carrossel percorra os itens.
<code>.carousel(number)</code>	Percorre o carrossel para um quadro específico (baseado em 0, semelhante a uma matriz). Retorna ao chamador antes que o item de destino seja exibido (ou seja, antes que o <code>slid.bs.carousel</code> evento ocorra).

<code>.carousel('prev')</code>	Volta para o item anterior. Retorna ao chamador antes que o item anterior tenha sido mostrado (isto é, antes que o <code>slid.bs.carousel</code> ocorra).
<code>.carousel('next')</code>	Passa para o próximo item. Retorna ao chamador antes que o próximo item seja mostrado (isto é, antes que o <code>slid.bs.carousel</code> ocorra).
<code>.carousel('dispose')</code>	Destrói o carrossel de um elemento, ou seja, remover a funcionalidade do carrossel de um elemento.

A classe carousel do Bootstrap expõe dois eventos (**slide** e **slid**) para conexão com a funcionalidade carrossel.

Eventos	Descrição
<code>slide.bs.carousel</code>	Este evento dispara imediatamente quando o método da instância do slide é invocado.
<code>slid.bs.carousel</code>	Este evento é acionado quando o carrossel concluiu sua transição de slide.

Ambos os eventos têm as seguintes propriedades adicionais:

Propriedades dos eventos	Descrição
<code>direction</code>	A direção na qual o carrossel está deslizando (ou "left" ou "right").
<code>relatedTarget</code>	O elemento DOM que está sendo deslizado para o lugar como o item ativo.
<code>from</code>	O índice do item atual
<code>to</code>	O índice do próximo item

Buttons



O Bootstrap oferece diversas classes para estilização de botões em páginas web. Algumas das classes mais comuns são:

Botões	Descrição
<code>btn</code>	É a classe base para estilizar botões no Bootstrap. Todos os botões devem ter esta classe para que a formatação do botão funcione adequadamente.
<code>btn-primary</code>	Define um estilo de botão padrão com uma cor azul que indica uma ação principal.
<code>btn btn-secondary</code>	Define um estilo de botão secundário, com uma cor mais clara em comparação ao botão padrão.
<code>btn btn-success</code>	Define um estilo de botão verde para indicar sucesso.
<code>btn btn-danger</code>	Define um estilo de botão vermelho para indicar perigo ou uma ação de exclusão.
<code>btn btn-warning</code>	Define um estilo de botão amarelo para indicar advertência ou ação que requer atenção do usuário.
<code>btn btn-info</code>	Define um estilo de botão azul claro para indicar informações ou ação que leva a informações adicionais.
<code>btn btn-light</code>	Define um estilo de botão com fundo branco e borda escura para um botão simples e limpo.
<code>btn btn-dark</code>	Define um estilo de botão com fundo escuro e texto claro para um botão com design mais forte e dramático.

As classes `.btn` são projetadas para serem usadas com o elemento `<button>`. No entanto, você também pode usar essas classes em elementos `<a>` ou `<input>` (embora alguns navegadores possam aplicar uma renderização ligeiramente diferente).

Ao usar classes de botão em `<a>` elementos que são usados para acionar a funcionalidade in-page (como recolher conteúdo), em vez de vincular a novas páginas ou seções na página atual, esses links devem receber um `role="button"` para transmitir adequadamente sua finalidade para tecnologias assistivas, como leitores de tela.

Button tags	Descrição
Input	O elemento <code><input></code> com <code>type="button"</code> é usado para criar um botão simples que executa uma ação definida pelo desenvolvedor.
Submit	O elemento <code><input></code> com <code>type="submit"</code> é usado para enviar um formulário para processamento.
Reset	É usado para limpar os campos de um formulário. Ele geralmente é usado em conjunto com o <code>type="submit"</code> , que envia os dados do formulário para processamento.

Spinners



Spinners são indicadores de carregamento que são usados para indicar ao usuário que um processo está em andamento e que ele deve

aguardar um pouco mais. Eles são frequentemente usados em sites e aplicativos da web para indicar que o conteúdo está sendo carregado ou que uma ação está em andamento.



O Bootstrap fornece alguns spinners pré-fabricados que podem ser facilmente adicionados a um projeto. Um dos spinners mais comuns é o "spinner-border", que mostra uma animação em forma de círculo que gira para indicar que o processo está em andamento.

O tamanho dos spinners pode ser ajustado usando as classes "spinner-border-sm" e "spinner-border-lg". Além disso, você pode usar várias outras classes para personalizar o estilo e o comportamento dos spinners, como "text-muted" para definir o texto como cinza claro quando o spinner está em uso.

Border spinner

`<span class="spinner-border spinner-border-sm"></span>` é uma tag HTML que representa um indicador de carregamento giratório (spinner) fornecido pelo Bootstrap. Essa classe `spinner-border` é aplicada a um elemento `<span>` e é usada para criar um indicador visual de que algum processo está em andamento, como um carregamento de página, upload de arquivo, ou qualquer outra atividade assíncrona que possa levar algum tempo.

A classe `spinner-border-sm` é usada para definir o tamanho do spinner. Nesse caso, `sm` significa "pequeno", que resultará em um spinner menor. Quando o elemento `<span>` é adicionado dentro de outro elemento, como um botão, o spinner aparece dentro do botão. Ao combinar o spinner com um texto indicativo, como "Carregando...", o usuário pode ter uma melhor compreensão do que está acontecendo e ter a expectativa de que a ação seja concluída em breve.

```
<div class="spinner-border" role="status">  
  <span class="visually-hidden">Loading...</span>  
</div>
```

Tooltips

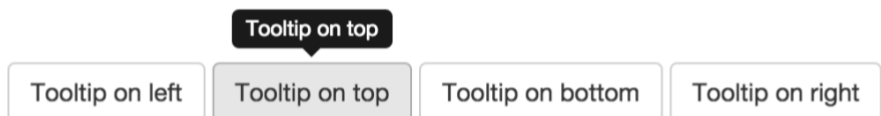
A classe `tooltip` é fornecida pelo Bootstrap e é usada em conjunto com o atributo `title` do elemento HTML para criar uma caixa de ferramentas que exibe informações adicionais quando o usuário passa o mouse sobre o elemento.

实例:



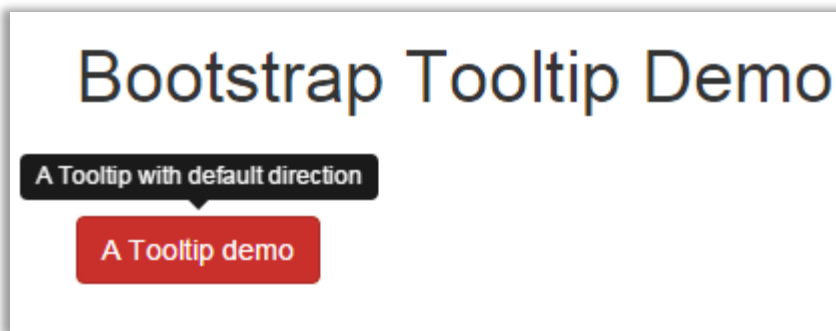
Four directions

实例:



```
<button type="button" class="btn btn-default" data-toggle="tooltip" data-placement="left"  
  title="Tooltip on left">Tooltip on left</button>
```

Tooltip é um recurso comum em interfaces de usuário de aplicativos da web e de desktop. É **uma caixa de ferramentas flutuante** que exibe **informações adicionais** ou contextuais quando o usuário passa o mouse sobre um elemento, sem precisar clicar ou navegar para outra página.



O Bootstrap é uma biblioteca popular de design responsivo para desenvolvimento web que fornece uma classe JavaScript tooltip para adicionar tooltips a elementos HTML. Os tooltips do Bootstrap podem ser usados para fornecer informações adicionais, dicas, erros, avisos ou mensagens de ajuda aos

usuários.

A sintaxe básica para criar um tooltip usando Bootstrap é adicionar a classe tooltip ao elemento HTML que deve mostrar o tooltip e definir o atributo title com o texto do tooltip (**data-toggle="tooltip"**). É possível personalizar a aparência, o comportamento e o posicionamento dos tooltips usando várias opções e classes do Bootstrap.

```
<button class="btn btn-primary" data-toggle="tooltip" data-placement="bottom" title="Clique aqui para enviar">Enviar</button>
```

- As dicas de ferramentas dependem da biblioteca de terceiros **Popper** para posicionamento. Você deve incluir **popper.min.js** antes de **bootstrap.js**, ou usar um **bootstrap.bundle.min.js** que contenha Popper. As dicas de ferramentas são opcionais por motivos de desempenho, portanto, você mesmo deve inicializá-las.
- As dicas de ferramentas **com títulos de comprimento zero nunca são exibidas**.
- Especifique container: 'body' para evitar problemas de renderização em componentes mais complexos (como nossos grupos de entrada, grupos de botões, etc).
- Acionar dicas de ferramentas em elementos ocultos não funcionará.
- As dicas de ferramentas .disabled ou disabled elementos devem ser acionadas em um elemento wrapper.
- Quando acionadas a partir de hiperlinks que abrangem várias linhas, as dicas de ferramentas serão centralizadas. Use white-space: nowrap; em seu <a>s para evitar esse comportamento.
- As dicas de ferramentas devem ser ocultadas antes que seus elementos correspondentes sejam removidos do DOM.
- As dicas de ferramenta podem ser acionadas graças a um elemento dentro de um shadow DOM.

Z-index

Vários componentes do Bootstrap utilizam z-index, a propriedade CSS que ajuda a controlar o layout fornecendo um terceiro eixo para organizar o conteúdo. Utilizamos uma escala de índice z padrão no Bootstrap que foi projetada para navegar adequadamente em camadas, dicas de ferramentas e popovers, modais e muito mais.

Esses valores mais altos começam em um número arbitrário, alto e específico o suficiente para evitar conflitos. Precisamos de um conjunto padrão deles em nossos componentes em camadas - dicas de ferramentas, popovers, navbars, menus suspensos, modais - para que possamos ser razoavelmente consistentes nos comportamentos. Não há razão para não termos usado 100+ ou 500+. Não encorajamos a personalização desses valores individuais; se você alterar um, provavelmente precisará alterar todos eles.

Z-index é uma propriedade CSS que determina a ordem de empilhamento de elementos em uma página web. Ela é usada para controlar a posição de um elemento em relação a outros elementos em um layout. Cada elemento em uma página tem uma "camada" e a propriedade z-index define a ordem de sobreposição dessas camadas. Um elemento com um z-index maior será exibido sobre um elemento com um z-index menor.

Por exemplo, se um elemento A tiver um z-index de 2 e outro elemento B tiver um z-index de 1, o elemento A será exibido na frente do elemento B, mesmo que o elemento B esteja acima do elemento A no código HTML.

O valor da propriedade z-index pode ser um número inteiro positivo ou negativo. Quanto maior o número positivo, maior a posição do elemento na pilha. Quando um elemento tem um valor negativo para z-index, ele é exibido atrás dos elementos com z-index positivo e do conteúdo padrão da página. A propriedade z-index é frequentemente usada em conjunto com outros recursos CSS, como position, para posicionar elementos na página e definir a ordem em que eles são exibidos.

Alinhamento de Texto

Para alinhamento à esquerda, direita e central, estão disponíveis classes responsivas que usam os mesmos pontos de interrupção da largura da janela de visualização do sistema de grade.

```
<p class="text-left">Texto alinhado à esquerda em todos os tamanhos de viewport.</p>
<p class="text-center">Centralize o texto alinhado em todos os tamanhos de viewport.</p>
<p class="text-right">Texto alinhado à direita em todos os tamanhos de viewport.</p>

<p class="text-sm-left">Texto alinhado à esquerda em viewports de tamanho SM (pequeno) ou maior.</p>
```

```
<p class="text-md-left">Texto alinhado à esquerda em viewports de tamanho MD (médio) ou maior.</p>
<p class="text-lg-left">Texto alinhado à esquerda em viewports de tamanho LG (grande) ou maior.</p>
<p class="text-xl-left">Texto alinhado à esquerda em viewports de tamanho XL (extragrande) ou maior. </p>
```

As classes de alinhamento do Bootstrap 5

O Bootstrap 5 oferece uma variedade de classes de alinhamento que podem ser usadas para alinhar conteúdo na página. Aqui estão as principais classes de alinhamento no Bootstrap 5:

- **text-start**: alinha o conteúdo à esquerda (início) do elemento pai. É a **opção padrão** de alinhamento de texto em um elemento HTML.
- **text-center**: alinha o conteúdo ao centro do elemento pai.
- **text-end**: alinha o conteúdo à direita (fim) do elemento pai, **no contexto flex**.
- **align-baseline**: alinha o conteúdo em relação à linha de base do elemento pai.
- **align-top**: alinha o conteúdo ao topo do elemento pai.
- **align-middle**: alinha o conteúdo no centro vertical do elemento pai.
- **align-bottom**: alinha o conteúdo na parte inferior do elemento pai.
- **align-text-top**: alinha o texto ao topo do elemento pai.
- **align-text-bottom**: alinha o texto na parte inferior do elemento pai.

Além dessas classes, há também classes de alinhamento específicas para layout flex, como justify-content-start, justify-content-center e justify-content-end, que alinham os elementos filhos horizontalmente no contêiner flex. As classes de alinhamento flex também incluem align-items-start, align-items-center e align-items-end, que alinham os elementos filhos verticalmente no contêiner flex. Todas essas classes de alinhamento podem ser usadas em conjunto com outras classes do Bootstrap para criar layouts responsivos e visualmente agradáveis. Vejamos de forma objetiva essas classes

Propriedades	Descrição
justify-content-start	Alinha o conteúdo ao início do container, ou seja, à esquerda.
justify-content-center	Centraliza o conteúdo horizontalmente no container.
justify-content-end	Alinha o conteúdo ao final do container, ou seja, à direita.
align-items-start	Alinha os itens verticalmente no início do container, ou seja, no topo.
align-items-center	Centraliza os itens verticalmente no container.
align-items-end	É uma classe de alinhamento do Bootstrap que é usada para alinhar elementos filhos verticalmente na parte inferior do contêiner pai. É

	usada em contêineres flex e pode ser aplicada ao elemento pai ou aos elementos filhos dentro dele.
align-self-stretch	É uma classe de alinhamento do Bootstrap que é usada para esticar um elemento filho dentro do contêiner pai para ocupar todo o espaço disponível na direção vertical. É útil quando se deseja que um elemento se estenda para o tamanho do contêiner, mesmo que o conteúdo interno não o justifique.

Classes de posicionamento

No Bootstrap, existem várias classes de posicionamento que podem ser usadas para controlar o layout dos elementos na página. Algumas das principais classes de posicionamento são:

Propriedades	Descrição
d-flex	Define um elemento como um flex container, permitindo que seus filhos sejam posicionados com flexbox.
flex-row	Define a direção do flex container como horizontal.
flex-column	Define a direção do flex container como vertical.
justify-content-*	Define como os itens dentro do flex container são alinhados ao longo do eixo principal. As opções disponíveis são
align-items-*	Define como os itens dentro do flex container são alinhados ao longo do eixo secundário. As opções disponíveis são
align-self-*	Define como um item específico dentro do flex container é alinhado ao longo do eixo secundário. As opções disponíveis são as mesmas que align-items-*.
order-*	Define a ordem dos itens dentro do flex container. Os itens são ordenados de acordo com o valor numérico da classe, do menor para o maior.
mx-auto	Centraliza um elemento horizontalmente em seu contêiner.
my-auto	Centraliza um elemento verticalmente em seu contêiner.

float-right	É uma classe de posicionamento do Bootstrap que é usada para mover um elemento para a direita em relação ao contêiner pai. Ela pode ser usada em qualquer elemento que tenha largura definida, como imagens, botões e divs.
position-static	Define que um elemento deve ter posicionamento estático, ou seja, não ter nenhum tipo de posicionamento especial.
position-relative	Define que um elemento deve ter posicionamento relativo em relação ao seu contêiner pai.
position-absolute	Define que um elemento deve ter posicionamento absoluto em relação ao seu contêiner pai ou em relação ao corpo da página.
position-fixed	Define que um elemento deve ter posicionamento fixo em relação à janela do navegador.
position-sticky	Define que um elemento deve ter posicionamento fixo até que seja rolado para fora da tela.
top-0, bottom-0, left-0, right-0	Define que um elemento deve ser posicionado no topo, na parte inferior, na parte esquerda ou na parte direita de seu contêiner pai ou do corpo da página, respectivamente.
mx-auto	Centraliza horizontalmente um elemento dentro de seu contêiner pai.
my-auto	Centraliza verticalmente um elemento dentro de seu contêiner pai.
text-center	Centraliza horizontalmente o texto dentro de um elemento, como um <p> ou um <div>.
text-end	Alinha o texto à direita dentro de um elemento, como um <p> ou um <div>.
text-start	alinha o texto à esquerda dentro de um elemento, como um <p> ou um <div>.

Classes de cor

No Bootstrap, há diversas classes para definir a cor dos elementos, dentre elas:

Propriedades	Descrição
<code>text-muted</code>	é uma classe de cor do Bootstrap que é usada para alterar o tom do texto para um cinza mais suave. Geralmente é usada para adicionar um texto de descrição ou informação adicional que não é tão importante quanto o conteúdo principal da página.
<code>text-primary</code>	Define a cor do texto como a cor primária do tema do Bootstrap;
<code>text-secondary</code>	Define a cor do texto como a cor secundária do tema do Bootstrap;
<code>text-success</code>	Define a cor do texto como verde para indicar sucesso;
<code>text-danger</code>	Define a cor do texto como vermelho para indicar perigo;
<code>text-warning</code>	Define a cor do texto como amarelo para indicar um aviso;
<code>text-info</code>	Define a cor do texto como azul claro para indicar uma informação;
<code>text-dark</code>	Define a cor do texto como uma cor escura;
<code>text-light</code>	Define a cor do texto como uma cor clara;
<code>bg-primary</code>	Define a cor de fundo como a cor primária do tema do Bootstrap;
<code>bg-secondary</code>	Define a cor de fundo como a cor secundária do tema do Bootstrap;
<code>bg-success</code>	Define a cor de fundo como verde para indicar sucesso;
<code>bg-danger</code>	Define a cor de fundo como vermelho para indicar perigo;
<code>bg-warning</code>	Define a cor de fundo como amarelo para indicar um aviso;
<code>bg-info</code>	Define a cor de fundo como azul claro para indicar uma informação;
<code>bg-dark</code>	Define a cor de fundo como uma cor escura;
<code>bg-light</code>	Define a cor de fundo como uma cor clara.

Você deve ter percebido que, tanto `text-end` quanto `text-right` são classes que podem ser usadas para **alinhar texto à direita** em um elemento HTML. No entanto, a diferença entre elas está na **maneira como o alinhamento é tratado em diferentes contextos de layout**.

`text-end` é uma classe de alinhamento de texto do **Bootstrap 5** que alinha o texto à direita no contexto do **layout flex**. Ou seja, se o elemento pai que contém o texto estiver configurado como um contêiner flex (usando a classe `d-flex`), o texto será **alinhado à direita em relação ao eixo principal do contêiner flex**. Isso significa que, se o eixo principal estiver definido como `row`, o texto será alinhado à direita da coluna, e se o eixo principal estiver definido como `column`, o texto será alinhado à direita da linha.

text-right, por outro lado, é uma classe de alinhamento de texto que pode ser usada **em qualquer contexto de layout**, não apenas no contexto flex. Quando aplicada a um elemento de texto, essa classe alinha o texto à direita em relação à borda direita do elemento pai.

(STM - 2018) Acerca de Bootstrap e Javascript, julgue o próximo item.

Utilizando-se o Bootstrap, a expressão a seguir insere o ícone Imagem associada para resolução da questão no layout.

```
<span class="glyphicon-search"></span>
```

Comentários: A expressão acima está incorreta, o correto seria: `<span class="glyphicon glyphicon-search"></span>` Lembrando que é necessário incluir a biblioteca de ícones Glyphicons no seu projeto, que é fornecida com o Bootstrap, para que esse ícone funcione corretamente. (**Gabarito:** Errado)

RESUMO

Bootstrap



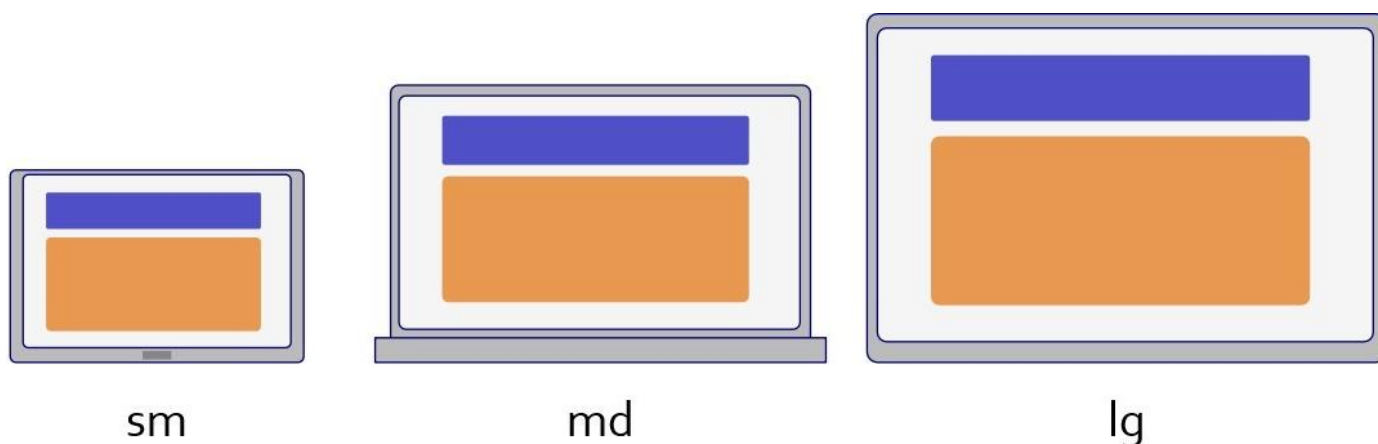
O Bootstrap é o framework CSS que foi lançado em 2011 pela equipe de design da Twitter e rapidamente se tornou uma das ferramentas mais populares para desenvolvimento web. Devido a sua facilidade de uso, ampla gama de recursos e vasta comunidade de desenvolvedores

O **Bootstrap** é um framework de front-end gratuito e de código aberto

Breakpoints

Breakpoints ou Pontos de interrupção são **larguras personalizáveis** que determinam **como seu layout responsivo se comporta** em dispositivos ou tamanhos de viewport no Bootstrap. O Bootstrap inclui **seis pontos de interrupção padrão**, às vezes chamados de níveis de grade, para construção responsiva. Esses pontos de interrupção podem ser personalizados se você estiver usando arquivos Sass de origem.

Breakpoint	Infixo de classe	Dimensões
X-Small	Nenhum	<576px
Small	sm	≥576px
Medium	md	≥768px
Large	lg	≥992px
Extra large	xl	≥1200 px
Extra extra large	xxl	≥1400px



Containers

Os contêineres são um bloco de construção fundamental do Bootstrap que contém, preenche e alinha seu conteúdo em um determinado dispositivo ou viewport. O Bootstrap vem com três contêineres diferentes:

- `.container`, que define um max-width em cada breakpoint responsivo.
- `.container-fluid`, que está width: 100% em todos os breakpoints.
- `.container-{breakpoint}`, que é width: 100% até o breakpoint especificado.

O sistema de grade do Bootstrap pode se adaptar a todos os **seis pontos de interrupção padrão** e a **quaisquer pontos de interrupção personalizados**. As **seis camadas de grade** padrão são as seguintes:

- Extra small (xs)
- Small (sm)
- Medium (md)
- Large (lg)
- Extra large (xl)
- Extra extra large (xxl)

A tabela ilustra como cada contêiner max-width, ou seja, o tamanho máximo do contêiner, se compara ao `.container` original e ao `.container-fluid` em cada breakpoint.

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
<code>.container</code>	100%	540px	720px	960px	1140px	1320 px
<code>.container-sm</code>	100%	540px	720px	960px	1140px	1320 px
<code>.container-md</code>	100%	100%	720px	960px	1140px	1320 px

.container-lg	100%	100%	100%	960px	1140px	1320 px
.container-xl	100%	100%	100%	100%	1140px	1320 px
.container-xxl	100%	100%	100%	100%	100%	1320 px
.container-fluid	100%	100%	100%	100%	100%	100%

Sass

O Bootstrap gera uma série de classes de contêiner predefinidas para ajudá-lo a criar os layouts desejados. Você pode personalizar essas classes de contêiner predefinidas modificando o mapa **Sass** (encontrado em `_variables.scss`) que as alimenta.

Grid system

O **sistema grid Bootstrap** (grid sysem ou sistema de grade) usa vários containers, linhas e colunas para arranjar e alinhar conteúdo. Ele é feito com flexbox e é, totalmente, responsivo. O **sistema de grades** do Bootstrap utiliza um layout de **12 colunas**. Isso significa que você pode **dividir sua página em até 12 colunas de largura igual**, permitindo que você organize o conteúdo de sua página de forma **flexível e responsiva**. Cada coluna é representada por uma classe CSS, como `".col-lg-4"`, que indica que a coluna ocupará 4 das 12 colunas disponíveis na tela larga (lg).

Propriedades	Descrição
Grid system suporta seis breakpoints responsivos	Os breakpoints são baseados em min-widthconsultas de mídia, o que significa que afetam esse ponto de interrupção e todos os que estão acima dele (por exemplo, <code>.col-sm-4</code> aplica-se a sm, md, lg, xle xxl). Isso significa que você pode controlar o dimensionamento e o comportamento do contêiner e da coluna em cada ponto de interrupção.
Os contêineres centralizam e preenchem horizontalmente seu conteúdo	Use <code>.container</code> para uma largura de pixel responsiva, <code>.container-fluid</code> para width: 100%todas as viewports e dispositivos, ou um contêiner responsivo (por exemplo, <code>.container-md</code>) para uma combinação de larguras fluidas e de pixel.
Linhas são wrappers para colunas	Cada coluna tem uma horizontal padding(chamada de calha) para controlar o espaço entre elas. Isso paddingé então neutralizado nas linhas com margens negativas para garantir que o conteúdo em suas

	colunas esteja visualmente alinhado no lado esquerdo. As linhas também oferecem suporte a classes modificadoras para aplicar uniformemente o dimensionamento de colunas e classes de medianiz para alterar o espaçamento de seu conteúdo.
As colunas são incrivelmente flexíveis	Existem 12 colunas de modelo disponíveis por linha, permitindo que você crie diferentes combinações de elementos que abrangem qualquer número de colunas. As classes de coluna indicam o número de colunas de modelo a serem expandidas (por exemplo, col-4abrange quatro). widths são definidos em porcentagens para que você sempre tenha o mesmo tamanho relativo.
Gutters são responsivas e personalizáveis	As classes de gutter estão disponíveis em todos os breakpoints, com os mesmos tamanhos dos espaçamentos de margem. É possível mudar os gutters horizontais com as classes .gx-, gutters verticais com .gy- ou todos os gutters com as classes .g-*. Também está disponível a classe .g-0 para remover os gutters completamente.
Variáveis Sass, mapas e mixins alimentam o Grid system	Se você não quiser usar as classes de grade pré-definidas, é possível criar suas próprias classes usando o código Sass que alimenta o Grid system. Além disso, também estão incluídas propriedades CSS personalizadas que permitem a você aproveitar as variáveis Sass, o que oferece ainda mais flexibilidade. Em outras palavras, você pode personalizar e adaptar o sistema de grade do Bootstrap de acordo com as suas necessidades específicas.

Bordas

Os utilitários de borda (border utilities) no Bootstrap são um conjunto de classes CSS que podem ser aplicadas a elementos de página para adicionar bordas, remover bordas ou ajustar a aparência de bordas existentes. Essas classes podem ser usadas para adicionar detalhes visuais a elementos na página ou para melhorar o espaçamento e o alinhamento de elementos. As classes de utilidade de borda mais comuns são:

Propriedades	Descrição
--------------	-----------

<code>.border-top, .border-bottom, .border-start, .border-end</code>	Adiciona uma borda apenas em um dos lados do elemento;
<code>.border-2, .border-3, .border-4</code>	Adiciona uma borda mais grossa com a espessura especificada em pixels;
<code>.border-rounded</code>	Adiciona uma borda arredondada;
<code>.border-top-rounded, .border-bottom-rounded, .border-start-rounded, .border-end-rounded</code>	Adiciona uma borda arredondada apenas em um dos lados do elemento.

Classes de alinhamento do Bootstrap 5

O Bootstrap 5 oferece uma variedade de classes de alinhamento que podem ser usadas para alinhar conteúdo na página. Aqui estão as principais classes de alinhamento no Bootstrap 5:

Propriedades	Descrição
<code>text-start</code>	Alinha o conteúdo à esquerda (início) do elemento pai. É a opção padrão de alinhamento de texto em um elemento html.
<code>text-center</code>	Alinha o conteúdo ao centro do elemento pai.
<code>text-end</code>	Alinha o conteúdo à direita (fim) do elemento pai, no contexto flex.
<code>align-baseline</code>	Alinha o conteúdo em relação à linha de base do elemento pai.
<code>align-top</code>	Alinha o conteúdo ao topo do elemento pai.
<code>align-middle</code>	Alinha o conteúdo no centro vertical do elemento pai.
<code>align-bottom</code>	Alinha o conteúdo na parte inferior do elemento pai.
<code>align-text-top</code>	Alinha o texto ao topo do elemento pai.
<code>align-text-bottom</code>	Alinha o texto na parte inferior do elemento pai.

Classes de posicionamento

As classes de posicionamento do Bootstrap permitem posicionar elementos em relação a um determinado ponto na tela. Algumas das principais classes são:

Propriedades	Descrição
<code>position-static</code>	Posição padrão de um elemento e não modifica seu posicionamento.
<code>position-relative</code>	Posiciona um elemento em relação à sua posição normal.
<code>position-absolute</code>	Posiciona um elemento em relação ao elemento pai mais próximo que não tenha posição estática.
<code>position-fixed</code>	Posiciona um elemento em relação à janela do navegador, o que significa que ele permanece na mesma posição, mesmo que a página seja rolada.
<code>position-sticky</code>	Posiciona um elemento com base em sua posição na página, mas fica "grudado" quando atinge uma determinada posição de rolagem.
<code>float-start</code>	Posiciona um elemento na esquerda em relação ao fluxo de conteúdo normal.
<code>float-end</code>	Posiciona um elemento na direita em relação ao fluxo de conteúdo normal.
<code>float-none</code>	Define que o elemento não deve ser posicionado com base no fluxo de conteúdo normal.
<code>float-right</code>	É uma classe de posicionamento do Bootstrap que é usada para mover um elemento para a direita em relação ao contêiner pai. Ela pode ser usada em qualquer elemento que tenha largura definida, como imagens, botões e divs.
<code>float-left</code>	Posiciona um elemento na esquerda em relação ao fluxo de conteúdo normal.
<code>clearfix</code>	Classe de utilidade usada para limpar os floats que foram aplicados a outros elementos, a fim de evitar que os elementos subsequentes sejam posicionados incorretamente.

Classes de cor

O Bootstrap 5 fornece um conjunto de classes de cor que permitem definir rapidamente as cores dos elementos na página. Essas classes podem ser usadas para definir a cor do texto, de fundo, de borda, de gradiente e de foco de elementos HTML, senão, vejamos.

Propriedades	Descrição
--------------	-----------

<code>.text-primary</code>	Define a cor do texto para a cor primária do tema;
<code>.text-secondary</code>	Define a cor do texto para a cor secundária do tema;
<code>.text-success</code>	Define a cor do texto para a cor verde usada para indicar sucesso;
<code>.text-danger</code>	Define a cor do texto para a cor vermelha usada para indicar perigo ou erro;
<code>.text-warning</code>	Define a cor do texto para a cor amarela usada para indicar aviso ou precaução;
<code>.text-info</code>	Define a cor do texto para a cor azul clara usada para fornecer informações;
<code>.text-light</code>	Define a cor do texto para uma cor clara, geralmente usada em um fundo escuro;
<code>.text-dark</code>	Define a cor do texto para uma cor escura, geralmente usada em um fundo claro.
<code>text-muted</code>	É uma classe de cor do Bootstrap que é usada para alterar o tom do texto para um cinza mais suave. Geralmente é usada para adicionar um texto de descrição ou informação adicional que não é tão importante quanto o conteúdo principal da página.
<code>.text-white</code>	Define a cor do texto como branco.

Além das classes de cor do texto, o Bootstrap também oferece classes de cor de fundo, como:

Propriedades	Descrição
<code>.bg-primary</code>	Define a cor de fundo para a cor primária do tema;
<code>.bg-secondary</code>	Define a cor de fundo para a cor secundária do tema;
<code>.bg-success</code>	Define a cor de fundo para a cor verde usada para indicar sucesso;
<code>.bg-danger</code>	Define a cor de fundo para a cor vermelha usada para indicar perigo ou erro;
<code>.bg-warning</code>	Define a cor de fundo para a cor amarela usada para indicar aviso ou precaução;
<code>.bg-info</code>	Define a cor de fundo para a cor azul clara usada para fornecer informações;

<code>.bg-light</code>	Define a cor de fundo para uma cor clara, geralmente usada em um fundo escuro;
<code>.bg-dark</code>	Define a cor de fundo para uma cor escura, geralmente usada em um fundo claro.

Propriedades	Descrição
<code>justify-content-start</code>	Alinha os elementos filhos do contêiner flex ao início (lado esquerdo) do contêiner. É a opção padrão de alinhamento.
<code>justify-content-center</code>	Centraliza os elementos filhos do contêiner flex horizontalmente.
<code>justify-content-end</code>	Alinha os elementos filhos do contêiner flex ao final (lado direito) do contêiner.
<code>align-items-start</code>	Alinha os elementos filhos do contêiner flex ao início (topo) do contêiner.
<code>align-items-center</code>	Centraliza os elementos filhos do contêiner flex verticalmente.
<code>align-items-end</code>	Usada para alinhar elementos filhos verticalmente na parte inferior do contêiner pai. É usada em contêineres flex e pode ser aplicada ao elemento pai ou aos elementos filhos dentro dele.
<code>align-self-stretch</code>	Usada para esticar um elemento filho dentro do contêiner pai para ocupar todo o espaço disponível na direção vertical. É útil quando se deseja que um elemento se estenda para o tamanho do contêiner, mesmo que o conteúdo interno não o justifique.

Carousel

O **Carousel** é um componente do Bootstrap que permite **exibir uma sequência de imagens**, textos ou outros conteúdos de forma rotativa em uma página web. O componente é conhecido também como slider ou slideshow. Exemplo básico de uso do Carousel no Bootstrap:

```
<div id="carouselExampleSlidesOnly" class="carousel slide" data-ride="carousel">
  <div class="carousel-inner">
    <div class="carousel-item active">
      
    </div>
    <div class="carousel-item">
      
    </div>
    <div class="carousel-item">
      
    </div>
  </div>
</div>
```

```
</div>  
</div>  
</div>
```

O Carousel do Bootstrap é composto por vários elementos que trabalham juntos para criar a funcionalidade desejada. Alguns desses elementos são:

Classe	Descrição
<code>.carousel</code>	é a classe que define o contêiner principal do Carousel, que envolve todos os outros elementos do componente.
<code>.carousel-inner</code>	é a classe que envolve todos os <code>.carousel-item</code> e é responsável por exibir um único item do Carousel de cada vez.
<code>.carousel-item</code>	é a classe que define cada item individual do Carousel, que pode incluir uma imagem, um texto ou outros elementos HTML.
<code>.carousel-control-prev</code> e <code>.carousel-control-next</code>	são as classes usadas para criar os botões de controle de avanço e retrocesso do Carousel. Esses botões permitem que o usuário navegue para o próximo ou para o item anterior na sequência.
<code>.carousel-indicators</code>	é a classe usada para criar os indicadores de posição do Carousel, que permitem ao usuário saber em que item da sequência ele está.

Métodos	Descrição
<code>.carousel(options)</code>	Inicializa o carrossel com opções opcionais <code>object</code> e começa a percorrer os itens.
<code>.carousel('cycle')</code>	Percorre os itens do carrossel da esquerda para a direita.
<code>.carousel('pause')</code>	Impede que o carrossel percorra os itens.

<code>.carousel(number)</code>	Percorre o carrossel para um quadro específico (baseado em 0, semelhante a uma matriz). Retorna ao chamador antes que o item de destino seja exibido (ou seja, antes que o <code>slid.bs.carousel</code> evento ocorra).
<code>.carousel('prev')</code>	Volta para o item anterior. Retorna ao chamador antes que o item anterior tenha sido mostrado (isto é, antes que o <code>slid.bs.carousel</code> evento ocorra).
<code>.carousel('next')</code>	Passa para o próximo item. Retorna ao chamador antes que o próximo item seja mostrado (isto é, antes que o <code>slid.bs.carousel</code> evento ocorra).
<code>.carousel('dispose')</code>	Destrói o carrossel de um elemento, ou seja, remover a funcionalidade do carrossel de um elemento.

A classe carousel do Bootstrap expõe dois eventos (**slide e slid**) para conexão com a funcionalidade carrossel.

Eventos	Descrição
<code>slide.bs.carousel</code>	Este evento dispara imediatamente quando o método da instância do slide é invocado.
<code>slid.bs.carousel</code>	Este evento é acionado quando o carrossel concluiu sua transição de slide.

Ambos os eventos têm as seguintes propriedades adicionais:

Propriedades dos eventos	Descrição
<code>direction</code>	A direção na qual o carrossel está deslizando (ou "left" ou "right").

relatedTarget	O elemento DOM que está sendo deslizado para o lugar como o item ativo.
from	O índice do item atual
to	O índice do próximo item

Spinners



Spinners são indicadores de carregamento que são usados para indicar ao usuário que um processo está em andamento e que ele deve

aguardar um pouco mais. Eles são frequentemente usados em sites e aplicativos da web para indicar que o conteúdo está sendo carregado ou que uma ação está em andamento.

Tooltips

A classe tooltip é fornecida pelo Bootstrap e é usada em conjunto com o atributo title do elemento HTML para criar uma caixa de ferramentas que exibe informações adicionais quando o usuário passa o mouse sobre o elemento.

实例：

Tooltip on the left

Tooltip on the top

Tooltip on the bottom

Tooltip on the right

Four directions

实例：

Tooltip on top

Tooltip on left

Tooltip on top

Tooltip on bottom

Tooltip on right

```
<button type="button" class="btn btn-default" data-toggle="tooltip" data-placement="left" title="Tooltip on left">Tooltip on left</button>
```

Z-index

Vários componentes do Bootstrap utilizam z-index, a propriedade CSS que ajuda a controlar o layout fornecendo um terceiro eixo para organizar o conteúdo. Utilizamos uma escala de índice z padrão no Bootstrap que foi projetada para navegar adequadamente em camadas, dicas de ferramentas e popovers, modais e muito mais.

REFERÊNCIAS

https://www.w3schools.com/bootstrap/bootstrap_ver.asp

QUESTÕES COMENTADAS

Diversas Bancas

1. (CESPE – STM– 2018) Acerca de Bootstrap e Javascript, julgue o item.

Utilizando-se o Bootstrap, a expressão a seguir insere o ícone  no leiaute. `<span class="glyphicon-search"></span>`

Comentários:

A expressão acima está incorreta, o correto seria: `<span class="glyphicon glyphicon-search"></span>` Lembrando que é necessário incluir a biblioteca de ícones Glyphicons no seu projeto, que é fornecida com o Bootstrap, para que esse ícone funcione corretamente

Gabarito: Errado

2. (CESPE –STM– 2018) Acerca de Bootstrap e Javascript, julgue o item.

O sistema de grades do Bootstrap baseia-se em um leiaute de 12 colunas.

Comentários:

Exatamente, pessoal! o Grid System (sistema de grades) do bootstrap possui doze colunas! permitindo que você organize o conteúdo de sua página de forma flexível e responsiva. Cada coluna é representada por uma classe CSS, como `".col-lg-4"`, que indica que a coluna ocupará 4 das 12 colunas disponíveis na tela larga (lg).

Gabarito: Certo

3. (CEPUERJ – UERJ – 2021) O Bootstrap prevê uma estilização padrão para formulários, bastando que se marque um div container com a classe form-group para cada conjunto de controles do formulário. No Bootstrap também é possível estilizar um formulário em linha, nesse caso, um valor de marcação válido seria:

- a) form-responsive
- b) form-control
- c) form-inline
- d) form-line

Comentários:

Para estilizar um formulário em linha, usamos o **form-inline** que é uma classe Bootstrap utilizada para criar formulários horizontais. Com essa classe, os campos de entrada (inputs), selects e textareas são exibidos lado a lado, em linha, em vez de um em cima do outro. Isso pode ser útil para formulários com poucos campos ou quando se deseja economizar espaço na tela

Gabarito: Letra C

4. (IFSP – IF SP– 2022) O carousel é recurso de slideshow da Biblioteca Bootstrap, com o objetivo de mostrar vários conteúdos, como se fosse um carrossel. Selecione a alternativa correta em relação a esse recurso.

- a) As classes carousel-inner, carousel-item e carousel-control são utilizadas com a tag "ol" do HTML, sem perda de funcionalidade.
- b) As classes carousel-inner, carousel-item e carousel-control são utilizadas com a tag "div" do HTML, sem perda de funcionalidade.
- c) As classes carousel-inner e carousel-item são utilizadas com a tag "a" do HTML, enquanto a classe carousel-control é utilizada com a tag "div" do HTML, sem perda de funcionalidade em todos esses casos.
- d) As classes carousel-inner e carousel-item são utilizadas com a tag "div" do HTML, enquanto a classe carousel-control é utilizada com a tag "a" do HTML, sem perda de funcionalidade em todos esses casos.

Comentários:

Vamos relembrar como é a funcionalidade das classes carousel? No exemplo apresentamos `<div class="carousel-inner">` e `<div class="carousel-item">`, portanto carousel-inner, carousel-item e são utilizadas com a tag "div". Ademais, carousel-control é utilizada com a tag "a", senão vejamos, botões para avançar e retroceder a sequência de conteúdos, que são representados pelos elementos `<a>` com as classes .carousel-control-prev e .carousel-control-next carousel-control.

Gabarito: Letra D

5. (IFSUL – IF SUL – 2019) O Bootstrap é uma estrutura popular de HTML, CSS e JavaScript para o desenvolvimento de websites responsivos e móveis.

São classes do Bootstrap para estilizar botões

- a) button-success, button-info.
- b) btn , btn-default.
- c) btn-warning, button-danger.
- d) button-link, btn-primary.

Comentários:

Pessoal, essa questão foi anulada pois a classe btn-default é uma classe descontinuada, ou seja, não mais utilizada. btn e btn-default são classes de estilo de botão no Bootstrap, mas a classe btn-default não é mais usada na versão mais recente do framework (Bootstrap 5). Na versão 4 do Bootstrap, btn-default era uma classe para definir um botão com estilo padrão, mas foi substituída por btn-secondary na versão 5. Portanto, a classe btn é a classe base para estilizar botões no Bootstrap e deve ser incluída em todos os botões. Se for necessário um estilo de botão padrão, deve-se usar btn-secondary em vez de btn-default.

Gabarito: Anulada

6. (FAPEC – UFMS– 2022) O _____ é uma coleção de ferramentas de código aberto para o desenvolvimento de sites e aplicativos web. Inclui modelos de design baseados em HTML e CSS, um sistema de grade responsivo, componentes predefinidos e plug-ins em jQuery. Desenvolvido para uniformizar os componentes do Twitter. Foi liberado como código aberto em 2011.

Assinale a alternativa que completa corretamente a lacuna.

- a) Bootstrap.
- b) Mark text.
- c) Brave Browser.
- d) IconGenerator.
- e) Screencat.

Comentários:

Bootstrap! Bootstrap! De fato, ele inclui modelos de design baseados em HTML e CSS, um sistema de grade responsivo, componentes predefinidos e plug-ins em jQuery. Desenvolvido para uniformizar os componentes do Twitter, lembrando que ele foi lançado em 2011 pela equipe de design da Twitter! Vejamos o que são as demais alternativas? b) Mark text: é um editor de texto com suporte a edição de Markdown e outros recursos, como realce de sintaxe e autocompletar. c) Brave Browser: é um navegador da web de código aberto e baseado no Chromium que prioriza a privacidade e segurança do usuário, incluindo bloqueio de anúncios e rastreadores. d) IconGenerator: é uma ferramenta online que permite criar ícones personalizados para aplicativos móveis ou web.

Gabarito: Letra A

7. (FCC –PGE AM– 2022) Considere o fragmento de código abaixo, disponível em uma página web criada com Bootstrap 5, em condições ideais.

```
<div class="container-fluid">
```

```
<div class="row">  
  <div class=" I bg-primary">  
    Amazonas  
  </div>  
  <div class=" II bg-secondary">  
    Brasil  
  </div>  
</div>
```

Para que os dois contêineres internos sejam renderizados com divisão de 50% cada, em todas as telas, exceto nas extra pequenas, as lacunas I e II devem ser corretamente preenchidas com

- a) col-left-50 e col-right-50
- b) col-half-screen e col-half-screen
- c) col-sm-6 e col-sm-6
- d) col-lg-50 e col-lg-50
- e) col-sm-5 e col-sm-5

Comentários:

Para que os dois contêineres internos sejam renderizados com divisão de 50% cada, em todas as telas, exceto nas extra pequenas, é possível usar as classes col-sm-6 e col-sm-6. A col-sm-6 define que a coluna terá 50% de largura em telas pequenas ou maiores (a partir de 576 pixels de largura). A classe bg-primary e bg-secondary é usada para definir as cores de fundo dos contêineres, mas não interfere na largura.

Gabarito: Letra C

8. (FCC – TRT4– 2022) Para definir uma tag div como contêiner, o Bootstrap tem alguns nomes de classe predefinidos, como

- a) container-flex e container-grid.
- b) container-box e container-text.
- c) container e container-fluid.
- d) container-stretch e container-fixed.
- e) container-relative e container-absolute.

Comentários:

O Bootstrap possui algumas classes predefinidas para definir uma tag div como contêiner, sendo as principais: container: classe para definir um contêiner com largura fixa e centralizado na página. container-fluid: classe para definir um contêiner com largura total e preenchimento completo do

espaço disponível. As outras opções mencionadas (container-flex, container-grid, container-box, container-text, container-stretch, container-fixed, container-relative e container-absolute) não são nomes de classe predefinidos pelo Bootstrap para contêineres. Algumas dessas classes podem ser usadas para outros fins, mas não para criar contêineres.

Gabarito: Letra C

9. (FCC – TRF 22 – 2022) Uma das classes do Bootstrap 5 para definir estilos de botões criados por meio da tag <button> é a classe,

- a) btn-reset.
- b) btn-success.
- c) button-submit.
- d) btn-post.
- e) button-reset.

Comentários:

Mais uma questão sobre botões em bootstrap! A classe do Bootstrap 5 para definir estilos de botões criados por meio da tag button é a classe btn. A classe btn-success pode ser usada para definir estilos de botões. Vamos lembrar as principais? btn, btn-primary, btn-secondary, btn-success, btn-danger, btn-warning, btn-info, btn-light, btn-dark.

Gabarito: Letra B

10.(FCC – TRT 23 – 2022) No Bootstrap 5, para criar um contêiner flexbox e transformar filhos diretos em itens flex, utiliza-se, nesse contêiner, a classe

- a) d-flex
- b) flex-display
- c) flex-layout
- d) flex-model
- e) m-flex

Comentários:

No Bootstrap 5, para criar um contêiner flexbox e transformar filhos diretos em itens flex, é necessário adicionar a classe d-flex ao contêiner. A classe d-flex é uma das classes utilitárias do Bootstrap 5 para lidar com o sistema de flexbox. Ela define o contêiner como um contêiner flexível e transforma todos os seus filhos diretos em itens flexíveis. Com isso, os itens dentro do contêiner podem ser facilmente posicionados e organizados usando as outras classes utilitárias do sistema de flexbox do Bootstrap 5.

11.(FCC – TRT 5– 2022) Considere o contêiner abaixo, criado em uma página HTML com Bootstrap 5, em condições ideais.

```
<div class="..!...">  
  <h1>TRIBUNAL REGIONAL DO TRABALHO DA 5 REGIÃO</h1>  
  <h2>SALVADOR-BAHIA</h2>  
</div>
```

Para definir que este contêiner será responsivo, de largura fixa e aparecerá centralizado na tela, utiliza-se, na lacuna I, a classe

- a) align-center
- b) flex-box
- c) container
- d) fixed-box
- e) container-fluid

Comentários:

Para tornar o contêiner responsivo, de largura fixa e centralizado na tela, deve-se utilizar as classes container e text-center do Bootstrap 5. A classe container define o contêiner com uma largura responsiva, de acordo com o tamanho da tela, enquanto a classe text-center centraliza o conteúdo horizontalmente dentro do contêiner. <div class="container">. Portanto nosso gabarito é a Letra C. Entretanto o ideal seria class="container text-center"

12.(FCC – TRT 14 – 2022) Considere os contêineres abaixo, criados utilizando HTML e Bootstrap 5, em condições ideais.

```
<div class="...I...">Conteúdo 1</div>
```

```
<div class="...II... ">Conteúdo 2</div>
```

Para definir uma divisão de 25% e 75% em dispositivos pequenos, com largura de tela entre 576 pixels e 767 pixels, as lacunas I e II devem ser preenchidas, respectivamente, com

- a) col-small-25 e col-small-75.

- b) col-sm-3 e col-sm-9.
- c) col-25 e col-75.
- d) col-md-3 e col-md-9.
- e) col-lg-3 e col-lg-9.

Comentários:

Para definir uma divisão de 25% e 75% em dispositivos pequenos (entre 576 e 767 pixels de largura de tela) utilizando Bootstrap 5, pode-se utilizar as classes col-sm-*, em que o * é substituído pelo número de colunas a serem ocupadas. Para que o primeiro contêiner ocupe 25% da largura total e o segundo contêiner ocupe 75% da largura total, pode-se usar as classes col-sm-3 e col-sm-9, respectivamente. Dessa forma, em dispositivos pequenos (entre 576 e 767 pixels de largura de tela), o primeiro contêiner ocupará 25% da largura total e o segundo contêiner ocupará 75% da largura total. Em dispositivos maiores, os contêineres ocuparão a largura total da tela, a menos que outras classes sejam adicionadas para alterar o comportamento em outras resoluções de tela.

Gabarito: Letra B

13.(FCC – TJ SC – 2021) Considere o trecho de uma página web abaixo, que utiliza Bootstrap.

```
<div class="container-fluid">  
  <div class="co">  
    <div class="..I.. bg-success">  
      TRIBUNAL DE JUSTIÇA DO ESTADO DE SANTA CATARINA  
    </div>  
    <div class="... II... bg-warning">  
      PODER JUDICIÁRIO DO ESTADO DE SANTA CATARINA  
    </div>  
  </div>  
</div>
```

Para que, em dispositivos pequenos (largura de tela de 576 pixels a 767 pixels), a divisão da esquerda use 25% da largura da linha e a divisão da direita use 75%, as lacunas I e II devem ser preenchidas, correta e respectivamente, com os nomes de classe

- a) col-sm-3 e col-sm-9
- b) col-sm-25 e col-sm-75
- c) col-small-25% e col-small-75%
- d) col-small-3 e col-small-9
- e) column-small-25 e column-small-75

Comentários:

CONHEÇA A BANCA! Gente, olha como a FCC gosta desse tipo de questão! São muito parecidas as questões! Repare a questão FCC – TRT 14 – 2022. Tem que aprender o perfil da banca e FAZER MUITAS QUESTÕES! Para que a divisão da esquerda use 25% da largura da linha e a divisão da direita use 75% em dispositivos pequenos (de 576 a 767 pixels de largura de tela) utilizando Bootstrap, pode-se usar as classes col-sm-*, onde * é substituído pelo número de colunas a serem ocupadas. Portanto, a classe para a divisão da esquerda deve ser col-sm-3 e a classe para a divisão da direita deve ser col-sm-9. Além disso, é necessário utilizar a classe container-fluid para criar um contêiner com largura total responsiva. Dessa forma, em dispositivos pequenos, a divisão da esquerda ocupará 25% da largura total da linha e a divisão da direita ocupará 75% da largura total da linha. Em dispositivos maiores, os contêineres ocuparão a largura total da linha, a menos que outras classes sejam adicionadas para alterar o comportamento em outras resoluções de tela. Lembrem-se, o grid do bootstrap possui 12 colunas!

Gabarito: Letra A

14.(FCC –SANASA – 2019) Considere o fragmento de uma página web abaixo, desenvolvido com Bootstrap4, onde todas as bibliotecas necessárias foram importadas no cabeçalho da página.

```
<div class="container-fluid">
  <div class="row">
    <div class="col-sm-4" style="background-color:red;">Fornecimento de água</div>
    <div class="col-sm-8" style="background-color:blue;">Tratamento de esgoto</div>
  </div>
</div>
```

Quando a janela (tela) tiver

- a) menos de 576 pixels de largura, as colunas serão empilhadas automaticamente uma sobre a outra.
- b) mais de 460 pixels de altura, as colunas aparecerão automaticamente uma sobre a outra.
- c) mais de 380 pixels de largura, as colunas aparecerão automaticamente uma ao lado da outra.
- d) menos de 768 pixels de largura, as colunas aparecerão automaticamente uma sobre a outra.
- e) mais de 800 pixels de largura, as colunas serão automaticamente colocadas uma sobre a outra.

Comentários:

Pessoal, quando a janela (tela) possuir menos de 576 pixels de largura, as colunas serão empilhadas automaticamente uma sobre a outra em dispositivos móveis, pois a classe col-sm-* define o comportamento para dispositivos pequenos com largura de tela igual ou superior a 576 pixels.

Dessa forma, o código HTML irá apresentar as duas colunas uma ao lado da outra em dispositivos com largura de tela maior ou igual a 576 pixels e irá empilhar as colunas uma sobre a outra em

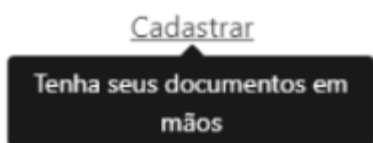
dispositivos com largura de tela menor que 576 pixels. Note que a classe container-fluid cria um contêiner com largura total responsiva, enquanto a classe row é usada para criar uma linha para as colunas. A classe col-sm-4 é usada para definir a largura da coluna de fornecimento de água em 4 colunas de largura para dispositivos com largura de tela de pelo menos 576 pixels, enquanto a classe col-sm-8 é usada para definir a largura da coluna de tratamento de esgoto em 8 colunas de largura para dispositivos com largura de tela de pelo menos 576 pixels. O estilo inline background-color define a cor de fundo de cada coluna para fins de demonstração.

Gabarito: Letra A

15.(FCC – Pref Manaus– 2019) Considere o fragmento de uma página web desenvolvida usando HTML, jQuery e Bootstrap 4. Considere que todas as bibliotecas necessárias foram referenciadas no cabeçalho da página.

```
<body>
<div class="container">
  <a href="#" data-toggle="a" data-placement="bottom" title="Tenha seus documentos em
mãos">Cadastrar</a>
</div>
<script>
$(document).ready(function() {
  $('[data-toggle="a"]')....I..();
});
</script>
</body>
```

Para que, ao levar o ponteiro do mouse sobre o link, apareça o que mostra a figura abaixo, a lacuna I deve ser preenchida por



- a) toggle.
- b) collapse.
- c) pophover.
- d) tooltip.
- e) toast.

Comentários:

A classe tooltip é fornecida pelo Bootstrap e é usada em conjunto com o atributo title do elemento HTML para criar uma caixa de ferramentas que exibe informações adicionais quando o usuário passa o mouse sobre o elemento.

No código da questão, o atributo `data-toggle="a"` indica que o elemento é um acionador de tooltip e o atributo `data-placement="bottom"` especifica que a caixa de ferramentas deve ser exibida abaixo do elemento. O código JavaScript dentro da tag `<script>` usa a função `$(document).ready()` para garantir que o código seja executado somente após a página ser carregada. A expressão `$('[data-toggle="a"]')...l..();` seleciona o elemento que possui o atributo `data-toggle="a"` e chama a função `tooltip()` do Bootstrap para inicializar o tooltip. A opção de posicionamento do tooltip também pode ser definida dentro da função `tooltip()` se o valor padrão especificado no atributo `data-placement` precisar ser substituído.

Gabarito: Letra D

16.(FCC – Pref Manaus– 2019) Considere o fragmento de código abaixo, em uma página que utiliza Bootstrap em condições ideais.

```
<div class="container-fluid">
  <div class="row">
    <div class="col-sm-3 col-md-6 col-lg-4 col-xl-2">
      <p>A</p>
    </div>
    <div class="col-sm-9 col-md-6 col-lg-8 col-xl-10">
      <p>B</p>
    </div>
  </div>
</div>
```

Esse fragmento resultará na divisão da largura da tela entre os dois contêineres na proporção de

- a) 20% / 80% em dispositivos de tela grande.
- b) 20% / 40% / 30% / 10% em todos os tipos de dispositivos.
- c) 60% / 40% em dispositivos de tela média.
- d) 25% / 75% em dispositivos de tela pequena.
- e) 33% / 66% em dispositivos de tela extragrande.

Comentários:

Pessoal, SEMPRE tenham em mente que o grid system possui 12 colunas! `col-sm-3` ($3/12$) = 25% e `col-sm-9` ($9/12$) = 75%, portanto, dispositivos pequenos (sm = small), (576px ou mais): o primeiro contêiner terá 3/12 da largura da linha (ou seja, 25%) e o segundo contêiner terá 9/12 da largura da linha (ou seja, 75%). Nosso gabarito é a letra D. Vejamos os demais dispositivos. Dispositivos médios (768px ou mais): o primeiro contêiner terá 6/12 da largura da linha (ou seja, 50%) e o segundo contêiner terá 6/12 da largura da linha (ou seja, 50%). Dispositivos grandes (992px ou

mais): o primeiro contêiner terá 4/12 da largura da linha (ou seja, 33,33%) e o segundo contêiner terá 8/12 da largura da linha (ou seja, 66,66%). Dispositivos extra grandes (1200px ou mais): o primeiro contêiner terá 2/12 da largura da linha (ou seja, 16,66%) e o segundo contêiner terá 10/12 da largura da linha (ou seja, 83,33%).

Gabarito: Letra D

17.(FCC – TJ MA – 2019) No desenvolvimento de sites muitas vezes é necessário criar botões como mostrados na figura abaixo, indicando algum procedimento de carregamento no site, em que a imagem de um pequeno círculo fica girando no interior do botão.



Usando Bootstrap 4, em condições ideais, este botão pode ser criado por meio do bloco de código abaixo.

```
<button class="btn btn-primary">  
  ..I..  
  Carregando..  
</button>
```

O código ficará correto se a indicação I for substituída pela instrução:

- a) `<spinner class="loading-circle"></spinner>`
- b) `<span class="spinner-circle"></span>`
- c) `<label class="shape-loading"></label>`
- d) `<span class="spinner-border spinner-border-sm"></span>`
- e) `<label class="spinner-loader"></label>`

Comentários:

Pessoal, vimos, quando tratamos sobre botões, que há spinners que realizam o famoso "loading" na página. No caso da questão, o gabarito é `<span class="spinner-border spinner-border-sm"></span>` que consiste em uma tag que representa um indicador de carregamento giratório (spinner) fornecido pelo Bootstrap. Essa classe `spinner-border` é aplicada a um elemento `<span>` e é usada para criar um indicador visual de que algum processo está em andamento, como um carregamento de página, upload de arquivo, ou qualquer outra atividade assíncrona que possa levar algum tempo.

Gabarito: Letra D

18.(FCC – TJ MA – 2019) O Bootstrap 4 usa flexbox em vez de float para manipular o layout da página web. Sendo assim, para criar um contêiner flexbox e transformar contêineres filhos diretos em itens flexíveis (flex), usa-se nesse contêiner a classe:

- a) flex-grow
- b) bg-flex
- c) d-flex
- d) flex-fill
- e) flex-stretch

Comentários:

Para criar um contêiner flexbox e transformar contêineres filhos diretos em itens flexíveis (flex) em Bootstrap 4, usa-se a classe "d-flex". A div com a classe "d-flex" se tornará um contêiner flexbox e seus filhos diretos (as divs com "Item 1", "Item 2" e "Item 3") se tornarão itens flexíveis dentro desse contêiner. Isso permitirá que eles sejam facilmente organizados em uma linha ou coluna, conforme especificado pelas outras classes do Bootstrap, como "flex-row" ou "flex-column".

Gabarito: Letra C

19.(FCC – Pref Manaus– 2019) Um programador criou a página web abaixo utilizando Bootstrap 4.

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css">
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.7/umd/popper.min.js"></script>
  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js"></script>
</head>
<body>
  <div class="container">
    <button type="button" class="btn btn-primary" ..... data-target="#a1">SEMEF</button>
    <div id="a1" class="collapse">
      A Secretaria Municipal de Finanças, Tecnologia da Informação e Controle Interno (SemeFi)
      integra a Administração Direta do Poder Executivo...
    </div>
  </div>
</body>
</html>
```

Para que, ao clicar uma vez no botão, o texto contido no contêiner com id="a1" apareça abaixo do botão e, ao clicar novamente o texto desapareça, alternando a cada clique, a lacuna I deverá ser preenchida com

- a) data-toggle="alternate"
- b) data-alternate="show-hide"
- c) data-toggle="collapse"

- d) data-alternate="true"
- e) data-toggle="show-hide"

Comentários:

Pessoal! Para que o botão possa alternar a exibição do conteúdo no contêiner, você precisa adicionar o atributo data-toggle="collapse" ao botão e definir o valor do atributo data-target para o ID do contêiner que deseja ocultar/exibir. Além disso, você precisa adicionar a classe "collapse" ao contêiner para torná-lo oculto por padrão. O atributo "data-toggle" é usado para especificar o tipo de ação que deve ser acionada no elemento ao qual ele é aplicado. No caso do Bootstrap, o valor "collapse" é usado para ocultar/exibir um elemento. Para alternar a exibição do conteúdo de um elemento ao clicar em um botão, você precisa usar o atributo "data-toggle" com o valor "collapse" e definir o valor do atributo "data-target" para o ID do elemento que deseja ocultar/exibir.

Gabarito: Letra C

20.(FCC –DPE AM– 2018) Considere o fragmento de código Bootstrap a seguir.

```
<div class="container-fluid">  
  <div class="row">  
    <div class="col-sm-3 col-md-6 col-lg-4" style="background-color:yellow;">Texto 1</div>  
    <div class="col-sm-9 col-md-6 col-lg-8" style="background-color:blue;">Texto 2</div>  
  </div>  
</div>
```

Quando a página que contém o fragmento de código acima for executada em condições ideais, o navegador vai renderizar os dois contêineres, um ao lado do outro, na proporção aproximada de

- a) 15%/85% para dispositivos de telas pequenas, 50%/50% para dispositivos de telas médias e 23%/76% para dispositivos de telas grandes.
- b) 25%/75% para dispositivos de telas pequenas, 50%/50% para dispositivos de telas médias e 33%/66% para dispositivos de telas grandes.
- c) 50%/50% para dispositivos de telas grandes, 25%/75% para dispositivos de telas médias e 30%/70% para dispositivos de telas pequenas.
- d) 25%/75% para dispositivos de telas grandes, 50%/50% para dispositivos de telas médias e 33%/66% para dispositivos de telas pequenas.
- e) 10%/90% para dispositivos de telas pequenas, 20%/80% para dispositivos de telas médias e 30%/70% para dispositivos de telas grandes.

Comentários:

O fragmento de código apresenta duas colunas, onde a primeira tem a classe `col-sm-3 col-md-6 col-lg-4` e a segunda tem a classe `col-sm-9 col-md-6 col-lg-8`. O Bootstrap 4 usa um sistema de grid responsivo baseado em 12 colunas, e as classes `"col-sm-3 col-md-6 col-lg-4"` e `"col-sm-9 col-md-6 col-lg-8"` especificam que o primeiro contêiner ocupa 3 colunas em telas pequenas, 6 colunas em telas médias e 4 colunas em telas grandes, enquanto o segundo contêiner ocupa 9 colunas em telas pequenas, 6 colunas em telas médias e 8 colunas em telas grandes. Isso resulta em proporções aproximadas de 25%/75% para telas pequenas, 50%/50% para telas médias e 33%/66% para telas grandes.

Gabarito: Letra B

21.(FCC –SABESP– 2018) Um Analista está utilizando Bootstrap 4 na criação de um site e deseja definir que um container deve usar toda a largura da tela.

Para isso, terá que utilizar na tag `<div>` o atributo `class` igual a

- a) `container`.
- b) `max-width`.
- c) `stretch`.
- d) `container-fluid`.
- e) `full-width`.

Comentários:

O Bootstrap 4 possui dois tipos de containers: `".container"` e `".container-fluid"`. A classe `".container"` é usada para criar um container com uma largura fixa que é responsiva, ou seja, se ajusta automaticamente para diferentes tamanhos de tela. Já a classe `".container-fluid"` é usada para criar um container que ocupa toda a largura da tela. Portanto, para definir que um container deve ocupar toda a largura da tela, o analista deve utilizar a classe `".container-fluid"`.

Gabarito: Letra D

22.(FCC –TST– 2017) Considere a página abaixo que utiliza Bootstrap em um ambiente de desenvolvimento web ideal.

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="bootstrap.min.css">
    <script src="jquery.min.js"></script>
    <script src="bootstrap.min.js"></script>
  </head>
```

```
<body>
  <button type="button" class="btn btn-info" data-toggle="..I.." data-target="#container">Mais
informações</button>
  <div id="container" class="collapse">
    <p>Aqui deverá estar o conteúdo desejado.</p>
  </div>
</body>
</html>
```

Para que, ao clicar no botão, o conteúdo do container seja exibido e, ao clicar novamente o conteúdo seja ocultado, alternando a cada clique, a lacuna I deverá ser preenchida com

- a) collapse
- b) toggle
- c) alternate
- d) show-hide
- e) modal-show

Comentários:

O atributo "data-toggle" é usado para especificar o tipo de ação que deve ser acionada no elemento ao qual ele é aplicado. No caso do Bootstrap, o valor "collapse" é usado para ocultar/exibir um elemento. Para alternar a exibição do conteúdo de um elemento ao clicar em um botão, você precisa usar o atributo "data-toggle" com o valor "collapse" e definir o valor do atributo "data-target" para o ID do elemento que deseja ocultar/exibir.

Gabarito: Letra A

23.(FCC –ARTESP– 2017) Um Especialista em Tecnologia da Informação deseja criar um layout com duas colunas utilizando Bootstrap, para dispositivos pequenos, com largura de tela de 768 pixels por 991 pixels. A coluna da esquerda deve ocupar aproximadamente 33,3% e a da direita 66,6% da tela. O Analista utilizará em uma página HTML as instruções abaixo.

```
<div class="...I...">....</div>
<div class="..I..I..">....</div>
```

Para obter o layout desejado, as lacunas I e II devem conter, respectivamente, os valores

- a) col-md-3 e col-md-9
- b) col-lg-33 e col-lg-66
- c) col-sm-4 e col-sm-8
- d) col-small-3 e col-small-7
- e) col-lg-3 e col-lg-9

Comentários:

Gente, é sério! Tem que saber isso! Questão muito recorrente na FCC! Para obter um layout com duas colunas, sendo a primeira com 4 colunas e a segunda com 8 colunas em telas pequenas, as lacunas I e II devem ser preenchidas com "col-sm-4" e "col-sm-8", respectivamente.

No Bootstrap, o sistema de grid é usado para criar layouts responsivos. As classes "col-{breakpoint}-{number}" são usadas para definir o número de colunas que um elemento deve ocupar em uma determinada largura de tela, onde "breakpoint" indica o tamanho da tela e "number" indica o número de colunas. Por exemplo, "col-sm-4" define que o elemento deve ocupar 4 colunas em telas pequenas (a partir de 576px).

Gabarito: Letra C

24.(FCC –TST – 2017) O plugin modal do bootstrap permite gerar uma caixa de diálogo sobre a página atual. O elemento <div> pai do modal deve ter como valor do atributo id o mesmo valor de um atributo do elemento usado para disparar o modal. Este atributo usado para disparar o modal é o

- a) data-toggle.
- b) data-target.
- c) modal-window.
- d) modal-trigger.
- e) modal-target.

Comentários:

O plugin modal do Bootstrap permite gerar uma caixa de diálogo sobre a página atual. Para disparar o modal, é necessário definir o atributo "data-target" em um elemento que acione o modal. O valor desse atributo deve ser o mesmo que o atributo "id" do elemento pai do modal. Dessa forma, quando o elemento que aciona o modal é clicado, o plugin do Bootstrap busca o elemento pai do modal com o mesmo "id" definido no "data-target" e o exibe na tela como uma caixa de diálogo.

Gabarito: Letra B

25.(FCC –TRF5 – 2017) Considere o código fonte da página abaixo, que faz parte de um site responsivo.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <title>TRF5</title>
    <meta charset="utf-8">
    <link rel="stylesheet" href="bootstrap.min.css">
    <script src="jquery.min.js"></script>
    <script src="bootstrap.min.js"></script>
  </head>
  <body>
    <div class="caixa">
      <ul class="...I...">
        <li><a href="#">Anterior</a></li>
        <li><a href="#">Próximo</a></li>
      </ul>
    </div>
  </body>
</html>
```

Para que este código crie dois botões para paginação, "Anterior" e "Próximo", um ao lado do outro, utilizando Bootstrap, a lacuna I deve ser corretamente preenchida com.

- a) top-pagination ou bottom-pagination.
- b) back-advance ou history-go.
- c) media-go ou media-pagination.
- d) pager ou pagination.
- e) modal ou collapse.

Comentários:

Para criar dois botões para paginação, "Anterior" e "Próximo", um ao lado do outro, utilizando Bootstrap, a lacuna I deve ser preenchida com pager ou pagination. <ul class="pagination"> é uma classe do framework Bootstrap que permite criar elementos de paginação em um site.

Ao utilizar a classe "pagination", cada item da lista será um botão de página, permitindo que o usuário navegue entre diferentes páginas de conteúdo. Esses botões podem ser personalizados para se adequarem ao design do site e são responsivos, ou seja, se adaptam a diferentes tamanhos de tela.

Gabarito: Letra D

26.(FEPESE – CELESC – 2022) O sistema de grid Bootstrap permite até quantas colunas?

- a) 16
- b) 12
- c) 9
- d) 8
- e) 6

Comentários:

AAAA! Existe questão assim SIM! 😁 É nesse momento que vocês correm logo para próxima questão felizes! Dissemos inúmeras vezes, o sistema de grid Bootstrap permite até 12 colunas! O sistema de grid Bootstrap é um dos principais recursos do framework, permitindo que o desenvolvedor crie layouts responsivos para dispositivos móveis e desktops. Ele divide a página em 12 colunas, permitindo que o desenvolvedor organize o conteúdo em diferentes combinações de colunas para diferentes tamanhos de tela. As classes de grid do Bootstrap são baseadas em três partes principais: o prefixo do tamanho da tela (col-sm, col-md, col-lg), o número de colunas que uma coluna ocupa (1-12) e uma opção de offset, que permite empurrar uma coluna para a direita em um determinado número de colunas.

Gabarito: Letra B

27.(FEPESE – CELESC – 2022) Qual classe Bootstrap 5 centraliza um texto em uma tag HTML <p>?

- a) align
- b) center
- c) middle
- d) justified
- e) text-center

Comentários:

A classe Bootstrap 5 para centralizar um texto em um parágrafo (tag HTML <p>) é text-center. Essa classe pode ser aplicada diretamente na tag <p> ou em um elemento pai que contém a tag <p>. Já asbemos nosso gabarito, letra e, agora vejamos os demais itens. a) align: é um atributo HTML que define o alinhamento horizontal de um elemento em relação ao seu contêiner pai. Esse atributo pode ter os valores left (esquerda), right (direita) ou center (centro). b) center: é uma propriedade CSS que centraliza horizontalmente um elemento em relação ao seu contêiner pai. c) middle: é uma propriedade CSS que centraliza verticalmente um elemento em relação ao seu contêiner pai. d) justified: é uma propriedade CSS que alinha o texto de forma que cada linha tenha a mesma largura, criando uma margem adicional entre as palavras.

28.(FEPESE – CELESC – 2022) Qual componente Bootstrap pode ser definido como a classe de um elemento <div>, por exemplo, para implementar um slideshow de modo a alternar entre elementos, sejam imagens ou slides de texto?

- a) list
- b) flip-flop
- c) carousel
- d) slideshow
- e) mosaic

Comentários:

O componente Bootstrap para implementar um slideshow é o Carousel. A classe a ser utilizada em um elemento <div> é carousel, e as imagens ou slides de texto são adicionados como elementos filhos com a classe carousel-item. É possível personalizar o comportamento do Carousel utilizando as classes de controle carousel-control-prev e carousel-control-next.

29.(FGV – CGE SC– 2023) A maneira correta de criar um layout de grid responsivo usando o Bootstrap é usar a classe

- a) "row" em um elemento div e adicionar classes "col-*" para criar as colunas.
- b) "grid" em um elemento div e adicionar classes "row-*" para criar as linhas.
- c) "responsive " em um elemento div e adicionar classes "col-*" para criar as colunas.
- d) "flex" em um elemento div e adicionar classes "grid-*" para criar as linhas.
- e) "container" em um elemento div e adicionar as classes "row" e "col-*" para criar o layout.

Comentários:

Pessoal, a maneira correta de criar um layout de grid responsivo usando o Bootstrap é usar a classe .row para criar uma linha de grade e, em seguida, adicionar classes de coluna para especificar a largura de cada coluna dentro da linha. As classes de coluna são representadas por .col-{breakpoint}-{largura}, onde o breakpoint é o tamanho da tela (por exemplo, sm, md, lg) e a largura é o número de colunas que a coluna ocupará dentro do grid (de 1 a 12). Por exemplo, .col-sm-6 significa que a coluna terá largura de 6 de 12 colunas na tela de tamanho pequeno. Portanto, o correto é a letra a) "row" em um elemento div e adicionar classes "col-*" para criar as colunas

30.(FGV –Sefaz AM – 2022) O Bootstrap é um framework web com código-fonte aberto amplamente utilizado em desenvolvimento de aplicações web.

Analise o código da página web a seguir.

```
1 <!doctype html>
2 <html>
3
4   <head>
5     <link href="/bootstrap.min.css"
6         rel="stylesheet">
7   </head>
8
9   <body>
10    <p>NOME</p>
11  </body>
12
13 </html>
```

Para alinhar o texto NOME à direita utilizando Bootstrap, versão 5, é suficiente utilizar na tag <p> o atributo class igual a

- a) text-end.
- b) text-muted.
- c) float-right.
- d) align-items-end.
- e) align-self-stretch.

Comentários:

Para alinhar o texto NOME à direita utilizando Bootstrap versão 5, é necessário utilizar na tag <p> o atributo class igual a "text-end". Portanto, a opção correta é a letra a). Vamos relembrar sobre a classe text-end? text-end é uma classe de alinhamento de texto do Bootstrap 5 que alinha o texto à direita no contexto do layout flex. Ou seja, se o elemento pai que contém o texto estiver configurado como um contêiner flex (usando a classe d-flex), o texto será alinhado à direita em relação ao eixo principal do contêiner flex. Isso significa que, se o eixo principal estiver definido como row, o texto será alinhado à direita da coluna, e se o eixo principal estiver definido como column, o texto será alinhado à direita da linha.

Gabarito: Letra A

31.(FGV –TRT16 – 2022) Observe a declaração abaixo no contexto de páginas Web.

```
<div class="jumbotron">
  <h1>Teste</h1>
  <p>Frase inicial...</p>
</div>
```

Assinale o framework usualmente associado à classe utilizada.

- a) Angular Material.
- b) Bootstrap.
- c) CSS.
- d) JSON.
- e) SpringBoot.

Comentários:

Pessoal, trata-se do Bootstrap. Entretanto, vejamos um pouco mais sobre `<div class="jumbotron">`! A classe "jumbotron" no Bootstrap é usada para criar um destaque visual em uma seção específica de uma página da web. É comumente usada para apresentar informações importantes, como uma introdução ou um call-to-action. A classe aplica um fundo de cor sólida e adiciona um grande espaçamento em torno do conteúdo, dando uma sensação de destaque e ênfase.

Gabarito: Letra B

32.(Instituto AOCP – PRODEB – 2018) É possível considerar o Bootstrap como um kit de desenvolvimento básico que é composto por uma gama de componentes prontos que auxiliam no desenvolvimento de aplicações web mobile de forma simples e clara, tirando a necessidade de um conhecimento profundo em JavaScript e CSS. O que é possível fazer com Bootstrap?

- a) Todo o desenvolvimento web mobile que não tem ligação com html.
- b) Desenho de telas para serem acessadas em um navegador web ou dispositivo mobile.
- c) Validação de processo de campos de entradas de dados.
- d) Toda tarefa que não seja relacionada com o desenho de uma tela.
- e) Calcular datas a partir de campos inputs.

Comentários:

É possível usar o Bootstrap para criar layouts responsivos, componentes de interface do usuário, formulários, tabelas, tipografia, botões, ícones, navegação e muitos outros elementos de interface para uma aplicação web. Além disso, o Bootstrap fornece recursos para melhorar a acessibilidade e a usabilidade em dispositivos móveis, bem como personalização e extensibilidade para atender às necessidades específicas de um projeto. O uso do Bootstrap pode acelerar o desenvolvimento de aplicativos web e garantir uma aparência consistente e profissional em diferentes dispositivos e plataformas.

Gabarito: Letra B

33.(Instituto AOCP – PRODEB – 2018) A tipografia é muito importante em qualquer trabalho gráfico, seja em um ambiente físico ou até mesmo virtual. Ela tem muita influência no peso da informação e na forma como os usuários perceberão as informações que o conteúdo pode e deve transmitir. Assinale a alternativa correta em relação ao uso de recursos de tipografia no Bootstrap.

- a) As classes do Bootstrap `mute`, `primary`, `success`, `info`, `warning` e `danger` são usadas para se trabalhar com o alinhamento do texto.
- b) Não é possível fazer a alteração do tipo da fonte no Bootstrap.
- c) É possível alinhar um texto utilizando as classes `left-text`, `right-text` e `center-text`.
- d) A tag de parágrafo `<p>` recebe uma margem inferior, além de uma classe especial chamada `lead` que destaca melhor o parágrafo em relação aos outros.
- e) Não é possível o uso das tags de negrito (`<b>`) e itálico (`<i>`) no bootstrap.

Comentários:

No Bootstrap, a tag `<p>` recebe uma margem inferior padrão e a classe `.lead` pode ser usada para destacar um parágrafo em relação aos outros, aumentando o tamanho da fonte e adicionando um espaçamento maior entre as linhas. O Bootstrap também oferece classes para ajustar o tamanho da fonte e espaçamento entre linhas para outros elementos de texto, como títulos e listas. Ademais, vejamos os erros das outras alternativas. a) Essa afirmação está incorreta. As classes `mute`, `primary`, `success`, `info`, `warning` e `danger` não são usadas para alinhar o texto, mas sim para definir a cor de fundo do elemento e a cor do texto para destacar diferentes tipos de informações. b) Essa afirmação está parcialmente correta. O Bootstrap permite alterar o tipo da fonte por meio do uso de classes específicas, como por exemplo a classe `font-weight-bold` para definir uma fonte em negrito e a classe `font-italic` para definir uma fonte em itálico. c) Essa afirmação está incorreta. O Bootstrap utiliza as classes `text-left`, `text-right` e `text-center` para alinhar o texto à esquerda, direita ou centralizado, respectivamente. Por fim, a alternativa d) está incorreta. O Bootstrap permite o uso das tags de negrito (`<b>`) e itálico (`<i>`) normalmente, sem restrições.

Gabarito: Letra D

34.(UFRJ –UFRJ – 2018) Considere o trecho de código em uma página HTML a seguir:

```
1. Mensagens <span class="BOOTSTRAP">
  {{mensagens}}</span>
```

Em uma página HTML com bootstrap, versão 4.0, utilizando as definições de estilo (CSS) padrões do framework, para que a tag `span` apresente um componente `badge` em cores de fundo verde e letra branca, deve-se alterar a declaração `BOOTSTRAP` para:

- a) `badge success`

- b) btn badge-success
- c) badge badge-success
- d) btn btn-success
- e) btn success

Comentários:

Para criar um componente badge em Bootstrap, basta adicionar a classe "badge" à tag HTML, e para definir a cor de fundo do badge, pode-se utilizar as classes "badge-primary", "badge-secondary", "badge-success", "badge-danger", "badge-warning", "badge-info" ou "badge-light". Portanto, para apresentar um badge em cores de fundo verde e letra branca, a classe BOOTSTRAP deve ser substituída por "badge badge-success".

Gabarito: Letra C

35.(QUADRIX – CRM PR – 2018) No que se refere às tecnologias de desenvolvimento para web, julgue o item a seguir.

O Bootstrap é uma biblioteca gratuita e de código aberto com enfoque no desenvolvimento de interfaces de usuários em sistemas web.

Comentários:

Correto, pessoal! O Bootstrap é uma biblioteca muito popular e amplamente utilizada para o desenvolvimento de interfaces de usuário em sistemas web. Ele oferece uma ampla gama de componentes, como botões, menus, formulários, grades responsivas e muito mais, que permitem aos desenvolvedores criar interfaces atraentes e funcionais de maneira rápida e fácil. Além disso, o Bootstrap é altamente personalizável e pode ser facilmente estendido para atender às necessidades específicas de cada projeto.

Gabarito: Correto

LISTA DE QUESTÕES

Diversas Bancas

1. (CESPE – STM– 2018) Acerca de Bootstrap e Javascript, julgue o item.

Utilizando-se o Bootstrap, a expressão a seguir insere o ícone  no leiaute. `<span class="glyphicon-search"></span>`

2. (CESPE –STM– 2018) Acerca de Bootstrap e Javascript, julgue o item.

O sistema de grades do Bootstrap baseia-se em um leiaute de 12 colunas.

3. (CEPUERJ – UERJ – 2021) O Bootstrap prevê uma estilização padrão para formulários, bastando que se marque um div container com a classe form-group para cada conjunto de controles do formulário. No Bootstrap também é possível estilizar um formulário em linha, nesse caso, um valor de marcação válido seria:

- a) form-responsive
- b) form-control
- c) form-inline
- d) form-line

4. (IFSP – IF SP– 2022) O carousel é recurso de slideshow da Biblioteca Bootstrap, com o objetivo de mostrar vários conteúdos, como se fosse um carrossel. Selecione a alternativa correta em relação a esse recurso.

- a) As classes carousel-inner, carousel-item e carousel-control são utilizadas com a tag "ol" do HTML, sem perda de funcionalidade.
- b) As classes carousel-inner, carousel-item e carousel-control são utilizadas com a tag "div" do HTML, sem perda de funcionalidade.
- c) As classes carousel-inner e carousel-item são utilizadas com a tag "a" do HTML, enquanto a classe carousel-control é utilizada com a tag "div" do HTML, sem perda de funcionalidade em todos esses casos.
- d) As classes carousel-inner e carousel-item são utilizadas com a tag "div" do HTML, enquanto a classe carousel-control é utilizada com a tag "a" do HTML, sem perda de funcionalidade em todos esses casos.

5. (IFSUL – IF SUL – 2019) O Bootstrap é uma estrutura popular de HTML, CSS e JavaScript para o desenvolvimento de websites responsivos e móveis.

São classes do Bootstrap para estilizar botões

- a) button-success, button-info.
- b) btn , btn-default.
- c) btn-warning, button-danger.
- d) button-link, btn-primary.

6. (FAPEC – UFMS– 2022) O _____ é uma coleção de ferramentas de código aberto para o desenvolvimento de sites e aplicativos web. Inclui modelos de design baseados em HTML e CSS, um sistema de grade responsivo, componentes predefinidos e plug-ins em jQuery. Desenvolvido para uniformizar os componentes do Twitter. Foi liberado como código aberto em 2011.

Assinale a alternativa que completa corretamente a lacuna.

- a) Bootstrap.
- b) Mark text.
- c) Brave Browser.
- d) IconGenerator.
- e) Screencat.

7. (FCC –PGE AM– 2022) Considere o fragmento de código abaixo, disponível em uma página web criada com Bootstrap 5, em condições ideais.

```
<div class="container-fluid">
  <div class="row">
    <div class=" I bg-primary">
      Amazonas
    </div>
    <div class=" II bg-secondary">
      Brasil
    </div>
  </div>
</div>
```

Para que os dois contêineres internos sejam renderizados com divisão de 50% cada, em todas as telas, exceto nas extra pequenas, as lacunas I e II devem ser corretamente preenchidas com

- a) col-left-50 e col-right-50

- b) col-half-screen e col-half-screen
- c) col-sm-6 e col-sm-6
- d) col-lg-50 e col-lg-50
- e) col-sm-5 e col-sm-5

8. (FCC – TRT4– 2022) Para definir uma tag div como contêiner, o Bootstrap tem alguns nomes de classe predefinidos, como

- a) container-flex e container-grid.
- b) container-box e container-text.
- c) container e container-fluid.
- d) container-stretch e container-fixed.
- e) container-relative e container-absolute.

9. (FCC – TRF 22 – 2022) Uma das classes do Bootstrap 5 para definir estilos de botões criados por meio da tag <button> é a classe,

- a) btn-reset.
- b) btn-success.
- c) button-submit.
- d) btn-post.
- e) button-reset.

10.(FCC – TRT 23 – 2022) No Bootstrap 5, para criar um contêiner flexbox e transformar filhos diretos em itens flex, utiliza-se, nesse contêiner, a classe

- a) d-flex
- b) flex-display
- c) flex-layout
- d) flex-model
- e) m-flex

11.(FCC – TRT 5– 2022) Considere o contêiner abaixo, criado em uma página HTML com Bootstrap 5, em condições ideais.

```
<div class="..!...">  
  <h1>TRIBUNAL REGIONAL DO TRABALHO DA 5 REGIÃO</h1>  
  <h2>SALVADOR-BAHIA</h2>  
</div>
```

Para definir que este contêiner será responsivo, de largura fixa e aparecerá centralizado na tela, utiliza-se, na lacuna I, a classe

- a) align-center
- b) flex-box
- c) container
- d) fixed-box
- e) container-fluid

12.(FCC – TRT 14 – 2022) Considere os contêineres abaixo, criados utilizando HTML e Bootstrap 5, em condições ideais.

```
<div class="...I...">Conteúdo 1</div>
```

```
<div class="...II... ">Conteúdo 2</div>
```

Para definir uma divisão de 25% e 75% em dispositivos pequenos, com largura de tela entre 576 pixels e 767 pixels, as lacunas I e II devem ser preenchidas, respectivamente, com

- a) col-small-25 e col-small-75.
- b) col-sm-3 e col-sm-9.
- c) col-25 e col-75.
- d) col-md-3 e col-md-9.
- e) col-lg-3 e col-lg-9.

13.(FCC – TJ SC – 2021) Considere o trecho de uma página web abaixo, que utiliza Bootstrap.

```
<div class="container-fluid">  
  <div class="co">  
    <div class="..I.. bg-success">  
      TRIBUNAL DE JUSTIÇA DO ESTADO DE SANTA CATARINA  
    </div>  
    <div class="... II... bg-warning">  
      PODER JUDICIÁRIO DO ESTADO DE SANTA CATARINA  
    </div>  
  </div>  
</div>
```

Para que, em dispositivos pequenos (largura de tela de 576 pixels a 767 pixels), a divisão da esquerda use 25% da largura da linha e a divisão da direita use 75%, as lacunas I e II devem ser preenchidas, correta e respectivamente, com os nomes de classe

- a) col-sm-3 e col-sm-9
- b) col-sm-25 e col-sm-75
- c) col-small-25% e col-small-75%
- d) col-small-3 e col-small-9
- e) column-small-25 e column-small-75

14.(FCC – SANASA – 2019) Considere o fragmento de uma página web abaixo, desenvolvido com Bootstrap4, onde todas as bibliotecas necessárias foram importadas no cabeçalho da página.

```
<div class="container-fluid">
  <div class="row">
    <div class="col-sm-4" style="background-color:red;">Fornecimento de água</div>
    <div class="col-sm-8" style="background-color:blue;">Tratamento de esgoto</div>
  </div>
</div>
```

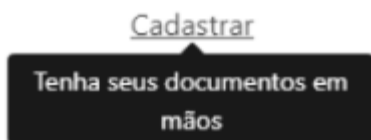
Quando a janela (tela) tiver

- a) menos de 576 pixels de largura, as colunas serão empilhadas automaticamente uma sobre a outra.
- b) mais de 460 pixels de altura, as colunas aparecerão automaticamente uma sobre a outra.
- c) mais de 380 pixels de largura, as colunas aparecerão automaticamente uma ao lado da outra.
- d) menos de 768 pixels de largura, as colunas aparecerão automaticamente uma sobre a outra.
- e) mais de 800 pixels de largura, as colunas serão automaticamente colocadas uma sobre a outra.

15.(FCC – Pref Manaus– 2019) Considere o fragmento de uma página web desenvolvida usando HTML, jQuery e Bootstrap 4. Considere que todas as bibliotecas necessárias foram referenciadas no cabeçalho da página.

```
<body>
  <div class="container">
    <a href="#" data-toggle="a" data-placement="bottom" title="Tenha seus documentos em
    mãos">Cadastrar</a>
  </div>
  <script>
    $(document).ready(function() {
      $('[data-toggle="a"]').I..();
    });
  </script>
</body>
```

Para que, ao levar o ponteiro do mouse sobre o link, apareça o que mostra a figura abaixo, a lacuna I deve ser preenchida por



- a) toggle.
- b) collapse.
- c) pophover.
- d) tooltip.
- e) toast.

16.(FCC – Pref Manaus– 2019) Considere o fragmento de código abaixo, em uma página que utiliza Bootstrap em condições ideais.

```
<div class="container-fluid">  
  <div class="row">  
    <div class="col-sm-3 col-md-6 col-lg-4 col-x1-2">  
      <p>A</p>  
    </div>  
    <div class="col-sm-9 col-md-6 col-lg-8 col-x1-10">  
      <p>B</p>  
    </div>  
  </div>  
</div>
```

Esse fragmento resultará na divisão da largura da tela entre os dois contêineres na proporção de

- a) 20% / 80% em dispositivos de tela grande.
- b) 20% / 40% / 30% / 10% em todos os tipos de dispositivos.
- c) 60% / 40% em dispositivos de tela média.
- d) 25% / 75% em dispositivos de tela pequena.
- e) 33% / 66% em dispositivos de tela extragrande.

17.(FCC – TJ MA – 2019) No desenvolvimento de sites muitas vezes é necessário criar botões como mostrados na figura abaixo, indicando algum procedimento de carregamento no site, em que a imagem de um pequeno círculo fica girando no interior do botão.



Usando Bootstrap 4, em condições ideais, este botão pode ser criado por meio do bloco de código abaixo.

```
<button class="btn btn-primary">
  ..I..
  Carregando..
</button>
```

O código ficará correto se a indicação I for substituída pela instrução:

- a) <spinner class="loading-circle"></spinner>
- b)
- c) <label class="shape-loading"></label>
- d)
- e) <label class="spinner-loader"></label>

18.(FCC – TJ MA – 2019) O Bootstrap 4 usa flexbox em vez de float para manipular o layout da página web. Sendo assim, para criar um contêiner flexbox e transformar contêineres filhos diretos em itens flexíveis (flex), usa-se nesse contêiner a classe:

- a) flex-grow
- b) bg-flex
- c) d-flex
- d) flex-fill
- e) flex-stretch

19.(FCC – Pref Manaus– 2019) Um programador criou a página web abaixo utilizando Bootstrap 4.

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css">
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.7/umd/popper.min.js"></script>
  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js"></script>
</head>
<body>
  <div class="container">
    <button type="button" class="btn btn-primary" ..... data-target="#al">SEMEF</button>
    <div id="al" class="collapse">
      A Secretaria Municipal de Finanças, Tecnologia da Informação e Controle Interno (Semef)
      integra a Administração Direta do Poder Executivo...
    </div>
  </div>
</body>
</html>
```

Para que, ao clicar uma vez no botão, o texto contido no contêiner com id="a1" apareça abaixo do botão e, ao clicar novamente o texto desapareça, alternando a cada clique, a lacuna I deverá ser preenchida com

- a) data-toggle="alternate"
- b) data-alternate="show-hide"
- c) data-toggle="collapse"
- d) data-alternate="true"
- e) data-toggle="show-hide"

20.(FCC –DPE AM– 2018) Considere o fragmento de código Bootstrap a seguir.

```
<div class="container-fluid">  
  <div class="row">  
    <div class="col-sm-3 col-md-6 col-lg-4" style="background-color:yellow;">Texto 1</div>  
    <div class="col-sm-9 col-md-6 col-lg-8" style="background-color:blue;">Texto 2</div>  
  </div>  
</div>
```

Quando a página que contém o fragmento de código acima for executada em condições ideais, o navegador vai renderizar os dois contêineres, um ao lado do outro, na proporção aproximada de

- a) 15%/85% para dispositivos de telas pequenas, 50%/50% para dispositivos de telas médias e 23%/76% para dispositivos de telas grandes.
- b) 25%/75% para dispositivos de telas pequenas, 50%/50% para dispositivos de telas médias e 33%/66% para dispositivos de telas grandes.
- c) 50%/50% para dispositivos de telas grandes, 25%/75% para dispositivos de telas médias e 30%/70% para dispositivos de telas pequenas.
- d) 25%/75% para dispositivos de telas grandes, 50%/50% para dispositivos de telas médias e 33%/66% para dispositivos de telas pequenas.
- e) 10%/90% para dispositivos de telas pequenas, 20%/80% para dispositivos de telas médias e 30%/70% para dispositivos de telas grandes.

21.(FCC –SABESP– 2018) Um Analista está utilizando Bootstrap 4 na criação de um site e deseja definir que um container deve usar toda a largura da tela.

Para isso, terá que utilizar na tag <div> o atributo class igual a

- a) container.
- b) max-width.
- c) stretch.
- d) container-fluid.
- e) full-width.

22.(FCC –TST– 2017) Considere a página abaixo que utiliza Bootstrap em um ambiente de desenvolvimento web ideal.

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="bootstrap.min.css">
    <script src="jquery.min.js"></script>
    <script src="bootstrap.min.js"></script>
  </head>
  <body>
    <button type="button" class="btn btn-info" data-toggle="..I.." data-target="#container">Mais
informações</button>
    <div id="container" class="collapse">
      <p>Aqui deverá estar o conteúdo desejado.</p>
    </div>
  </body>
</html>
```

Para que, ao clicar no botão, o conteúdo do container seja exibido e, ao clicar novamente o conteúdo seja ocultado, alternando a cada clique, a lacuna I deverá ser preenchida com

- a) collapse
- b) toggle
- c) alternate
- d) show-hide
- e) modal-show

23.(FCC –ARTESP– 2017) Um Especialista em Tecnologia da Informação deseja criar um layout com duas colunas utilizando Bootstrap, para dispositivos pequenos, com largura de tela de 768 pixels por 991 pixels. A coluna da esquerda deve ocupar aproximadamente 33,3% e a da direita 66,6% da tela. O Analista utilizará em uma página HTML as instruções abaixo.

```
<div class="...I...">....</div>
<div class="..I..I..">....</div>
```

Para obter o layout desejado, as lacunas I e II devem conter, respectivamente, os valores

- a) col-md-3 e col-md-9
- b) col-lg-33 e col-lg-66
- c) col-sm-4 e col-sm-8
- d) col-small-3 e col-small-7
- e) col-lg-3 e col-lg-9

24.(FCC –TST – 2017) O plugin modal do bootstrap permite gerar uma caixa de diálogo sobre a página atual. O elemento <div> pai do modal deve ter como valor do atributo id o mesmo valor de um atributo do elemento usado para disparar o modal. Este atributo usado para disparar o modal é o

- a) data-toggle.
- b) data-target.
- c) modal-window.
- d) modal-trigger.
- e) modal-target.

25.(FCC –TRF5 – 2017) Considere o código fonte da página abaixo, que faz parte de um site responsivo.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <title>TRF5</title>
    <meta charset="utf-8">
    <link rel="stylesheet" href="bootstrap.min.css">
    <script src="jquery.min.js"></script>
    <script src="bootstrap.min.js"></script>
  </head>
  <body>
    <div class="caixa">
      <ul class="...>
        <li><a href="#">Anterior</a></li>
        <li><a href="#">Próximo</a></li>
      </ul>
    </div>
  </body>
</html>
```

Para que este código crie dois botões para paginação, "Anterior" e "Próximo", um ao lado do outro, utilizando Bootstrap, a lacuna I deve ser corretamente preenchida com.

- a) top-pagination ou bottom-pagination.
- b) back-advance ou history-go.
- c) media-go ou media-pagination.
- d) pager ou pagination.
- e) modal ou collapse.

26.(FEPESE – CELESC – 2022) O sistema de grid Bootstrap permite até quantas colunas?

- a) 16
- b) 12
- c) 9
- d) 8
- e) 6

27.(FEPESE – CELESC – 2022) Qual classe Bootstrap 5 centraliza um texto em uma tag HTML <p>?

- a) align
- b) center
- c) middle
- d) justified
- e) text-center

28.(FEPESE – CELESC – 2022) Qual componente Bootstrap pode ser definido como a classe de um elemento <div>, por exemplo, para implementar um slideshow de modo a alternar entre elementos, sejam imagens ou slides de texto?

- a) list
- b) flip-flop
- c) carousel
- d) slideshow
- e) mosaic

29.(FGV – CGE SC– 2023) A maneira correta de criar um layout de grid responsivo usando o Bootstrap é usar a classe

- a) "row" em um elemento div e adicionar classes "col-*" para criar as colunas.
- b) "grid" em um elemento div e adicionar classes "row-*" para criar as linhas.
- c) "responsive " em um elemento div e adicionar classes "col-*" para criar as colunas.

- d) "flex" em um elemento div e adicionar classes "grid-*" para criar as linhas.
- e) "container" em um elemento div e adicionar as classes "row" e "col-*" para criar o layout.

30.(FGV –Sefaz AM – 2022) O Bootstrap é um framework web com código-fonte aberto amplamente utilizado em desenvolvimento de aplicações web.

Analise o código da página web a seguir.

```
1 <!doctype html>
2 <html>
3
4   <head>
5     <link href="/bootstrap.min.css"
6         rel="stylesheet">
7   </head>
8
9   <body>
10    <p>NOME</p>
11  </body>
12
13 </html>
```

Para alinhar o texto NOME à direita utilizando Bootstrap, versão 5, é suficiente utilizar na tag <p> o atributo class igual a

- a) text-end.
- b) text-muted.
- c) float-right.
- d) align-items-end.
- e) align-self-stretch.

31.(FGV –TRT16 – 2022) Observe a declaração abaixo no contexto de páginas Web.

```
<div class="jumbotron">
  <h1>Teste</h1>
  <p>Frase inicial...</p>
</div>
```

Assinale o framework usualmente associado à classe utilizada.

- a) Angular Material.
- b) Bootstrap.
- c) CSS.
- d) JSON.
- e) SpringBoot.

32.(Instituto AOCP – PRODEB – 2018) É possível considerar o Bootstrap como um kit de desenvolvimento básico que é composto por uma gama de componentes prontos que auxiliam no desenvolvimento de aplicações web mobile de forma simples e clara, tirando a necessidade de um conhecimento profundo em JavaScript e CSS. O que é possível fazer com Bootstrap?

- a) Todo o desenvolvimento web mobile que não tem ligação com html.
- b) Desenho de telas para serem acessadas em um navegador web ou dispositivo mobile.
- c) Validação de processo de campos de entradas de dados.
- d) Toda tarefa que não seja relacionada com o desenho de uma tela.
- e) Calcular datas a partir de campos inputs.

33.(Instituto AOCP – PRODEB – 2018) A tipografia é muito importante em qualquer trabalho gráfico, seja em um ambiente físico ou até mesmo virtual. Ela tem muita influência no peso da informação e na forma como os usuários perceberão as informações que o conteúdo pode e deve transmitir. Assinale a alternativa correta em relação ao uso de recursos de tipografia no Bootstrap.

- a) As classes do Bootstrap mute, primary, success, info, warning e danger são usadas para se trabalhar com o alinhamento do texto.
- b) Não é possível fazer a alteração do tipo da fonte no Bootstrap.
- c) É possível alinhar um texto utilizando as classes left-text, right-text e center-text.
- d) A tag de parágrafo <p> recebe uma margem inferior, além de uma classe especial chamada lead que destaca melhor o parágrafo em relação aos outros.
- e) Não é possível o uso das tags de negrito () e itálico (<i>) no bootstrap.

34.(UFRJ –UFRJ – 2018) Considere o trecho de código em uma página HTML a seguir:

```
1. Mensagens <span class="BOOTSTRAP">
{{mensagens}}</span>
```

Em uma página HTML com bootstrap, versão 4.0, utilizando as definições de estilo (CSS) padrões do framework, para que a tag span apresente um componente badge em cores de fundo verde e letra branca, deve-se alterar a declaração BOOTSTRAP para:

- a) badge success
- b) btn badge-success
- c) badge badge-success
- d) btn btn-success
- e) btn success

35. (QUADRIX – CRM PR – 2018) No que se refere às tecnologias de desenvolvimento para web, julgue o item a seguir.

O Bootstrap é uma biblioteca gratuita e de código aberto com enfoque no desenvolvimento de interfaces de usuários em sistemas web.

GABARITO

- | | | |
|-------------|-------------|-------------|
| 1. ERRADO | 13. LETRA A | 25. LETRA D |
| 2. CERTO | 14. LETRA A | 26. LETRA B |
| 3. LETRA C | 15. LETRA D | 27. LETRA E |
| 4. LETRA D | 16. LETRA D | 28. LETRA C |
| 5. ANULADA | 17. LETRA D | 29. LETRA A |
| 6. LETRA A | 18. LETRA C | 30. LETRA A |
| 7. LETRA C | 19. LETRA C | 31. LETRA B |
| 8. LETRA C | 20. LETRA B | 32. LETRA B |
| 9. LETRA B | 21. LETRA D | 33. LETRA D |
| 10. LETRA A | 22. LETRA A | 34. LETRA C |
| 11. LETRA C | 23. LETRA C | 35. CORRETO |
| 12. LETRA B | 24. LETRA B | |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.